
RT-Comops Backend

Cahier de conception

Architecture Multi-Tenant — ConceptionKernel V3

Cahier de conception .

Table des matières

Introduction Générale	1
1 kernel-core	6
1.1 Description du core	6
1.2 Rôle principal	6
1.3 Rôles secondaires	6
Diagramme de classes — kernel-core	8
Diagramme des cas d'utilisation — kernel-core	10
1.3.1 DS-K-01 : Provisionnement d'un nouveau tenant	11
DS-K-01 — Provisionnement d'un nouveau tenant	12
DS-K-02 — Relais Outbox vers Kafka	13
DS-K-03 — Routage réactif vers le schéma tenant	14
DS-K-04 — Écriture de la piste d'audit (pattern aspect)	15
DS-K-05 — Révocation JWT et inscription en liste noire	16
DS-K-06 — Validation JWT avec contrôle de la liste noire	17
DS-K-07 — Suspension d'un tenant	18
DS-K-08 — Supervision de l'outbox via health check	19
2 common-core	20
2.1 Description du core	20
2.2 Rôle principal	20
2.3 Rôles secondaires	20
Diagramme de classes — common-core	22
Diagramme des cas d'utilisation — common-core	24
DS-C-01 — Attachement d'une adresse polymorphique	25
2.3.1 DS-C-02 : Résolution d'une PartyRef	26
DS-C-02 — Résolution d'une PartyRef inter-cores	27
DS-C-03 — Pagination standardisée des résultats	28
DS-C-04 — Propagation automatique des champs BaseEntity	29
DS-C-05 — Gestion du contact principal (promotion/rétrogradation)	30
DS-C-06 — Construction d'une réponse d'erreur standardisée	31
3 actor-core	32
3.1 Description du core	32
3.2 Rôle principal	32
3.3 Rôles secondaires	32

Diagramme de classes — actor-core	34
Diagramme des cas d'utilisation — actor-core	36
DS-A-01 — Création d'un acteur	37
DS-A-02 — Suppression douce d'un acteur	38
DS-A-03 — Association acteur-organisation (relation d'emploi)	39
DS-A-04 — Qualification d'un acteur comme commercial (SalesPerson)	40
DS-A-05 — Mise à jour de l'encours d'un commercial	41
DS-A-06 — Approbation d'un BusinessActor et déclenchement du provisionnement	42
4 auth-core	43
4.1 Description du core	43
4.2 Rôle principal	43
4.3 Rôles secondaires	43
Diagramme de classes — auth-core	45
Diagramme des cas d'utilisation — auth-core	47
DS-AU-01 — Inscription et création de compte	48
DS-AU-02 — Authentification locale et émission de JWT	49
DS-AU-03 — Rotation du token de rafraîchissement	50
DS-AU-04 — Connexion via fournisseur OAuth2 externe	51
DS-AU-05 — Suspension d'un compte utilisateur	52
DS-AU-06 — Consultation du profil utilisateur connecté (/users/me)	53
5 roles-core	54
5.1 Description du core	54
5.2 Rôle principal	54
5.3 Rôles secondaires	54
Diagramme de classes — roles-core	56
Diagramme des cas d'utilisation — roles-core	58
DS-R-01 — Création d'un rôle avec ses permissions	59
DS-R-02 — Affectation d'un rôle à un utilisateur	60
DS-R-03 — Résolution des permissions avec cache Redis	61
DS-R-04 — Clone d'un rôle existant	62
DS-R-05 — Révocation d'une affectation de rôle	63
DS-R-06 — Instanciation des rôles par défaut lors du provisionnement d'un tenant	64
6 organization-core	65
6.1 Description du core	65
6.2 Rôle principal	65
6.3 Rôles secondaires	65
Diagramme de classes — organization-core	67
Diagramme des cas d'utilisation — organization-core	69
DS-O-01 — Création d'une organisation avec agence siège automatique	70

DS-O-02 — Création d'une agence opérationnelle	71
DS-O-03 — Mise à jour des horaires d'ouverture d'une agence	72
DS-O-04 — Lien fonctionnel entre un acteur et une organisation	73
DS-O-05 — Suspension d'une organisation et de ses agences	74
DS-O-06 — Ajout d'un point d'intérêt géographique à une agence	75
7 tp-core — Gestion des Tiers	76
7.1 Description du core	76
7.2 Rôle principal	76
7.3 Rôles secondaires	76
Diagramme de classes — tp-core	78
Diagramme des cas d'utilisation — tp-core	80
DS-TP-01 — Création d'un tiers avec rôle client	81
DS-TP-02 — Progression du stade commercial d'un prospect	82
DS-TP-03 — Conversion d'un prospect en client	83
DS-TP-04 — Enregistrement d'une interaction commerciale	84
DS-TP-05 — Ajout d'un compte bancaire à un tiers	85
DS-TP-06 — Recherche plein-texte de tiers (Elasticsearch / fallback PostgreSQL)	86
8 settings-core	87
8.1 Description du core	87
8.2 Rôle principal	87
8.3 Rôles secondaires	87
Diagramme de classes — settings-core	89
Diagramme des cas d'utilisation — settings-core	91
DS-S-01 — Consommation atomique d'une séquence documentaire	92
DS-S-02 — Résolution des paramètres par héritage organisation-agence	93
DS-S-03 — Configuration des paramètres d'une organisation	94
DS-S-04 — Alerte d'épuisement de séquence documentaire	95
DS-S-05 — Réinitialisation de séquence pour un nouvel exercice fiscal	96
DS-S-06 — Override de paramètres au niveau agence	97
9 product-core — Catalogue Produits	98
9.1 Description du core	98
9.2 Rôle principal	98
9.3 Rôles secondaires	98
Diagramme de classes — product-core	100
Diagramme des cas d'utilisation — product-core	102
DS-P-01 — Création d'un produit avec variante par défaut	103
DS-P-02 — Définition d'un prix pour une variante	104
DS-P-03 — Création d'un lot de fabrication	105
DS-P-04 — Résolution du prix effectif d'une variante à une date	106
DS-P-05 — Mise à jour de l'index Elasticsearch après modification produit	107

DS-P-06 — Ajout d’une traduction multi-lingue à un produit	108
10 inventory-core — Gestion des Stocks	109
10.1 Description du core	109
10.2 Rôle principal	109
10.3 Rôles secondaires	109
Diagramme de classes — inventory-core	111
Diagramme des cas d’utilisation — inventory-core	113
DS-I-01 — Enregistrement d’une entrée de stock (réception fournisseur)	114
DS-I-02 — Allocation de stock à la confirmation de commande	115
DS-I-03 — Validation d’un inventaire physique avec ajustements	116
DS-I-04 — Transformation de produits (exécution d’une recette)	117
DS-I-05 — Transfert de stock entre deux agences	118
DS-I-06 — Alerte de stock bas après mouvement	119
11 resource-core — Ressources Matérielles	120
11.1 Description du core	120
11.2 Rôle principal	120
11.3 Rôles secondaires	120
Diagramme de classes — resource-core	122
Diagramme des cas d’utilisation — resource-core	124
DS-RE-01 — Enregistrement d’une ressource matérielle	125
DS-RE-02 — Affectation d’une ressource à un acteur	126
DS-RE-03 — Envoi d’une ressource en maintenance	127
DS-RE-04 — Retour de maintenance vers l’état opérationnel	128
DS-RE-05 — Mise à jour de la localisation GPS d’une ressource	129
DS-RE-06 — Mise au rebut irréversible d’une ressource	130
12 sales-core — Commandes de Vente	131
12.1 Description du core	131
12.2 Rôle principal	131
12.3 Rôles secondaires	131
Diagramme de classes — sales-core	133
Diagramme des cas d’utilisation — sales-core	135
DS-SA-01 — Création d’une commande en brouillon	136
DS-SA-02 — Confirmation de commande avec vérification de stock	137
DS-SA-03 — Annulation d’une commande avec libération du stock alloué	138
DS-SA-04 — Marquage prête à facturer et déclenchement de la facturation	139
DS-SA-05 — Ajout d’une ligne article à une commande en brouillon	140
DS-SA-06 — Consultation paginée et filtrée du carnet de commandes	141
13 accounting-core — Comptabilité	142
13.1 Description du core	142
13.2 Rôle principal	142
13.3 Rôles secondaires	142

Diagramme de classes — accounting-core	144
Diagramme des cas d'utilisation — accounting-core	146
DS-AC-01 — Génération automatique d'une facture depuis un événement commande	147
DS-AC-02 — Validation (post) d'une écriture comptable en partie double	148
DS-AC-03 — Extourne d'une écriture comptable validée	149
DS-AC-04 — Règlement partiel ou total d'une facture	150
DS-AC-05 — Validation comptable d'une facture avec génération d'écritures	151
DS-AC-06 — Consultation du solde d'un compte sur une période	152
14 treasury-core — Trésorerie	153
14.1 Description du core	153
14.2 Rôle principal	153
14.3 Rôles secondaires	153
Diagramme de classes — treasury-core	155
Diagramme des cas d'utilisation — treasury-core	157
DS-TR-01 — Enregistrement d'un compte bancaire	158
DS-TR-02 — Création automatique d'une transaction bancaire depuis un règlement	159
DS-TR-03 — Import d'un relevé bancaire	160
DS-TR-04 — Appariement d'une transaction bancaire à une ligne de relevé	161
DS-TR-05 — Émission d'un chèque avec création de la transaction en attente	162
DS-TR-06 — Clôture définitive d'un rapprochement bancaire	163
15 administration-core — Gouvernance et Administration	164
15.1 Description du core	164
15.2 Rôle principal	164
15.3 Rôles secondaires	164
Diagramme de classes — administration-core	166
Diagramme des cas d'utilisation — administration-core	168
DS-AD-01 — Approbation d'un BusinessActor et provisionnement du tenant	169
DS-AD-02 — Mise à jour des options de plateforme avec audit	170
DS-AD-03 — Consultation filtrée de la piste d'audit	171
DS-AD-04 — Rejet d'une demande de gouvernance avec notification	172
DS-AD-05 — Supervision de l'état du système via le port de management	173
DS-AD-06 — Approbation d'une organisation soumise à validation	174
16 file-core — Gestion des Fichiers	175
16.1 Description du core	175
16.2 Rôle principal	175
16.3 Rôles secondaires	175
Diagramme de classes — file-core	177

Diagramme des cas d'utilisation — file-core	179
DS-F-01 — Téléversement d'un fichier avec validation MIME et checksum	180
DS-F-02 — Téléchargement d'un fichier (direct ou URL signée)	181
DS-F-03 — Association d'un fichier à une entité métier	182
DS-F-04 — Suppression sécurisée d'un fichier avec vérification des références	183
DS-F-05 — Listage des fichiers associés à une entité	184
DS-F-06 — Vérification périodique d'intégrité des fichiers stockés . . .	185
Conclusion Générale	186

Introduction Générale

Objet du document

Ce cahier d'analyse décrit l'architecture technique et fonctionnelle de la plateforme **RT-Comops Backend** (référence interne : IWM — *Internal Workflow Management*). Il constitue le document de référence pour la conception, le développement et l'évolution du système.

Pour chaque module fonctionnel — appelé *core* dans ce document — une analyse complète est fournie, comprenant : la description détaillée, les rôles primaire et secondaires, le diagramme de classes, le diagramme des cas d'utilisation, ainsi qu'un ensemble de diagrammes de séquences couvrant les interactions techniques les plus significatives.

Périmètre fonctionnel

La plateforme vise à couvrir l'intégralité du cycle opérationnel d'une entreprise, articulé autour des domaines suivants :

- **Gestion des identités** — acteurs, comptes utilisateurs, rôles et permissions
- **Structure organisationnelle** — organisations, agences, tiers, paramétrage
- **Catalogue et stocks** — produits, variantes, lots, mouvements de stock, transformations
- **Ressources matérielles** — équipements, véhicules, affectations
- **Commercial** — commandes de vente, cycle de facturation
- **Finance** — comptabilité générale, trésorerie, rapprochement bancaire
- **Transverse** — multi-tenancy, audit immuable, publication d'événements, fichiers

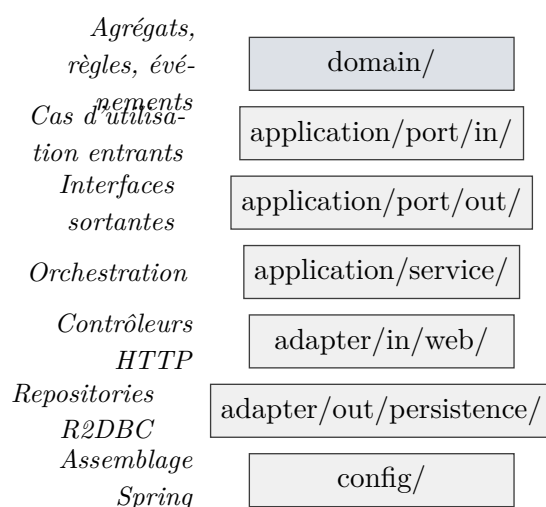
Vision architecturale

Monolithe modulaire hexagonal

Le système est conçu comme un **monolithe modulaire** à architecture hexagonale, organisé en seize cores fonctionnels plus un module de bootstrap. Cette approche garantit :

- une isolation stricte des domaines métier sans couplage fort,
- une testabilité maximale à chaque niveau,
- une extractibilité vers des microservices sans refactorisation du modèle domaine.

Chaque core suit la structure interne suivante :



Multi-tenancy à quatre niveaux

Le système gère le cloisonnement des données selon quatre niveaux de portée :

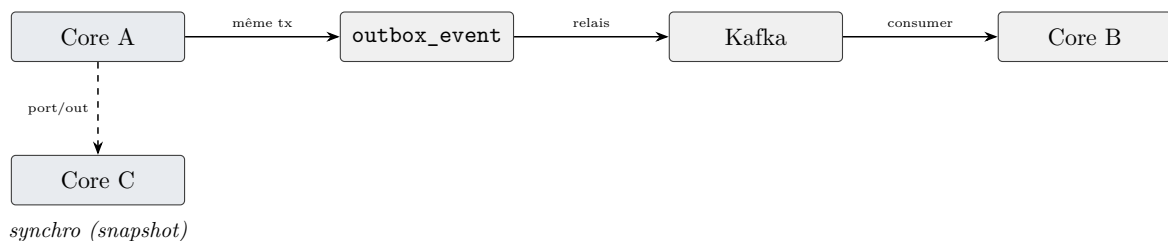
Niveau	Rôle	Isolation	Portée
SYSTEM	Administrateur plateforme	Schéma partagé	Tous les tenants
TENANT	Administrateur tenant	Schéma PostgreSQL dédié	Un tenant
ORGANIZATION	Unité métier	Logique dans le tenant	Une organisation
AGENCY	Unité opérationnelle	Logique dans l'organisation	Une agence

Chaque tenant dispose de son propre schéma PostgreSQL. Le routage de connexion est assuré par le composant **TenantRoutingConnectionFactory**, alimenté par le **TenantWebFilter** qui injecte l'identifiant tenant dans le contexte réactif Reactor à chaque requête.

Communication inter-cores

Les cores communiquent selon deux modalités complémentaires :

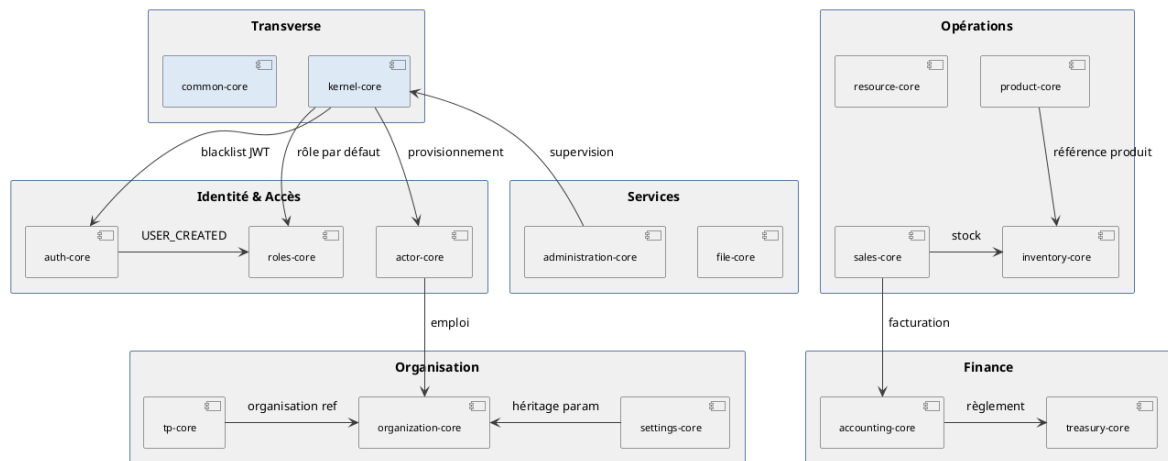
1. **Ports et adaptateurs (synchrone)** — un core définit une interface sortante (*port/out*), implémentée par un adaptateur du core cible. Le couplage est limité à des identifiants et des instantanés (*snapshots*) dénormalisés, jamais à des objets domaine directs.
2. **Outbox transactionnel + Kafka (asynchrone)** — lors d’une opération, l’événement domaine est écrit dans la table `kernel.outbox_event` au sein de la même transaction que l’agrégat. Un relais de fond le publie ensuite vers Kafka (`iwm.events.business`). Les cores abonnés consomment et mettent à jour leurs projections locales.



Pile technologique

Composant	Version	Rôle
Spring Boot / WebFlux	3.5.9	Serveur réactif HTTP
Java	21	Langage
PostgreSQL	16	Base relationnelle principale
R2DBC	—	Driver réactif PostgreSQL
Liquibase	—	Migrations de schéma
Apache Kafka	7.6.1	Bus d'événements
Redis	7	Cache permissions, verrous
Elasticsearch	8.15.3	Projections de recherche
Prometheus + Grafana	—	Métriques et supervision
Nginx	1.27	Passerelle HTTP, rate limiting

Cartographie des cores



Conventions du document

Diagrammes de classes

- Les classes concrètes sont représentées par un rectangle à trois compartiments (nom, attributs, opérations).
- Les interfaces sont indiquées en tirets avec le stéréotype «interface».
- Les classes abstraites apparaissent en italique.
- La multiplicité est notée près des extrémités de relation.

Diagrammes de séquences

- Les lignes de vie verticales représentent les composants du système.
- Les messages synchrones sont représentés par une flèche pleine.
- Les messages asynchrones (événements) sont représentés par une flèche en tirets.
- Les fragments combinés (`alt`, `opt`, `loop`) délimitent les blocs conditionnels.

Diagrammes de cas d'utilisation

- Les acteurs humains sont représentés par un pictogramme.
- Les systèmes ou acteurs externes sont représentés par un rectangle.
- Les relations «include» et «extend» suivent la notation UML standard.

1 kernel-core

1.1 Description du core

Le **kernel-core** constitue la colonne vertébrale technique de la plateforme. Il ne contient aucune logique métier fonctionnelle au sens applicatif, mais fournit l'ensemble des mécanismes transverses sur lesquels s'appuient tous les autres cores : gestion du cycle de vie des tenants, traçabilité immuable, publication fiable d'événements via le patron *outbox transactionnel*, et sécurité des jetons d'accès.

Toute requête traitée par le système transite nécessairement par les composants de ce core (filtres, contexte réactif, routage de connexion), ce qui en fait le seul point d'entrée garanti pour les préoccupations d'infrastructure partagée.

1.2 Rôle principal

Rôle principal : Garantir l'isolation stricte des tenants, la traçabilité immuable de toutes les opérations, et la publication transactionnellement sûre des événements domaine vers le bus de messages, sans introduire de couplage fonctionnel entre les cores métier.

1.3 Rôles secondaires

1. Provisionnement des tenants

Création et activation des schémas PostgreSQL, exécution des migrations Liquibase dans le schéma du nouveau tenant, et enregistrement dans le registre central (`TenantRegistry`).

2. Routage réactif des connexions

Le composant `TenantRoutingConnectionFactory` lit l'identifiant tenant depuis le contexte Reactor et dirige chaque opération R2DBC vers le schéma PostgreSQL approprié, sans configuration explicite dans le code applicatif.

3. Filtrage et propagation du contexte tenant

Le `TenantWebFilter` intercepte chaque requête HTTP, valide le header `X-Tenant-Id`, et injecte l'identifiant tenant dans le contexte Reactor pour toute la chaîne réactive.

4. Outbox transactionnel

Toute écriture d'événement domaine est stockée dans `kernel.outbox_event` dans la même transaction que l'agrégat concerné. Un planificateur asynchrone (`OutboxRelayScheduler`) publie ensuite ces événements vers Kafka, garantissant la cohérence au moins une fois (*at-least-once delivery*).

5. Piste d'audit immuable

L'entité `AuditTrail` est une table en insertion seule : aucune mise à jour ni suppression n'est autorisée. Elle enregistre chaque action significative avec l'acteur, l'heure, l'adresse IP et le différentiel de données.

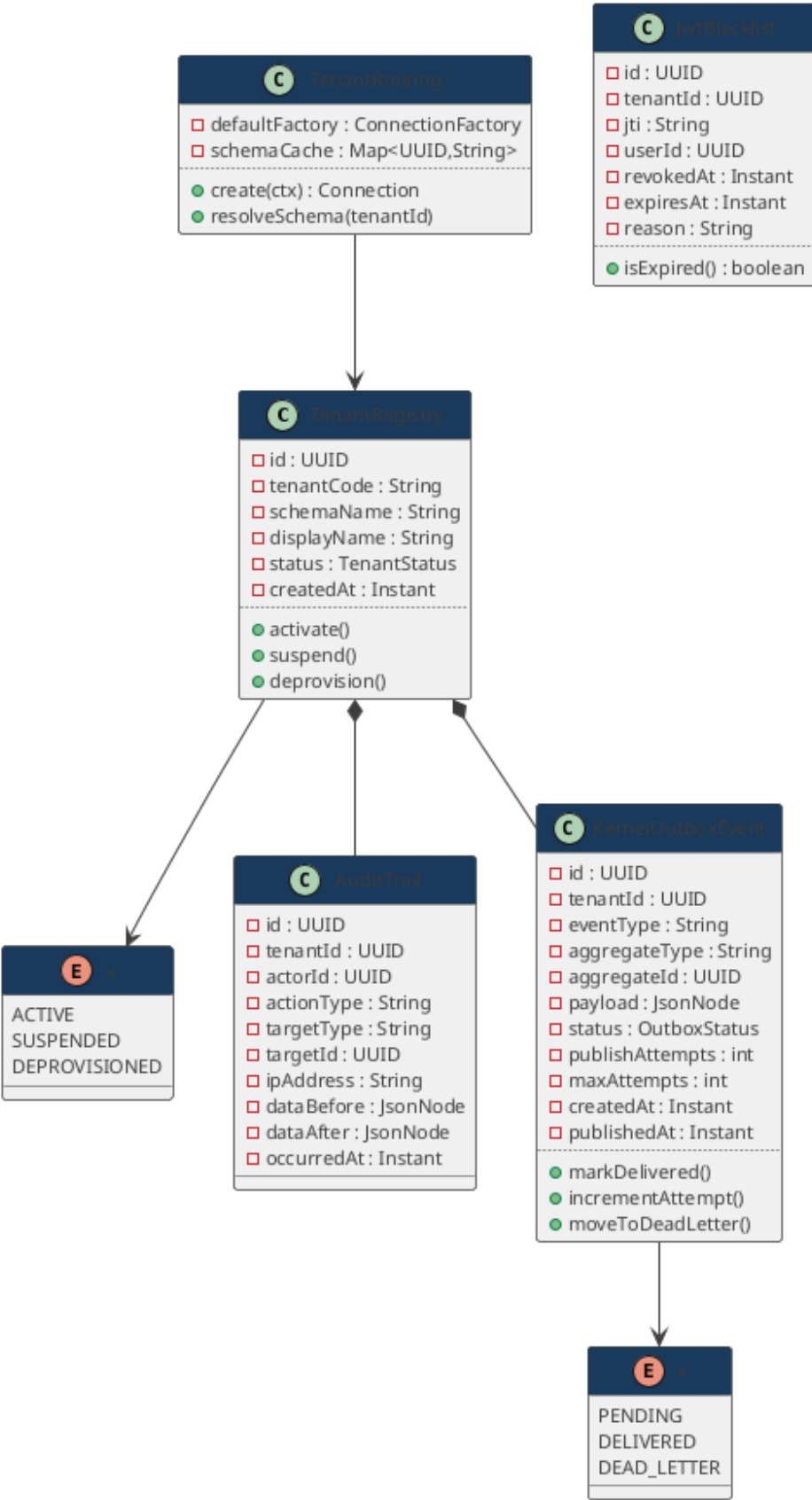
6. Gestion de la liste noire JWT

Les jetons révoqués (déconnexion, rotation forcée) sont inscrits dans `JwtBlacklist` et vérifiés systématiquement lors de chaque validation de token avant son expiration naturelle.

7. Supervision et observabilité

Exposition des métriques Micrometer / Prometheus relatives à l'état de l'outbox (nombre d'événements en attente, âge du plus ancien, dead-letter), et des health checks sur le cycle de vie des tenants.

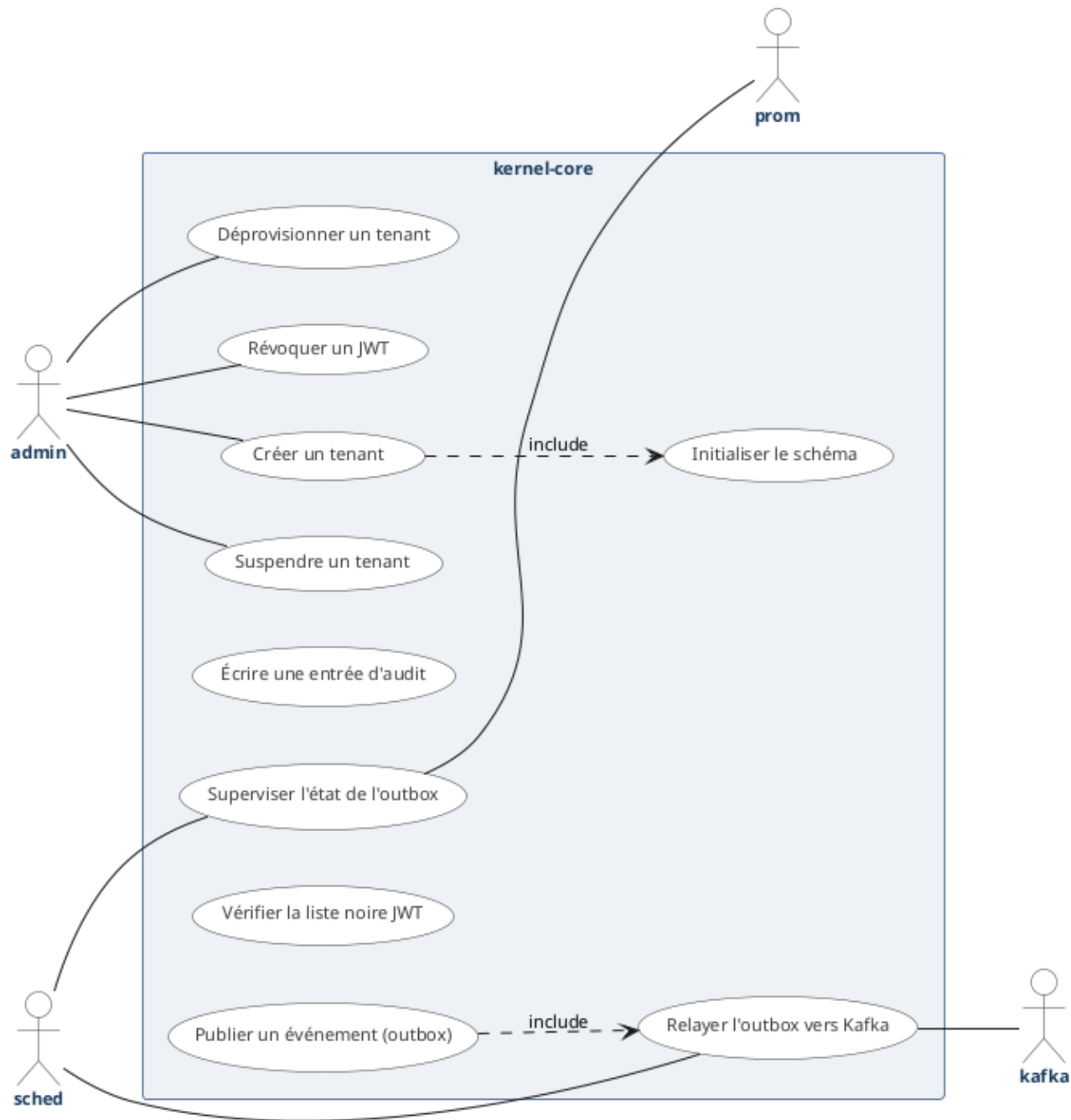
Diagramme de classes — kernel-core



Invariants métier du kernel-core :

- `tenantCode` est globalement unique (toutes bases confondues).
- `schemaName` est globalement unique.
- `status` ne peut régresser de `DEPROVISIONED` vers aucun autre état.
- `AuditTrail` est en insertion seule — aucune mise à jour ni suppression.
- `publishAttempts` ≥ 0 et `maxAttempts` > 0 .
- Un événement `DEAD_LETTER` n'est jamais re-tenté automatiquement.

Diagramme des cas d'utilisation — kernel-core



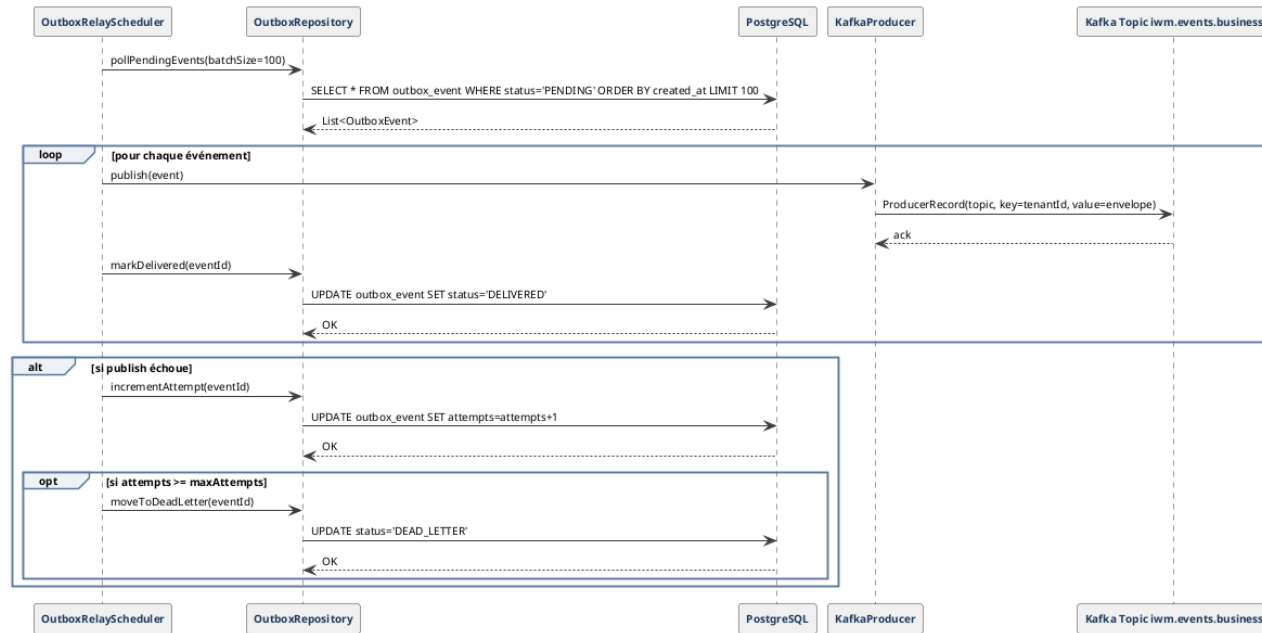
1.3.1 DS-K-01 : Provisionnement d'un nouveau tenant

Ce diagramme décrit la séquence complète d'activation d'un nouveau tenant, depuis la validation du **BusinessActor** jusqu'à la disponibilité opérationnelle du schéma.

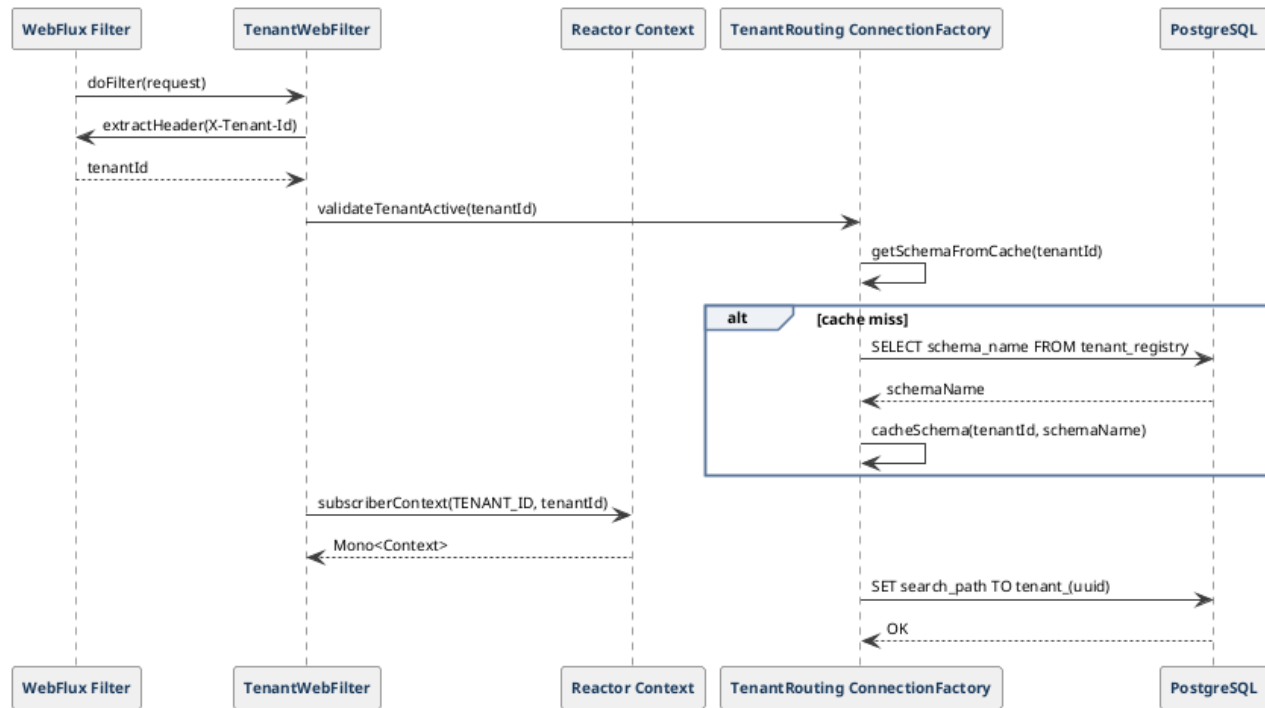
DS-K-01 — Provisionnement d'un nouveau tenant



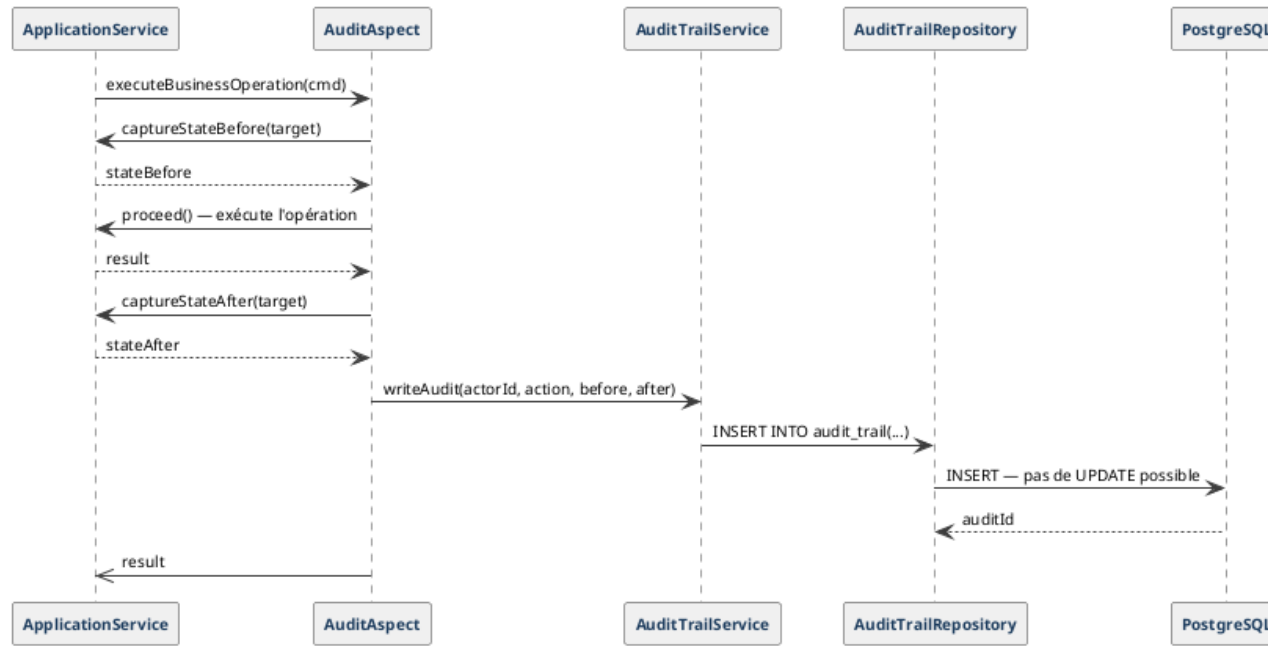
DS-K-02 — Relais Outbox vers Kafka



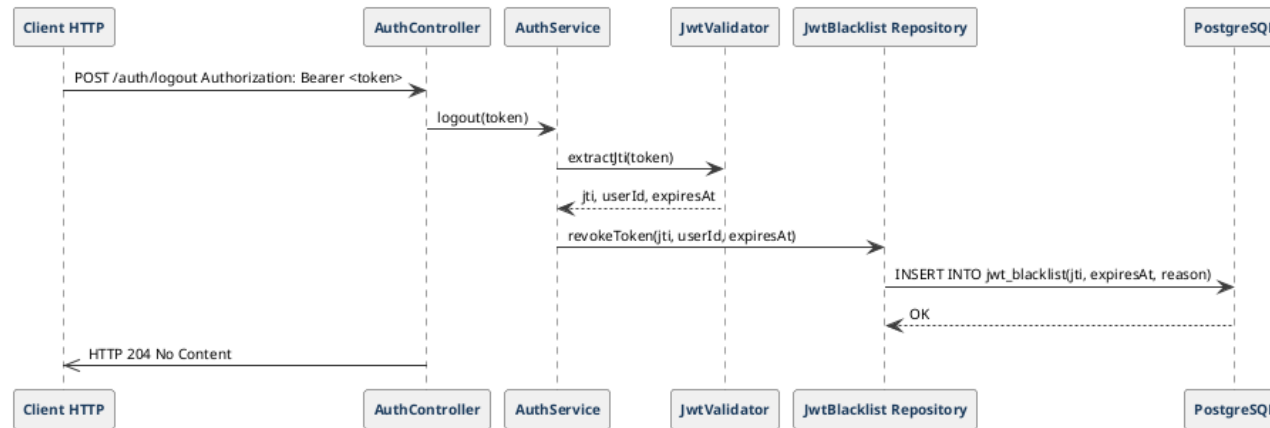
DS-K-03 — Routage réactif vers le schéma tenant



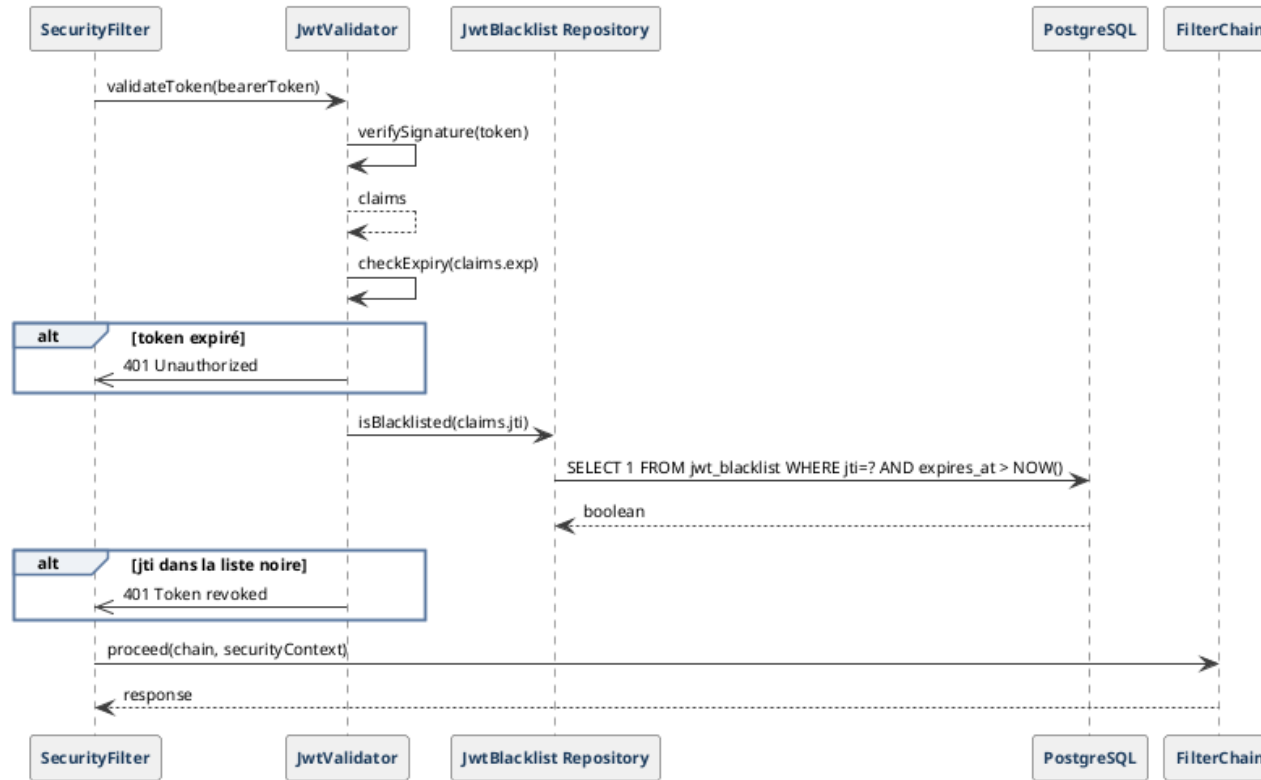
DS-K-04 — Écriture de la piste d'audit (pattern aspect)



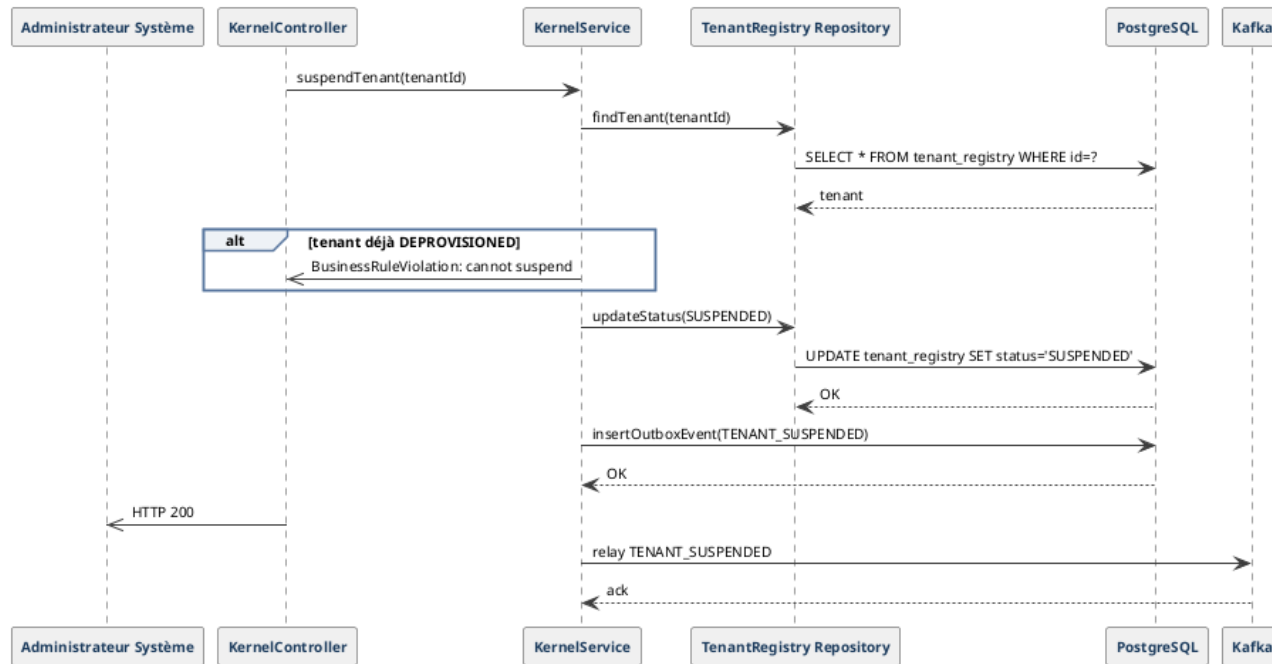
DS-K-05 — Révocation JWT et inscription en liste noire



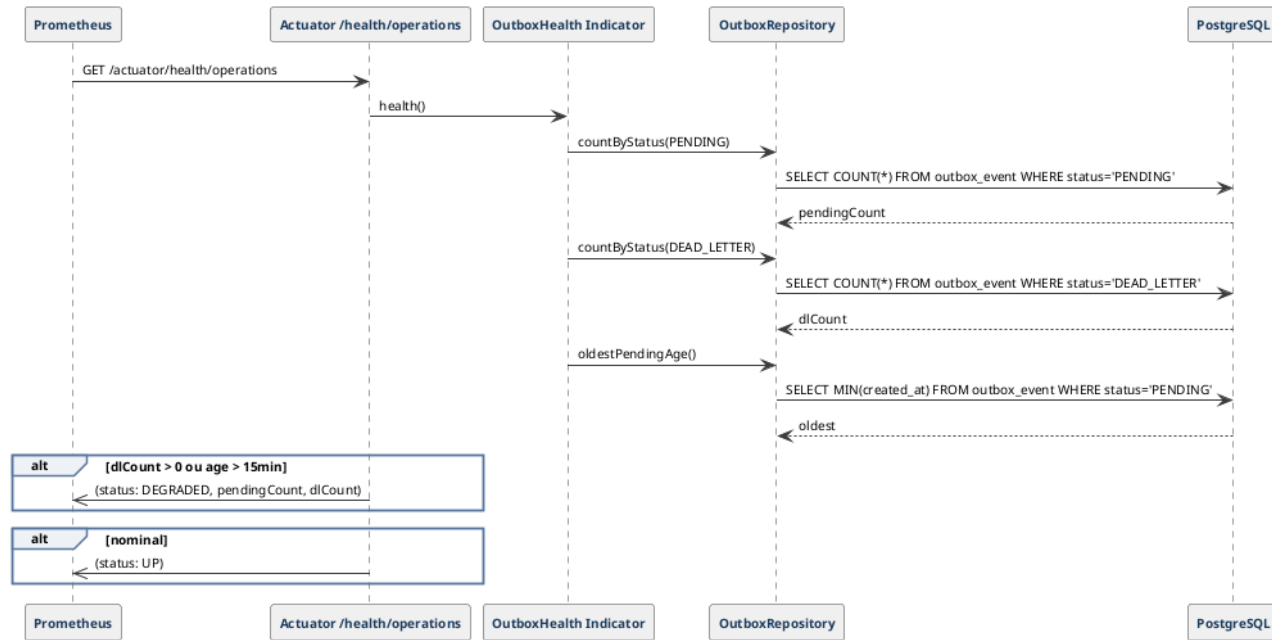
DS-K-06 — Validation JWT avec contrôle de la liste noire



DS-K-07 — Suspension d'un tenant



DS-K-08 — Supervision de l'outbox via health check



2 common-core

2.1 Description du core

Le **common-core** est la bibliothèque de types partagés de la plateforme. Il ne contient aucune logique applicative autonome, mais définit les contrats et abstractions qu'utilisent tous les autres cores pour maintenir une cohérence structurelle. On y trouve les entités de base, les types polymorphiques transverses (adresses, contacts), les références légères inter-cores, ainsi que les enveloppes standard de réponse API.

Ce core ne possède pas de contrôleur HTTP propre, ne publie pas d'événements domaine, et n'abonne aucun consommateur Kafka. Il est exclusivement un module de types partagés inclus comme dépendance de compilation par chaque core métier.

2.2 Rôle principal

Rôle principal : Fournir un socle commun de types, d'abstractions de persistance et de contrats d'interface qui garantissent la cohérence structurelle entre tous les cores, sans créer de couplage fonctionnel ni de dépendances circulaires.

2.3 Rôles secondaires

1. **Entité de base pour la persistance (`BaseEntity`)**

Classe abstraite dont héritent tous les agrégats persistants. Elle consolide les champs techniques obligatoires : identifiant UUID, identifiant tenant, horodatages de création et de modification, et références aux acteurs ayant réalisé ces opérations.

2. **Adresses polymorphiques (`Address`)**

Modèle d'adresse réutilisable qui s'attache à n'importe quel type de cible (`ACTOR`, `ORGANIZATION`, `AGENCY`, `THIRD_PARTY`, `RESOURCE`) avec support de plusieurs types (`BILLING`, `DELIVERY`, `HEADQUARTERS`).

3. **Contacts polymorphiques (`Contact`)**

Informations de contact (téléphone, e-mail, site web, fax) attachables aux mêmes cibles que les adresses, avec notion de contact primaire.

4. Référence légère inter-cores (PartyRef)

Pointeur minimal composé de deux champs (`partyType` et `partyId`) permettant à un core de référencer un acteur ou une organisation d'un autre core sans créer de jointure ni de dépendance forte.

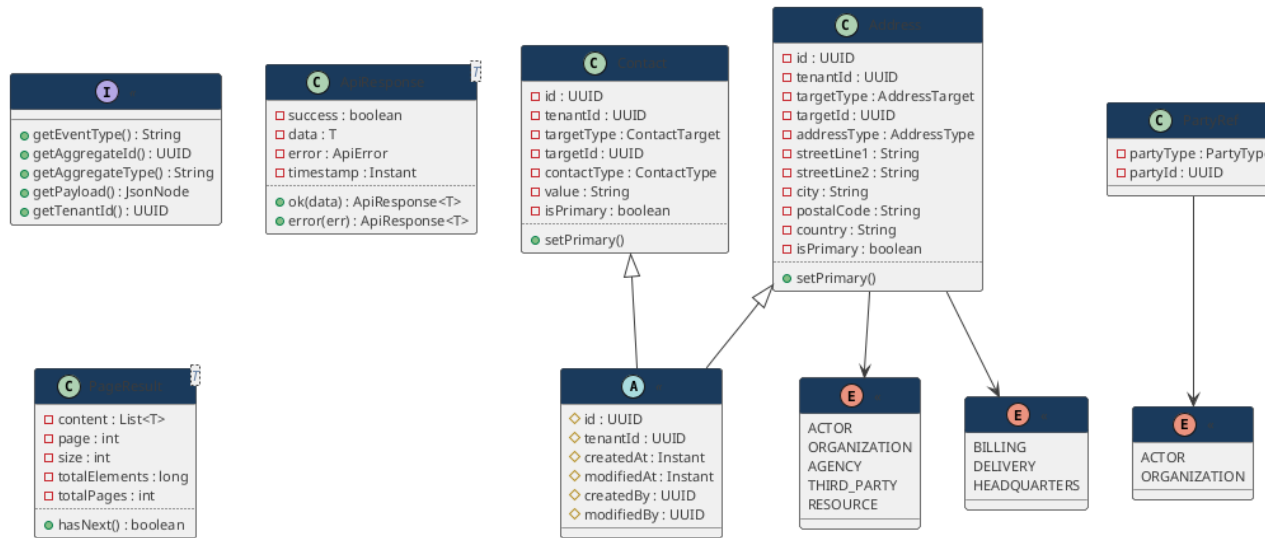
5. Contrat d'événement outbox (OutboxEventRecord)

Interface décrivant la structure qu'un core doit implémenter pour émettre un événement vers le relais kernel. Garantit l'homogénéité de l'enveloppe événementielle.

6. Enveloppes de réponse API

Types génériques `ApiResponse<T>` et `PageResult<T>` assurant un format de réponse JSON uniforme sur l'ensemble de l'API publique.

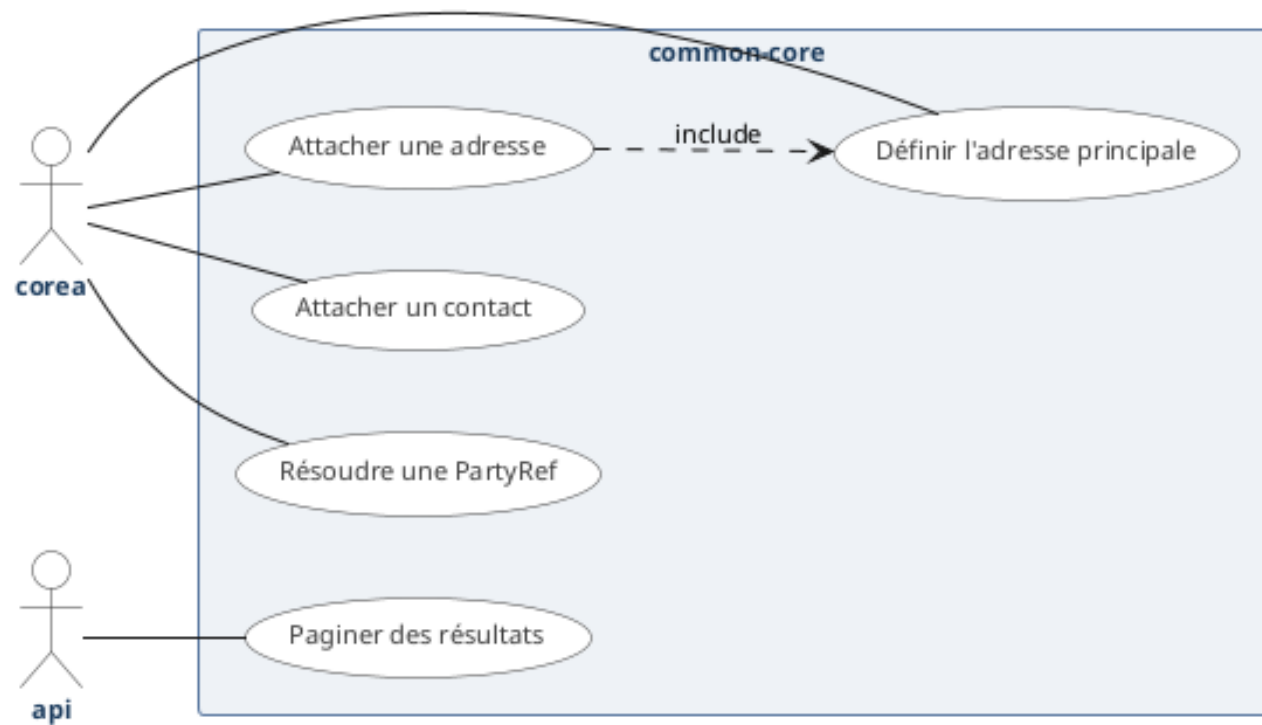
Diagramme de classes — common-core



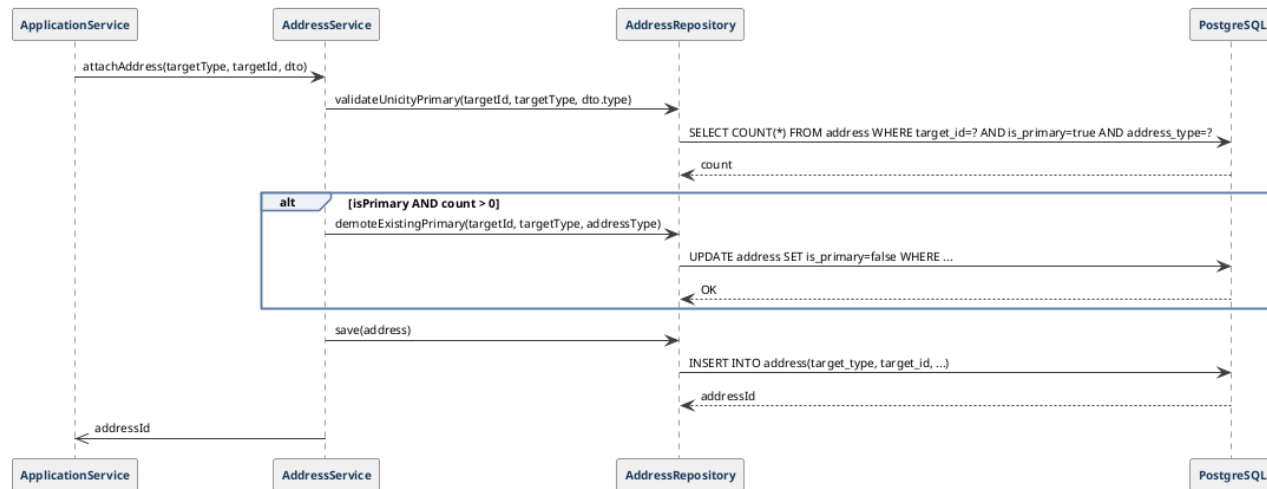
Invariants du common-core :

- Une seule adresse primaire par combinaison (targetId, targetType, addressType).
- PartyRef.partyType est limité aux valeurs ACTOR et ORGANIZATION.
- tenantId est obligatoire sur toutes les tables polymorphiques.
- BaseEntity ne contient aucune logique métier.

Diagramme des cas d'utilisation — common-core



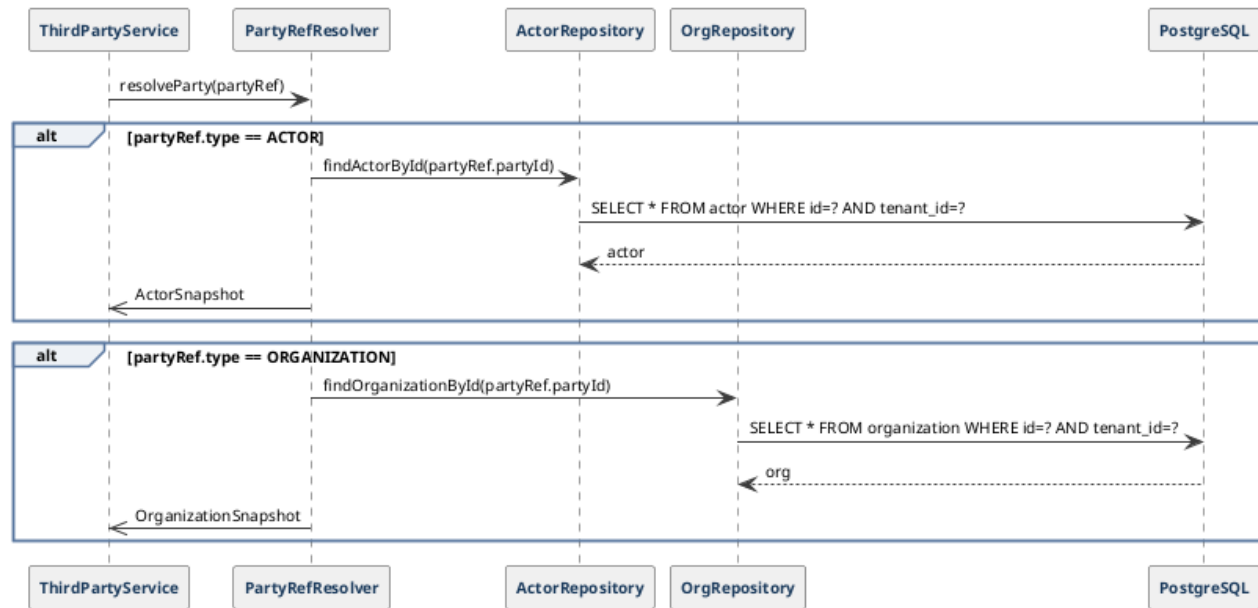
DS-C-01 — Attachement d'une adresse polymorphique



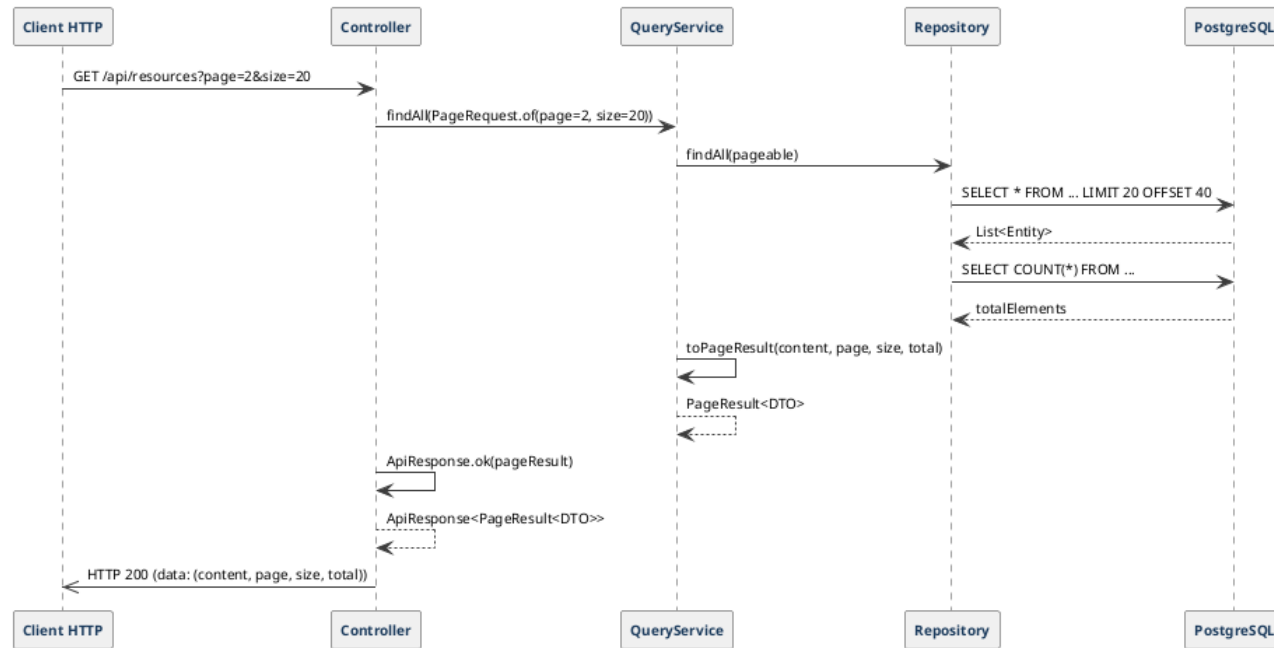
2.3.1 DS-C-02 : Résolution d'une PartyRef

Ce mécanisme est utilisé par le tp-core pour résoudre dynamiquement le type de la partie (acteur humain ou organisation) associée à un tiers.

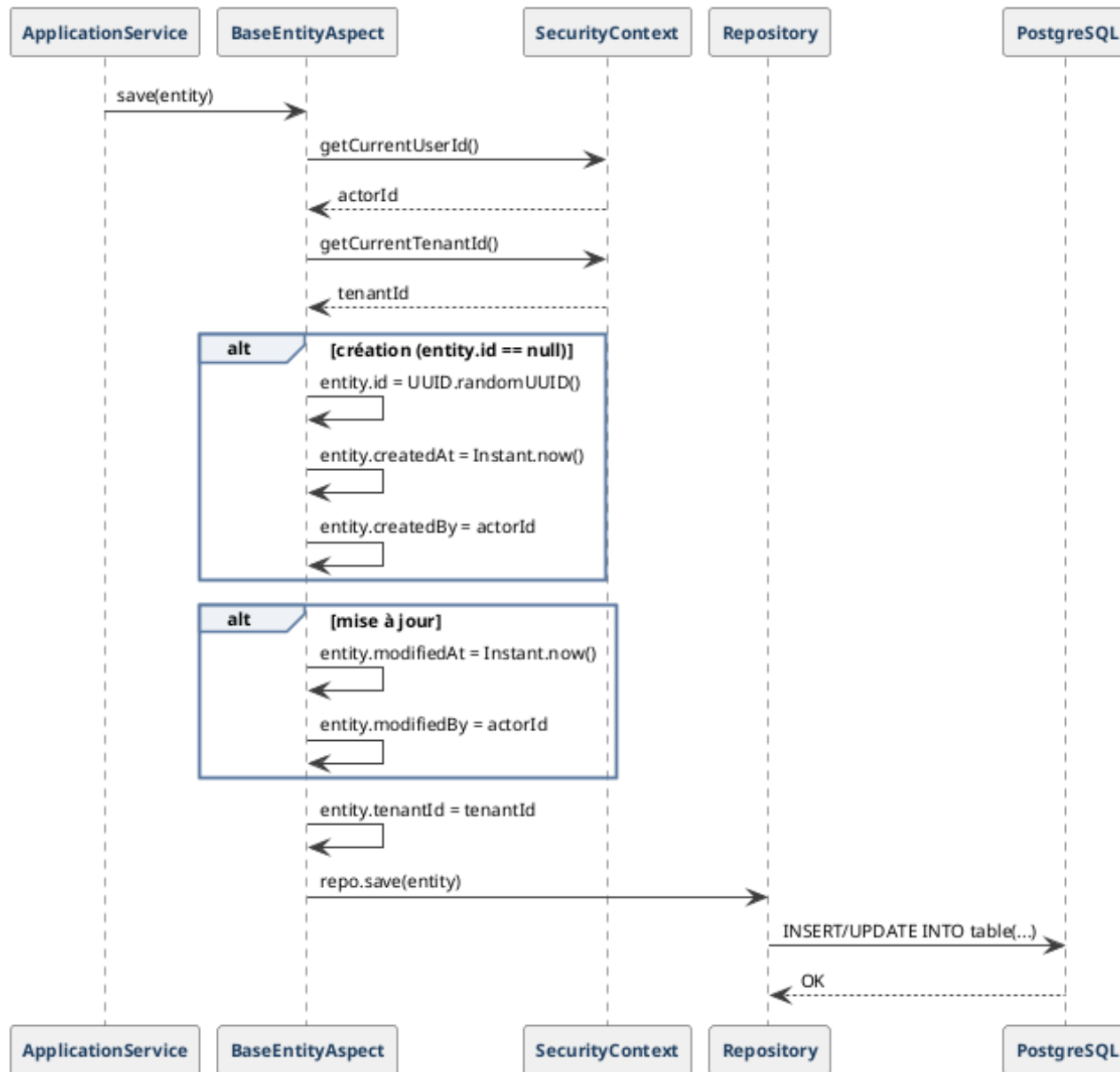
DS-C-02 — Résolution d'une PartyRef inter-cores



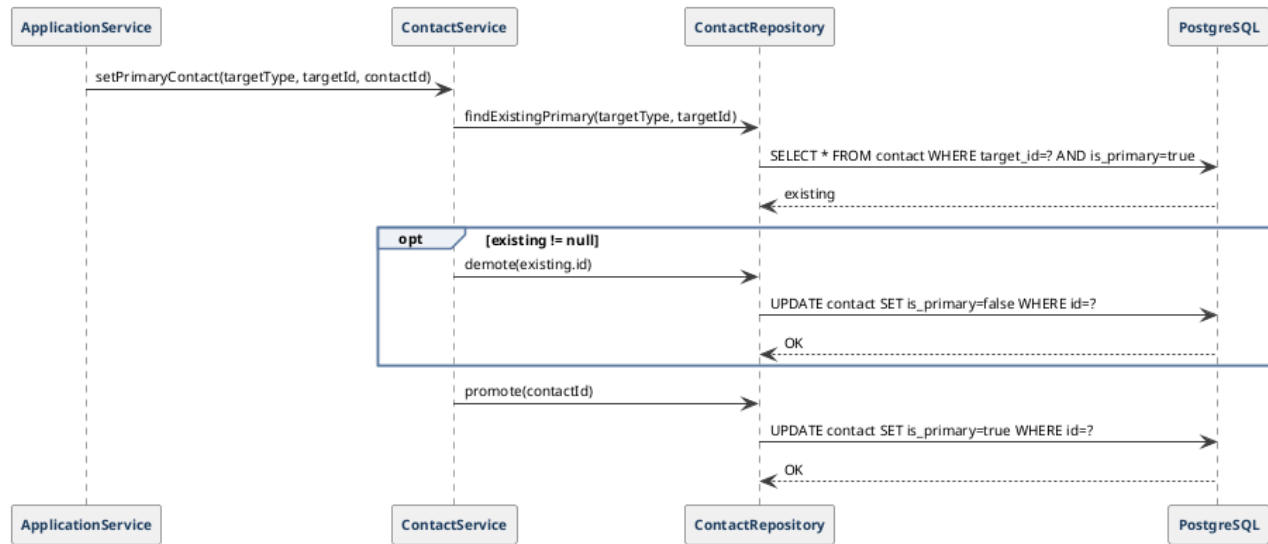
DS-C-03 — Pagination standardisée des résultats



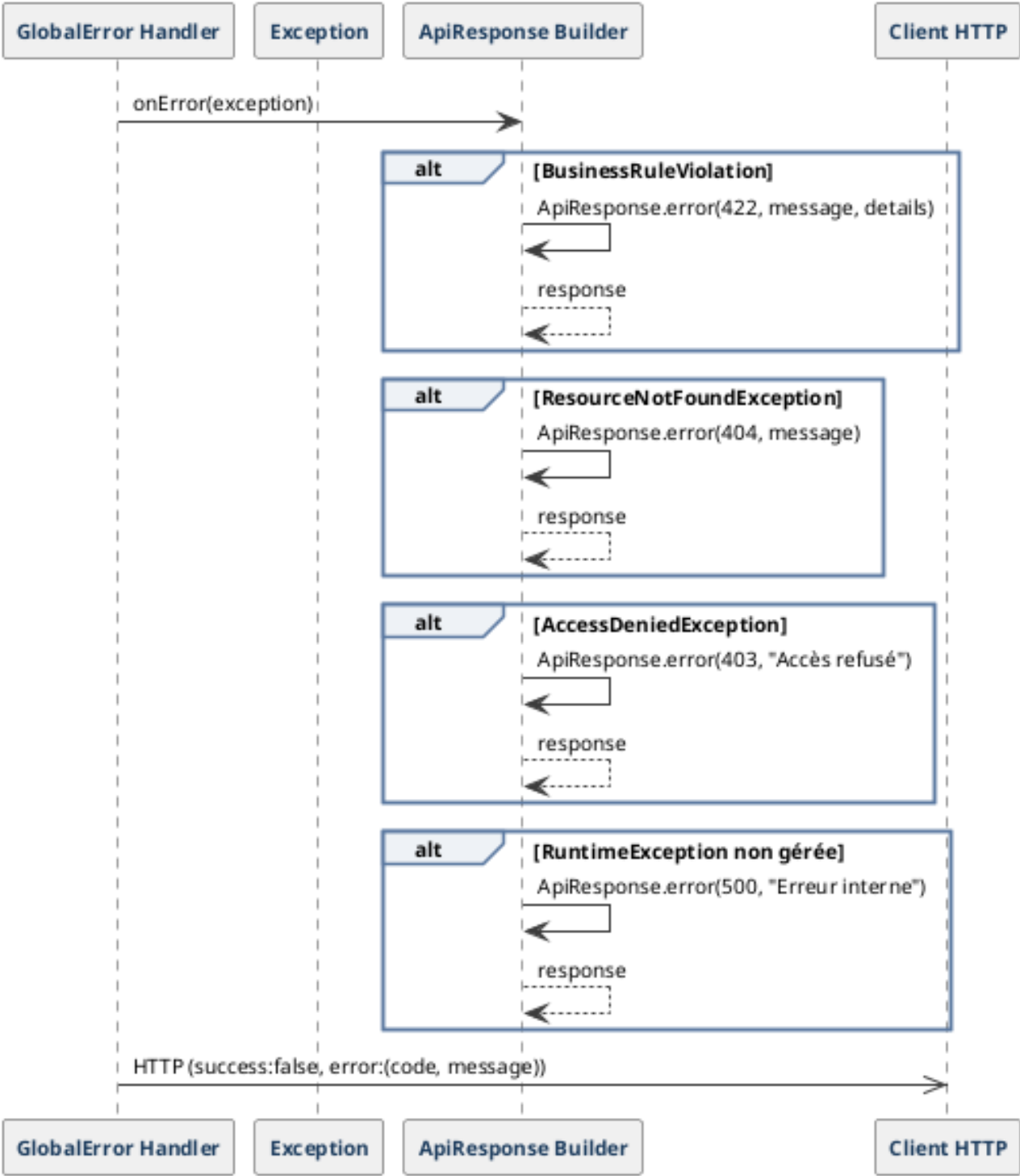
DS-C-04 — Propagation automatique des champs BaseEntity



DS-C-05 — Gestion du contact principal (promotion/rétrogradation)



DS-C-06 — Construction d’une réponse d’erreur standardisée



3 actor-core

3.1 Description du core

L'**actor-core** gère les identités humaines internes à la plateforme, c'est-à-dire les personnes physiques qui opèrent au sein du système avant toute notion commerciale. Un *Actor* représente une identité nominative (prénom, nom, e-mail, téléphone) qui peut être liée à plusieurs rôles organisationnels : employé d'une organisation, responsable commercial, ou propriétaire fondateur (*BusinessActor*).

Ce core est le point d'entrée du cycle de vie des personnes physiques dans la plateforme. Il précède et conditionne la création d'un compte utilisateur (auth-core) et toute affectation de rôle (roles-core). Il ne gère ni l'authentification ni les autorisations.

3.2 Rôle principal

Rôle principal : Gérer le cycle de vie complet des identités humaines internes (création, mise à jour, désactivation, réactivation) et maintenir leurs liens avec les entités organisationnelles sous forme d'emplois et de rôles commerciaux, dans un contexte strictement multi-tenant.

3.3 Rôles secondaires

1. Gestion des *BusinessActors*

Le *BusinessActor* est l'acteur fondateur d'un tenant. Il est soumis à un processus de vérification et d'approbation par l'administration système avant que son tenant puisse être provisionné. Un *BusinessActor* peut posséder plusieurs organisations.

2. Gestion des relations d'emploi (*Employee*)

Un acteur peut être lié à plusieurs organisations par des contrats d'emploi distincts. Chaque relation d'emploi possède un statut (**ACTIVE**, **TERMINATED**, **SUSPENDED**) et une date de début et de fin.

3. Gestion des commerciaux (*SalesPerson*)

Un acteur peut être qualifié comme responsable commercial pour une organisa-

tion donnée. Cette qualification porte des informations financières spécifiques : plafond de crédit, encours, taux de commission.

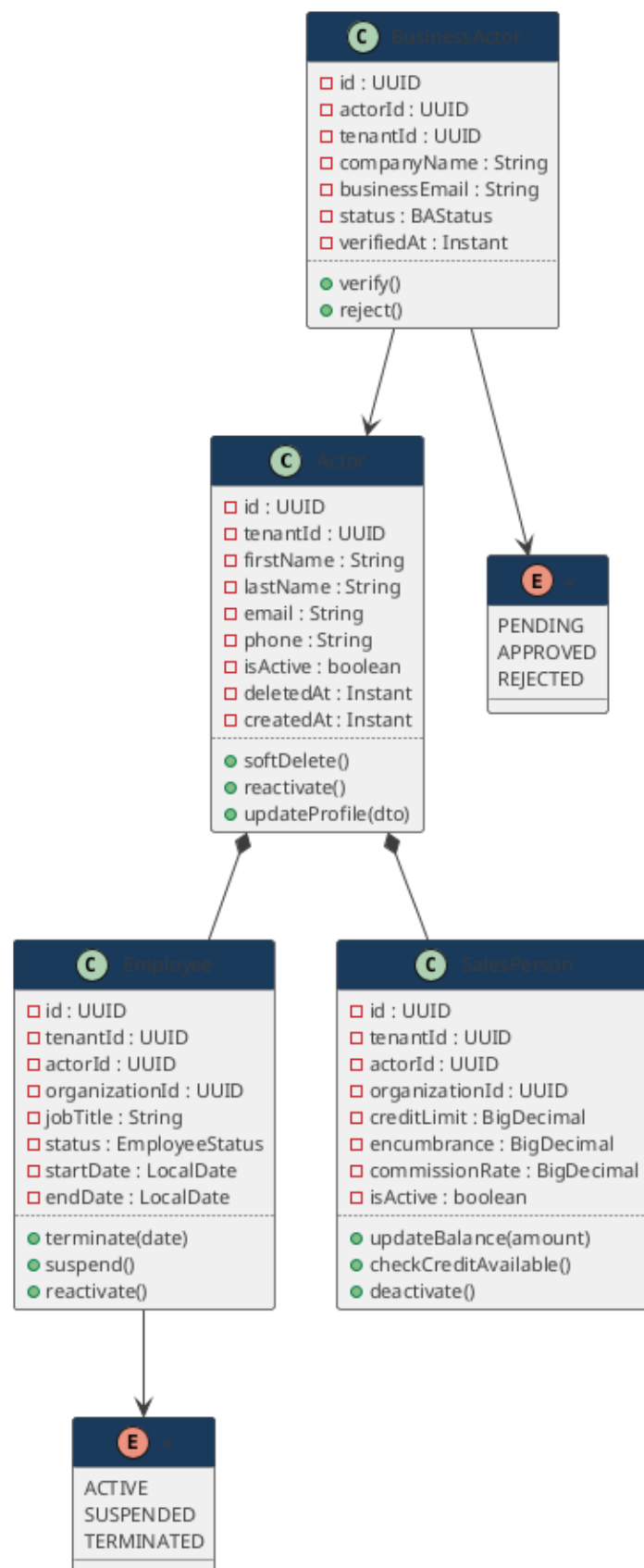
4. **Suppression douce (*soft delete*)**

Les acteurs ne sont jamais supprimés physiquement. La suppression douce préserve l'historique des opérations associées tout en marquant l'acteur comme inactif.

5. **Source de vérité pour les références humaines**

Tout autre core qui doit référencer une personne physique (par exemple le core tiers pour les mandataires) utilise uniquement l'identifiant d'acteur, sans naviguer vers l'objet Actor directement.

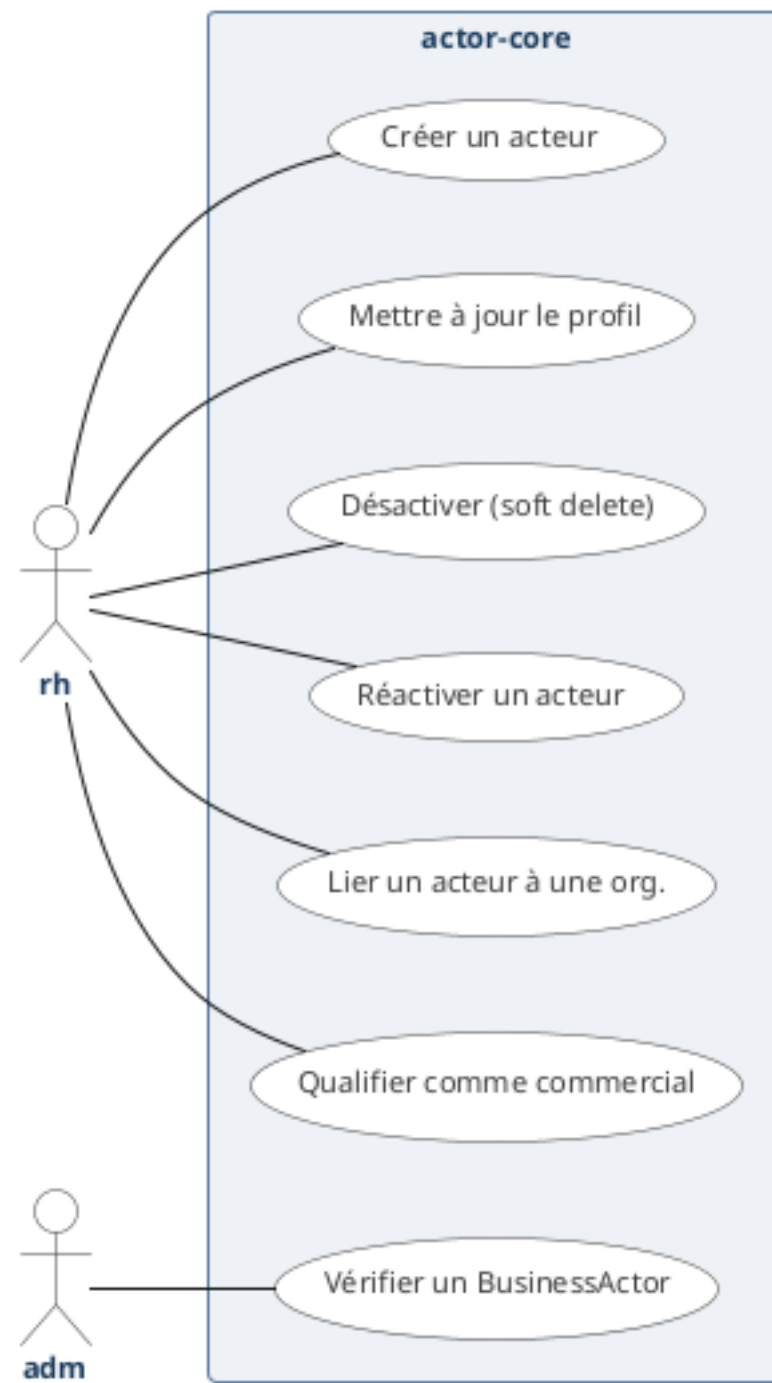
Diagramme de classes — actor-core



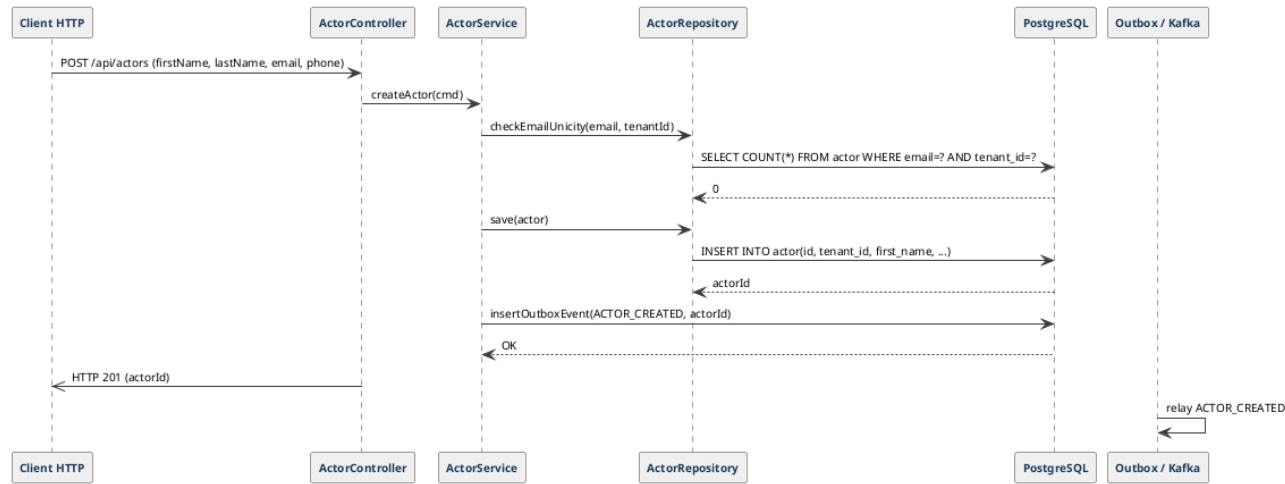
Invariants du actor-core :

- L'e-mail d'un Actor est unique par tenant (si renseigné).
- Un Actor supprimé (soft delete) ne peut pas avoir de nouvelles relations d'emploi actives.
- Un *BusinessActor* ne peut posséder qu'un seul tenant.
- `encumbrance` $\leq$ `creditLimit` pour un SalesPerson actif.
- Un SalesPerson est unique par (actorId, organizationId, tenantId).

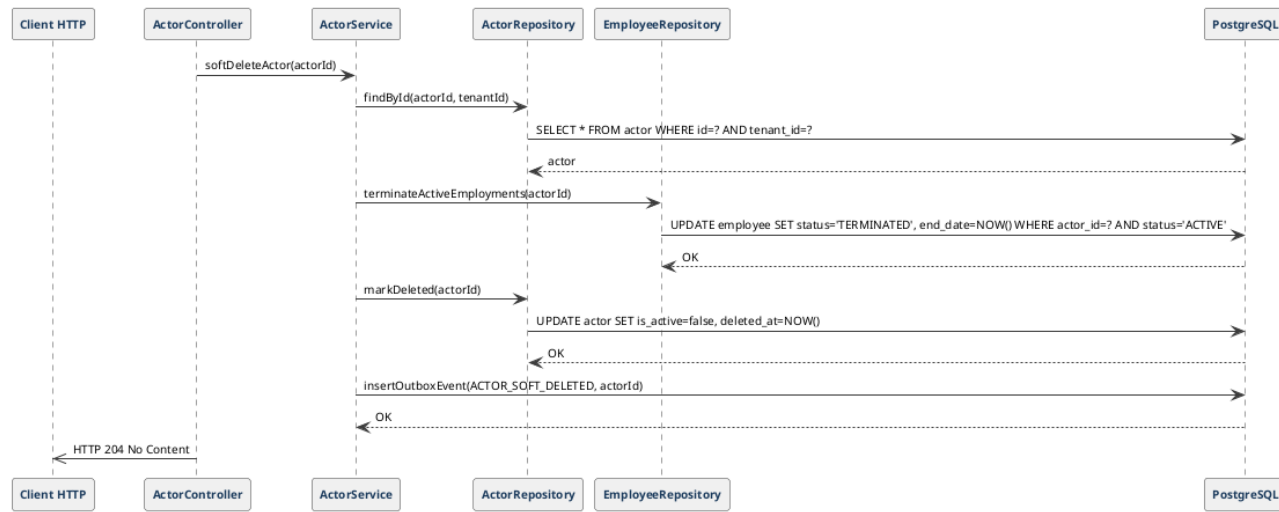
Diagramme des cas d'utilisation — actor-core



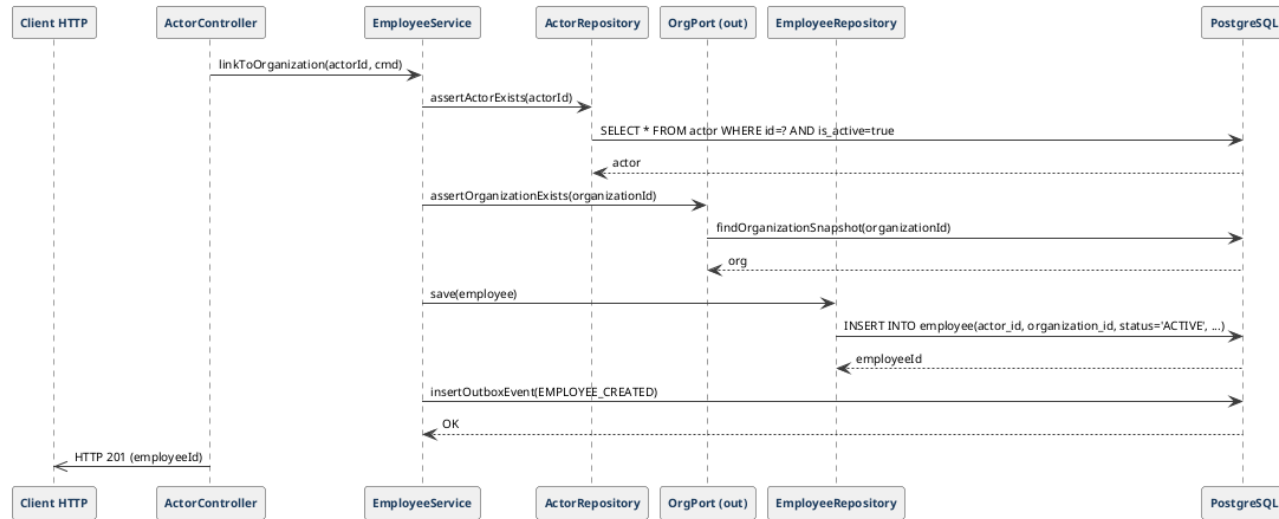
DS-A-01 — Création d'un acteur



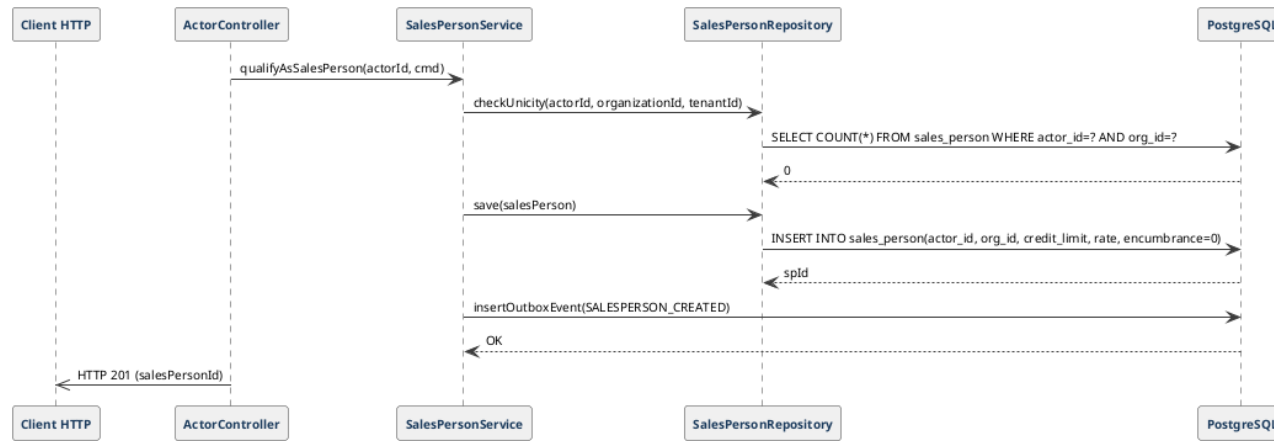
DS-A-02 — Suppression douce d'un acteur



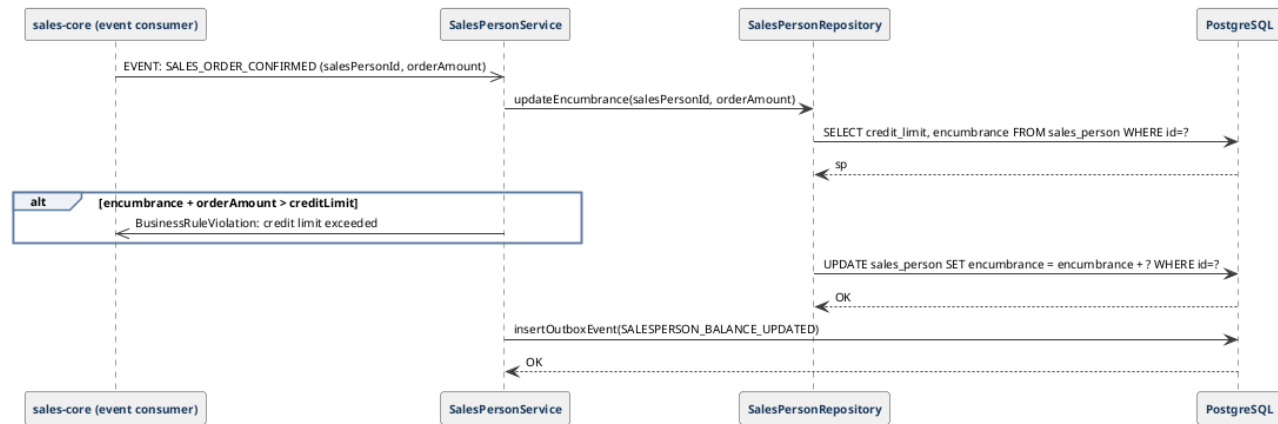
DS-A-03 — Association acteur-organisation (relation d'emploi)



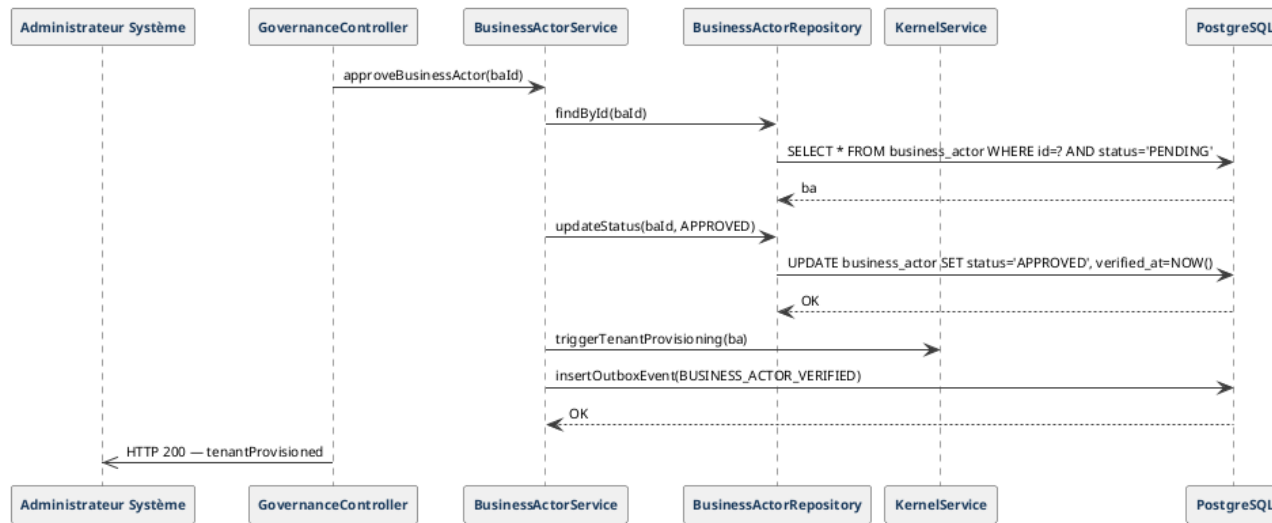
DS-A-04 — Qualification d'un acteur comme commercial (SalesPerson)



DS-A-05 — Mise à jour de l'encours d'un commercial



DS-A-06 — Approbation d'un BusinessActor et déclenchement du provisionnement



4 auth-core

4.1 Description du core

L'**auth-core** (YowAuth0-Core) est le module responsable de l'authentification technique et de la gestion des comptes utilisateurs. Il fait la distinction nette entre l'identité humaine (gérée par actor-core) et le compte technique permettant de se connecter au système.

Un *UserAccount* peut être lié à un Actor ou exister de manière autonome (compte de service, compte externe). Il supporte deux modes d'authentification : local (identifiant/mot de passe hashé) et fédéré (fournisseur externe OAuth2/OIDC).

Ce core génère et valide les jetons JWT, gère la rotation des tokens de rafraîchissement, et coordonne avec le kernel-core pour la révocation.

4.2 Rôle principal

Rôle principal : Gérer le cycle de vie complet des comptes techniques (création, activation, verrouillage, suspension), assurer l'authentification par émission de JWT signés, et coordonner la rotation et la révocation des tokens de rafraîchissement.

4.3 Rôles secondaires

1. Intégration de fournisseurs d'identité externes

Support des fournisseurs OAuth2/OIDC. Pour un compte externe, le champ `externalUserId` remplace le `passwordHash`.

2. Rotation des tokens de rafraîchissement

Chaque utilisation d'un refresh token produit un nouveau token et invalide l'ancien (rotation à usage unique). Le token révoqué est inscrit dans la liste noire du kernel-core.

3. Auto-service utilisateur

Exposition d'endpoints dédiés permettant à l'utilisateur connecté de consulter son profil (GET `/users/me`), mettre à jour son plan souscrit, et gérer son onboarding progressif.

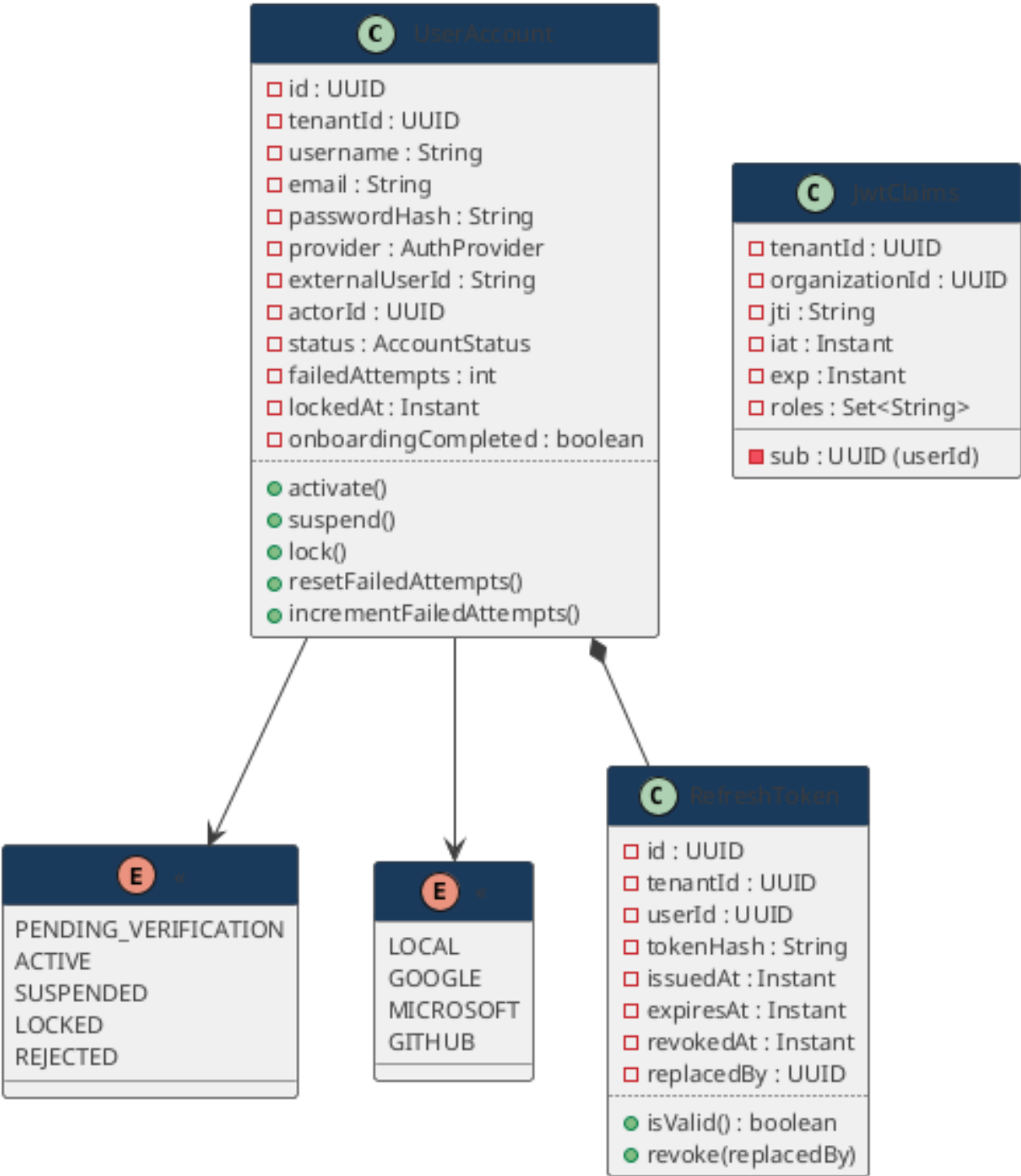
4. Gestion du cycle de vie des comptes

Transitions d'état contrôlées : `PENDING_VERIFICATION` → `ACTIVE` → `SUSPENDED` ou `LOCKED`. Le verrouillage automatique survient après un nombre configurable d'échecs de connexion.

5. Coordination avec roles-core

Lors de la création d'un compte, le auth-core émet un événement `USER_CREATED` consommé par roles-core pour l'assignation du rôle par défaut.

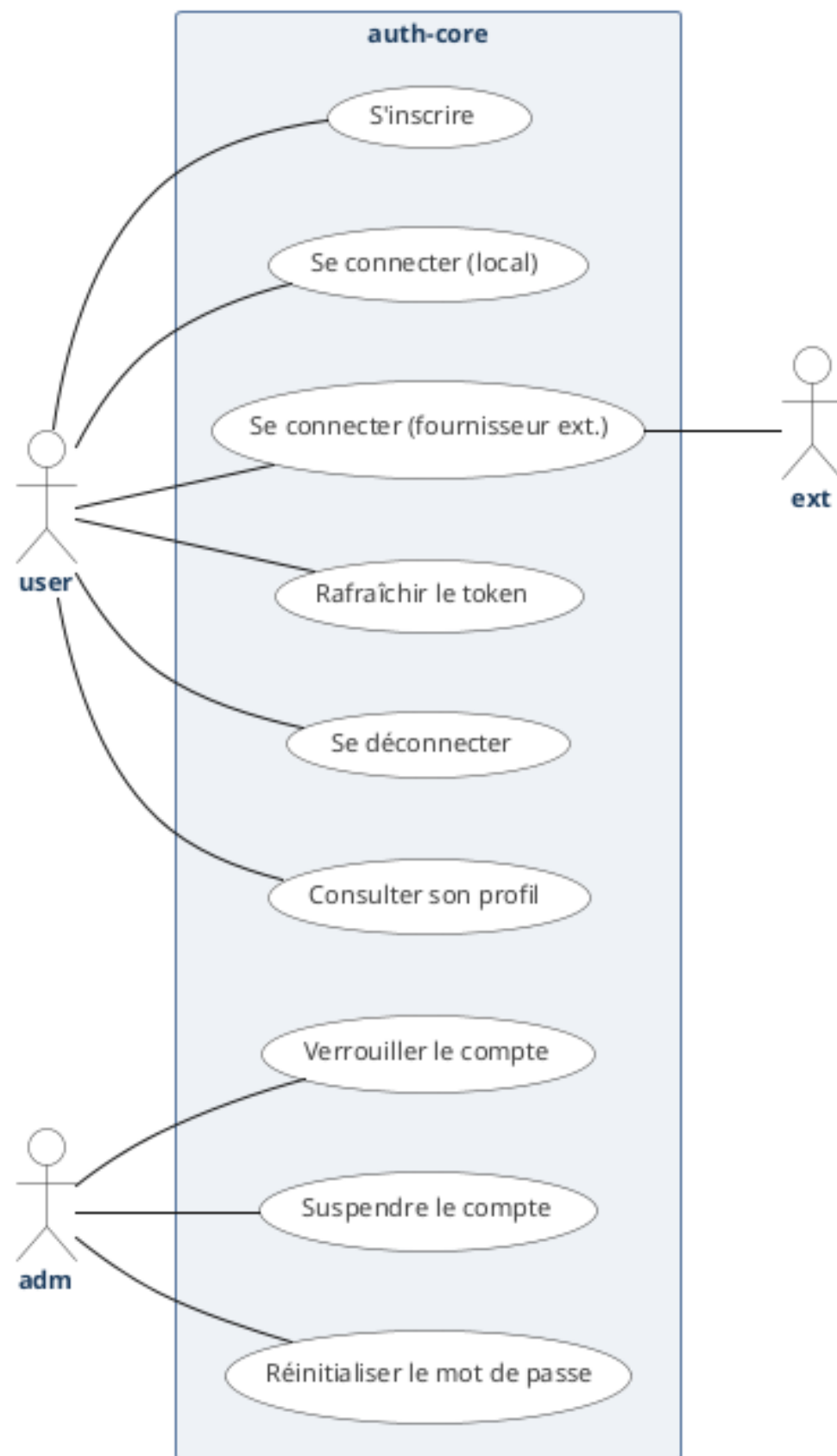
Diagramme de classes — auth-core



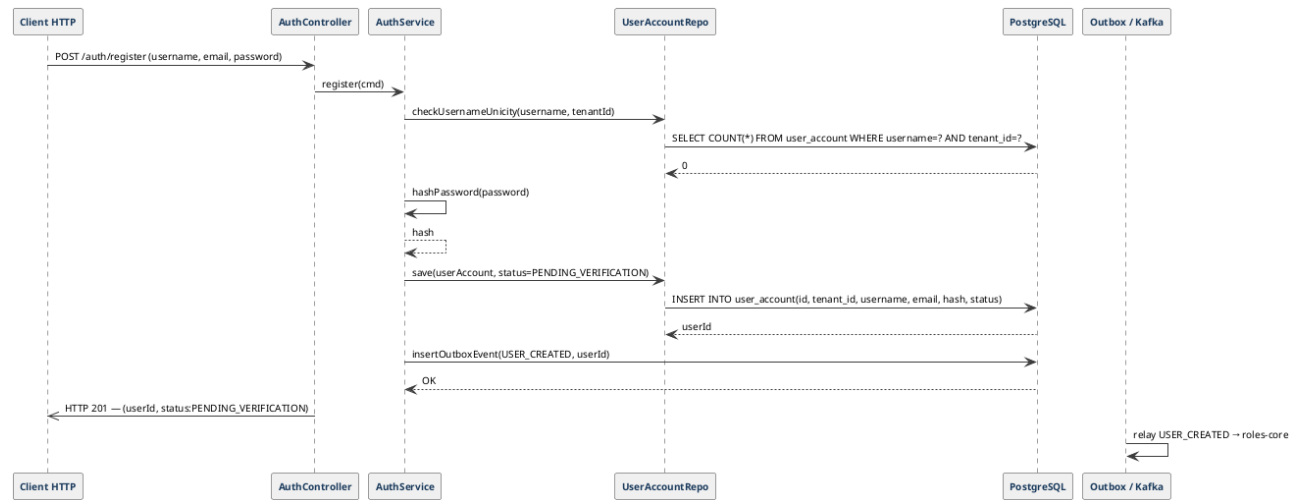
Invariants du auth-core :

- `username` est unique par tenant.
- `email` est unique par tenant.
- Pour le provider `LOCAL`, `passwordHash` est obligatoire.
- Pour les providers externes, `externalUserId` est obligatoire.
- Un compte `LOCKED` ne peut s'authentifier.
- Un `RefreshToken` ne peut être utilisé qu'une seule fois (rotation obligatoire).

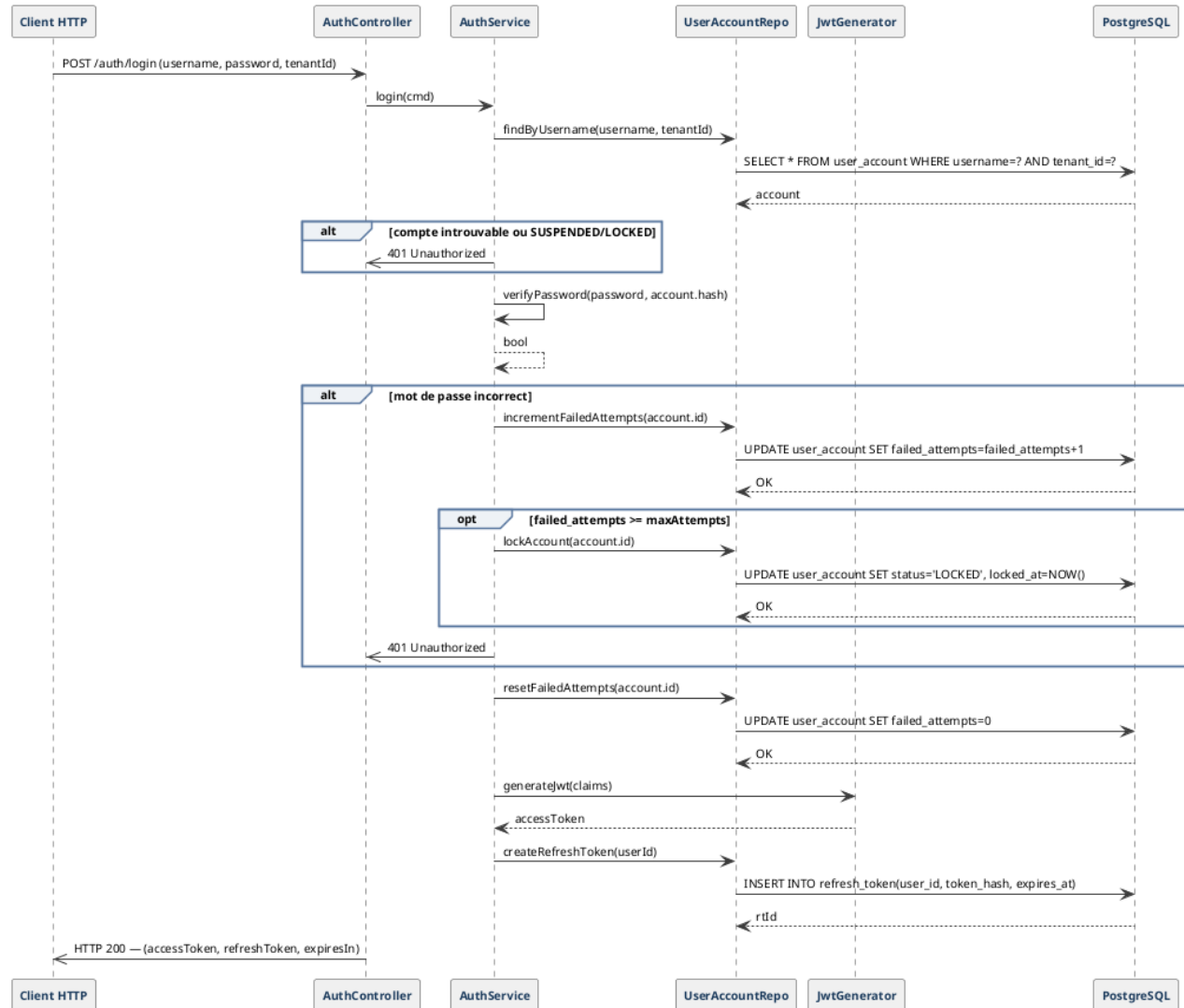
Diagramme des cas d'utilisation — auth-core



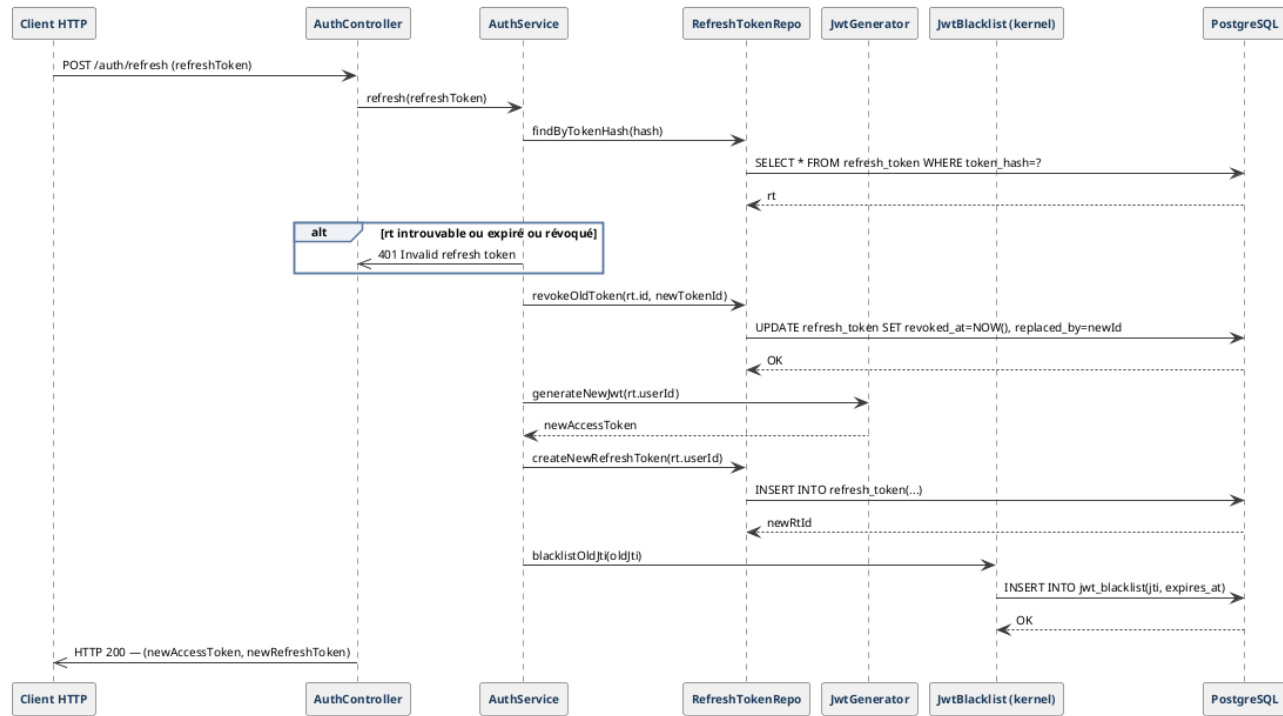
DS-AU-01 — Inscription et création de compte



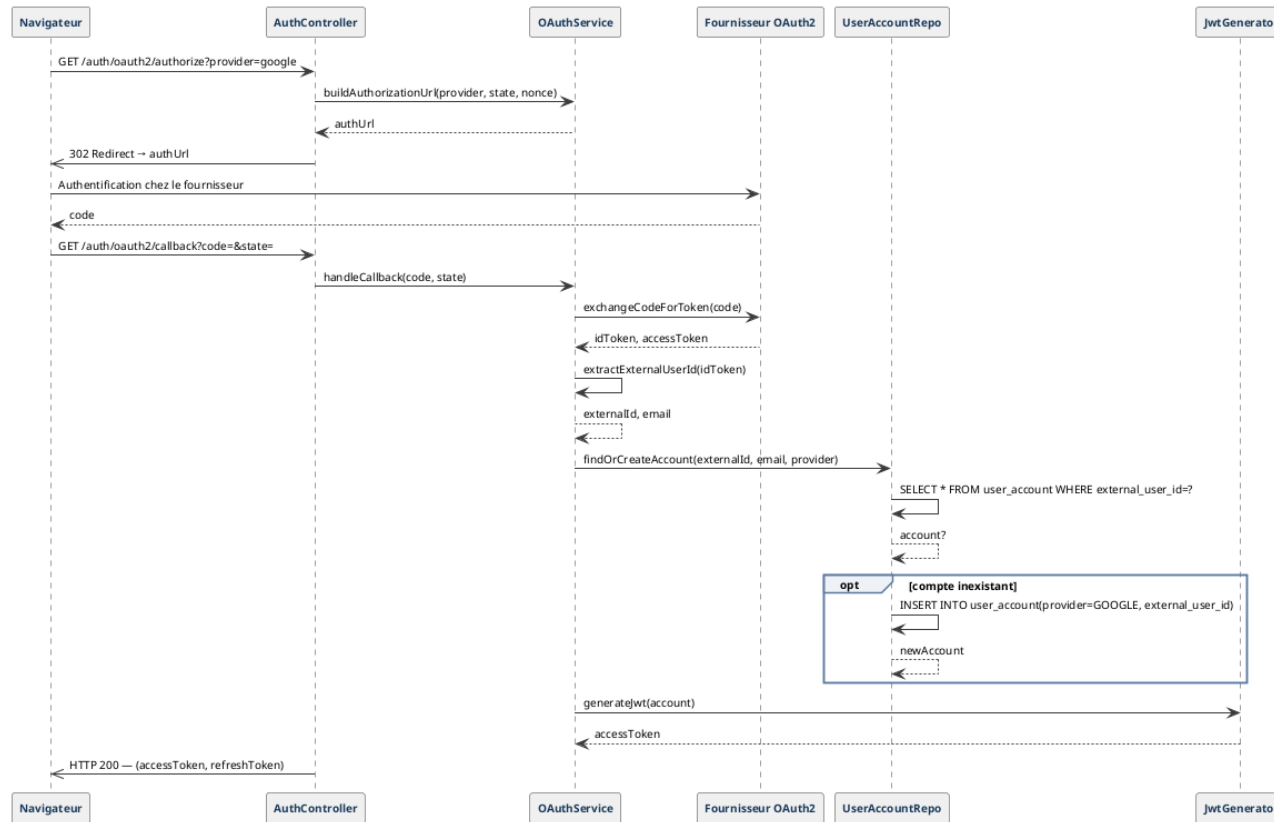
DS-AU-02 — Authentification locale et émission de JWT



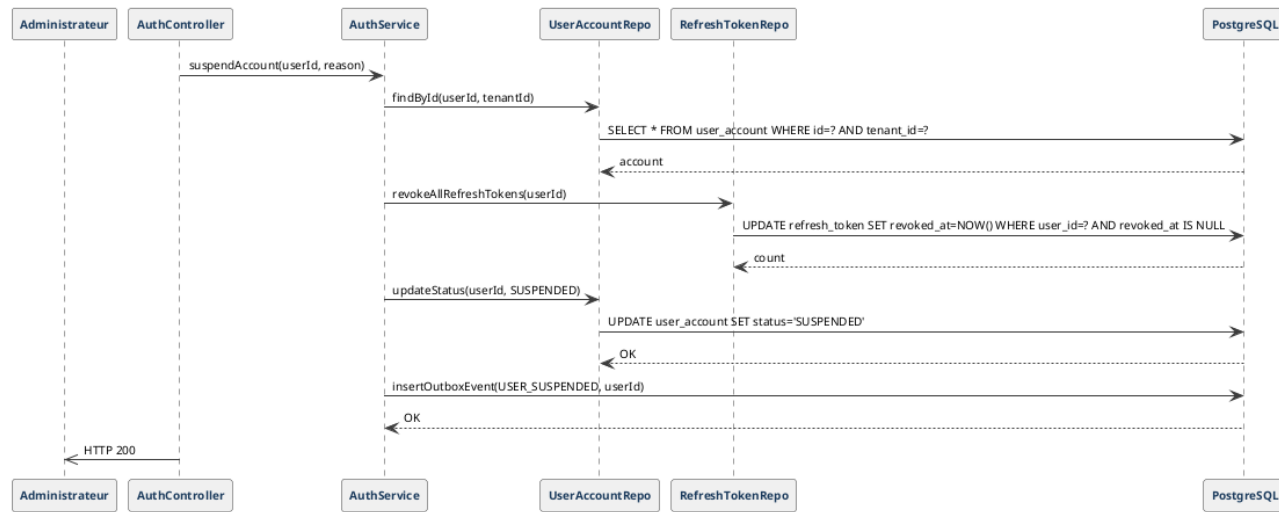
DS-AU-03 — Rotation du token de rafraîchissement



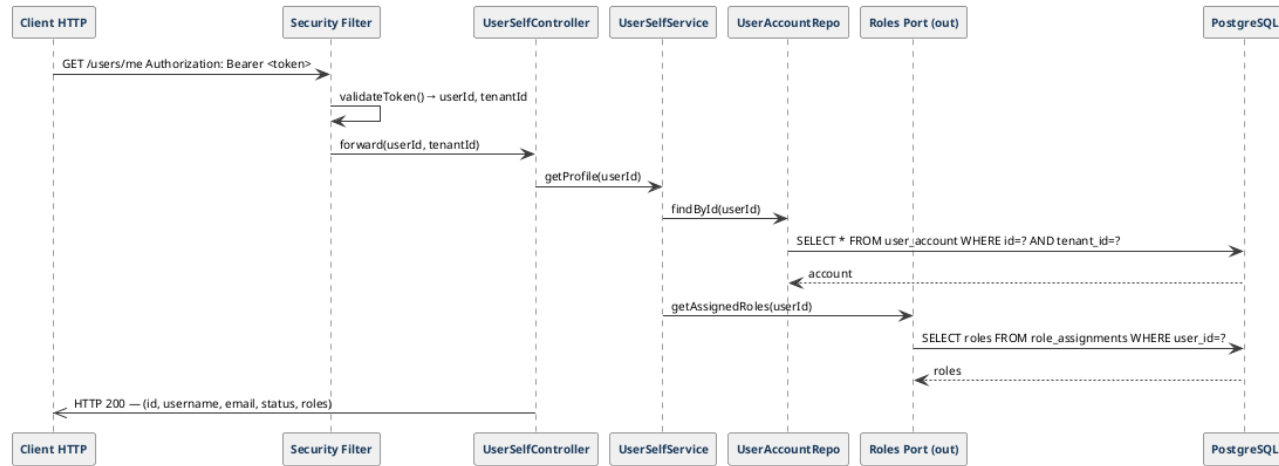
DS-AU-04 — Connexion via fournisseur OAuth2 externe



DS-AU-05 — Suspension d'un compte utilisateur



DS-AU-06 — Consultation du profil utilisateur connecté (/users/me)



5 roles-core

5.1 Description du core

Le **roles-core** est le moteur d'autorisation de la plateforme. Il implémente un contrôle d'accès basé sur les rôles (*RBAC*) avec portée multi-niveaux, entièrement découplé du mécanisme d'authentification géré par *auth-core*.

Le modèle repose sur trois concepts fondamentaux : la *Permission* (droit atomique sur une ressource), le *Rôle* (agrégat de permissions nommé), et l'*Affectation* (liaison utilisateur-rôle avec portée contextuelle). Les portées supportées sont : **SYSTEM**, **TENANT**, **ORGANIZATION** et **AGENCY**, permettant un contrôle fin selon le niveau organisationnel de l'action.

Pour des raisons de performance, les permissions résolues sont mises en cache dans Redis avec invalidation événementielle, de sorte que la majorité des vérifications d'autorisation ne sollicitent pas la base de données.

5.2 Rôle principal

Rôle principal : Définir et maintenir le catalogue de permissions et de rôles, gérer les affectations utilisateur-rôle avec portée multi-niveaux, et résoudre efficacement les autorisations à l'exécution via un cache Redis avec invalidation événementielle.

5.3 Rôles secondaires

1. Catalogue de permissions

Maintien d'un registre immuable de permissions définies par le système (code, ressource, action). Chaque permission est identifiée par une paire unique (`resource:action`), par exemple `sales:confirm_order`.

2. Gestion des rôles personnalisés

Les administrateurs peuvent créer, modifier, cloner et supprimer des rôles dans leur périmètre. La suppression d'un rôle révoque automatiquement toutes les affectations associées.

3. Templates de rôles par défaut

Lors du provisionnement d'un tenant, un ensemble de rôles standards est instan-

cié automatiquement (`GENERAL_ADMIN`, `ORG_ADMIN`, `SALES_MANAGER`, `ACCOUNTANT`, etc.).

4. Cache Redis des permissions

Les permissions d'un utilisateur sont sérialisées dans Redis avec une clé `user:{userId}:permissions` et un TTL de 5 minutes. Le cache est invalidé immédiatement sur événement `ROLE_ASSIGNED` ou `ROLE_REVOKED`.

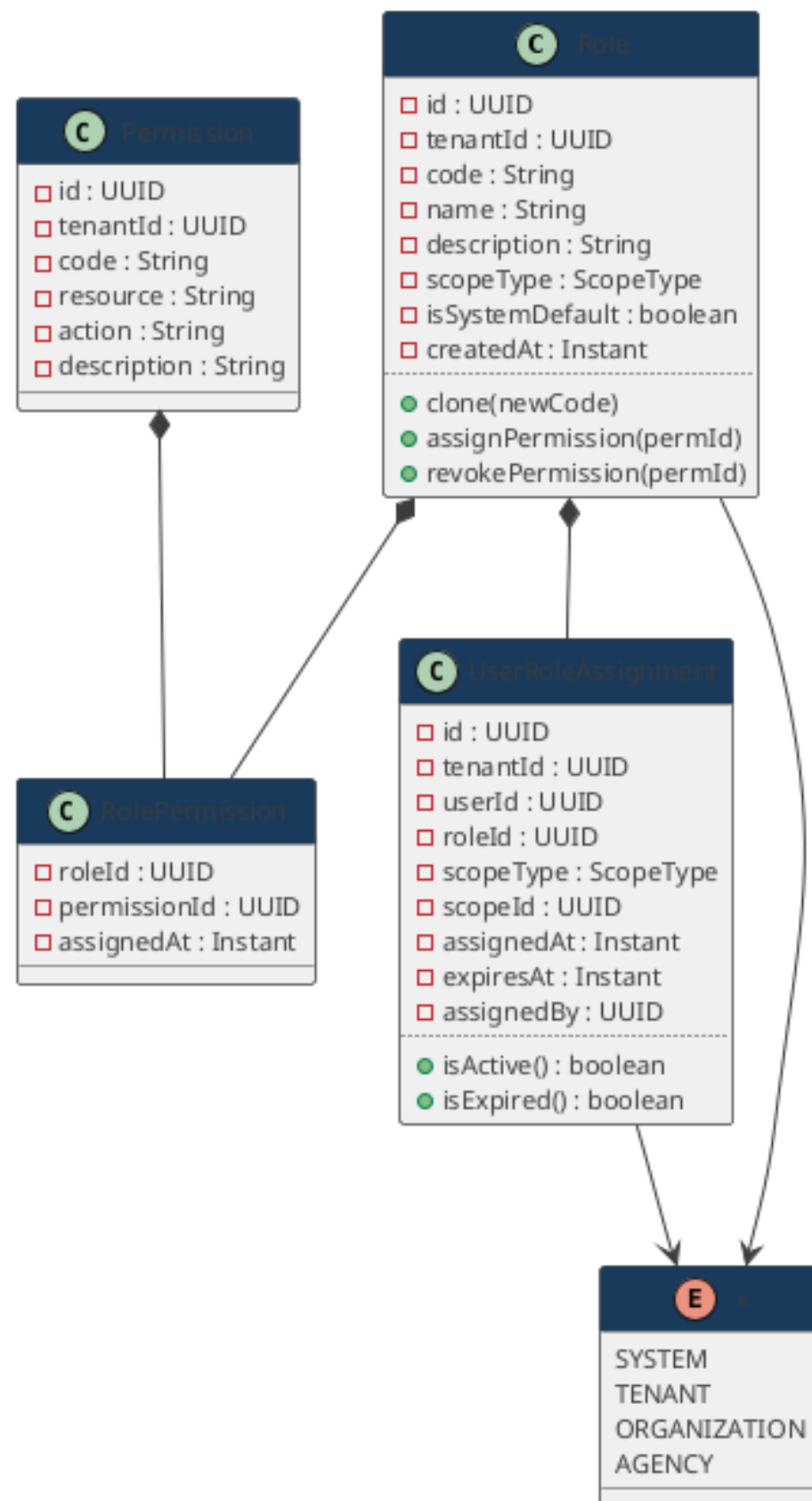
5. Portée des affectations

Une affectation peut être limitée à une organisation ou une agence précise, ce qui permet d'affecter un même utilisateur à des rôles différents selon le contexte organisationnel de sa requête.

6. Expiration des affectations

Une affectation peut être temporisée par un champ `expiresAt`. Les affectations expirées sont ignorées lors de la résolution des permissions.

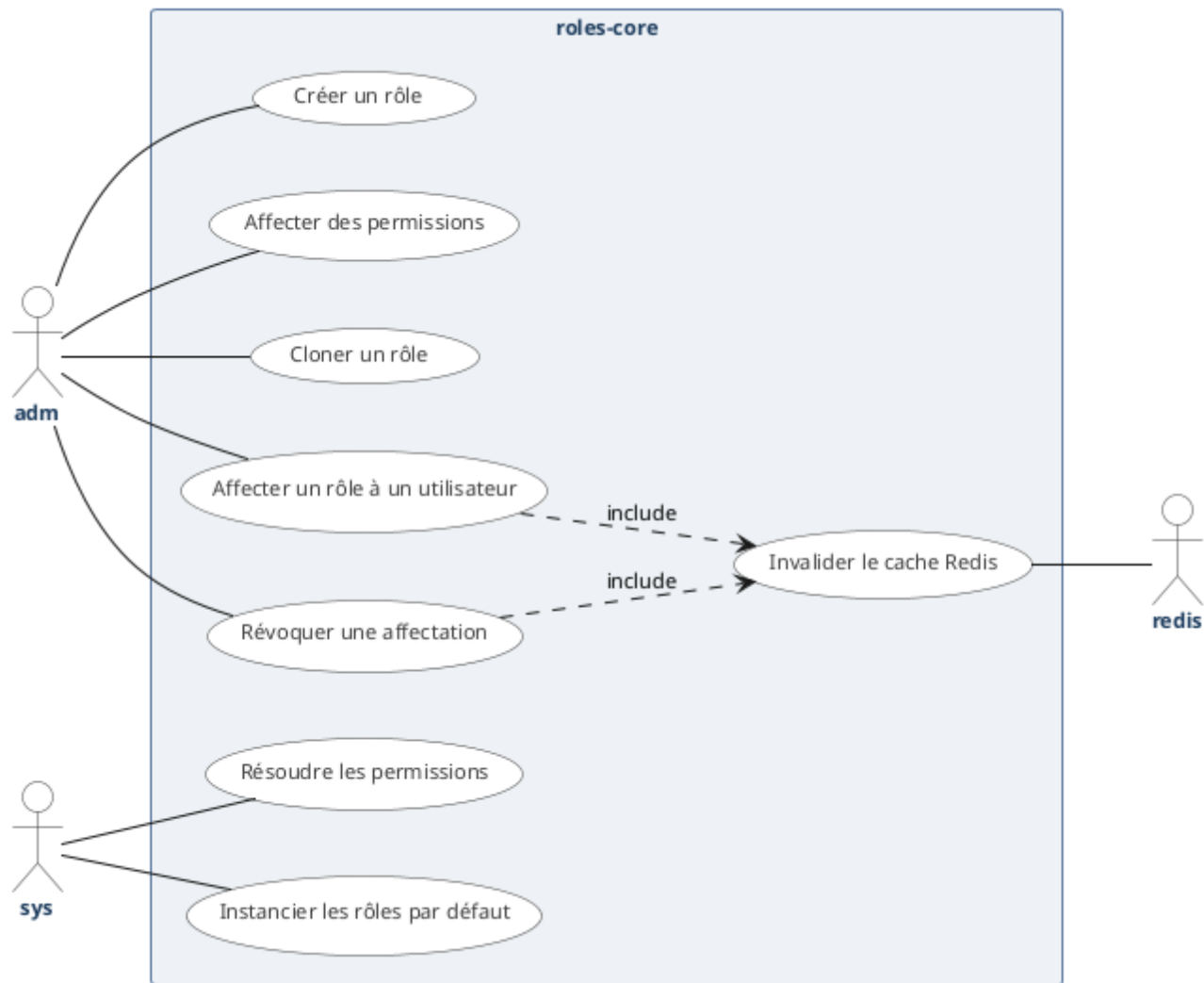
Diagramme de classes — roles-core



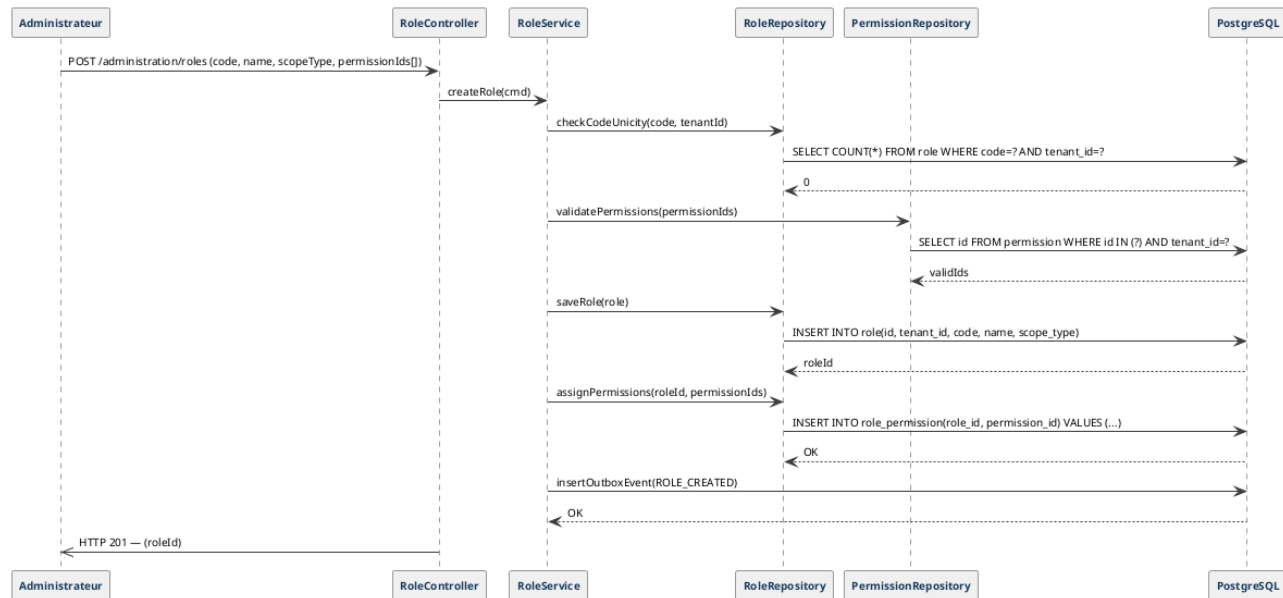
Invariants du roles-core :

- Le code d'un rôle est unique par tenant.
- La paire (`resource`, `action`) d'une permission est unique par tenant.
- Le triplet (`tenantId`, `userId`, `roleId`) d'une affectation est unique.
- Un rôle système (`isSystemDefault=true`) ne peut être supprimé.
- Une affectation expirée (`expiresAt < now`) est ignorée dans la résolution.

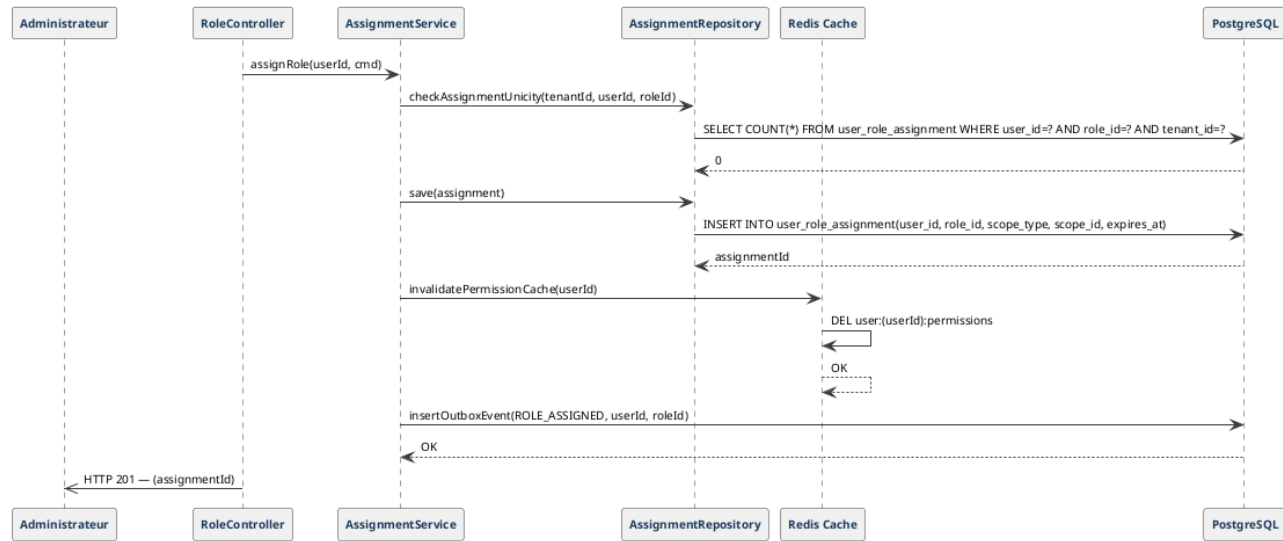
Diagramme des cas d'utilisation — roles-core



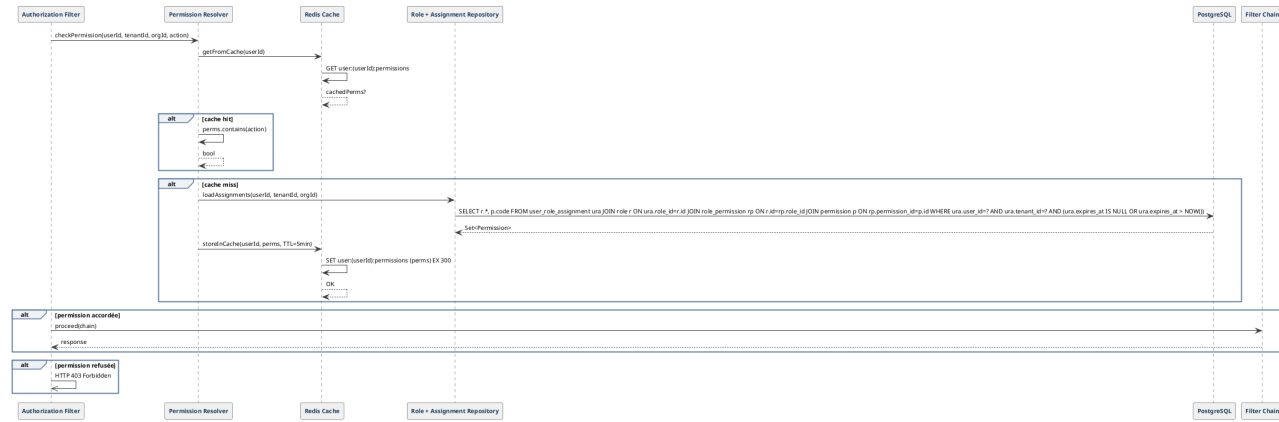
DS-R-01 — Création d'un rôle avec ses permissions



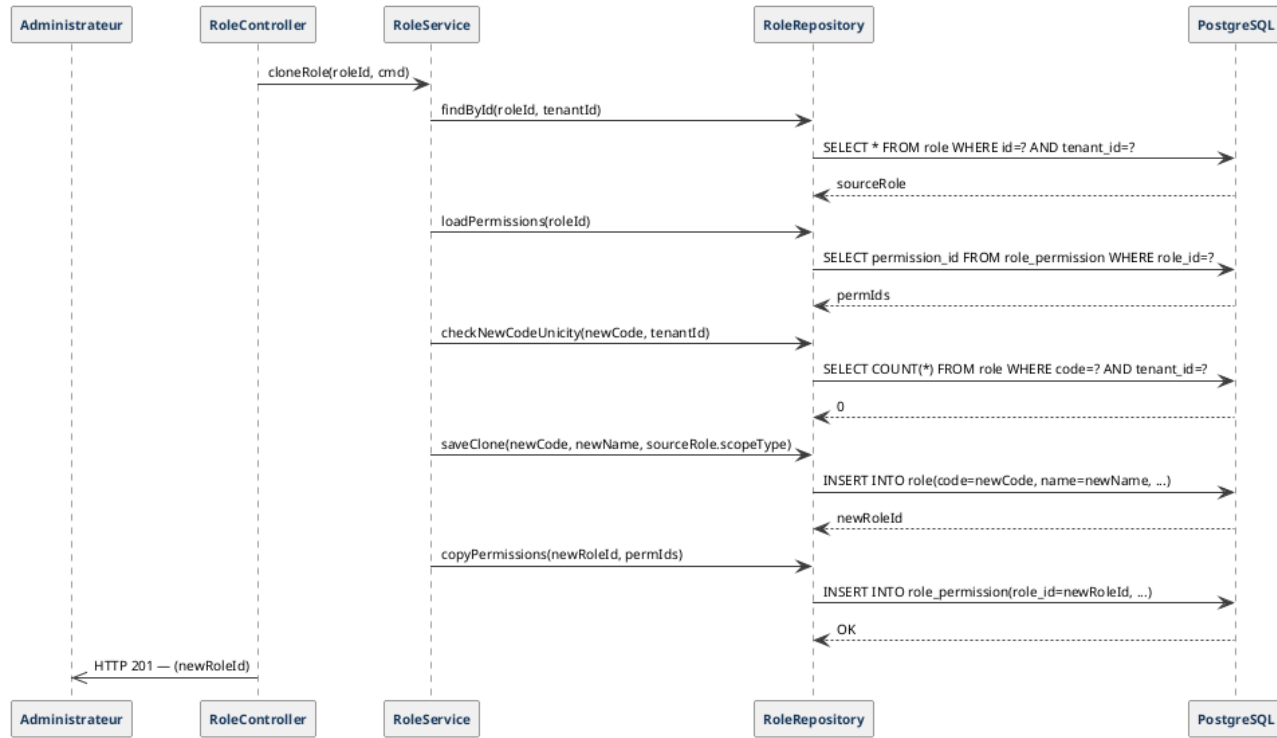
DS-R-02 — Affectation d'un rôle à un utilisateur



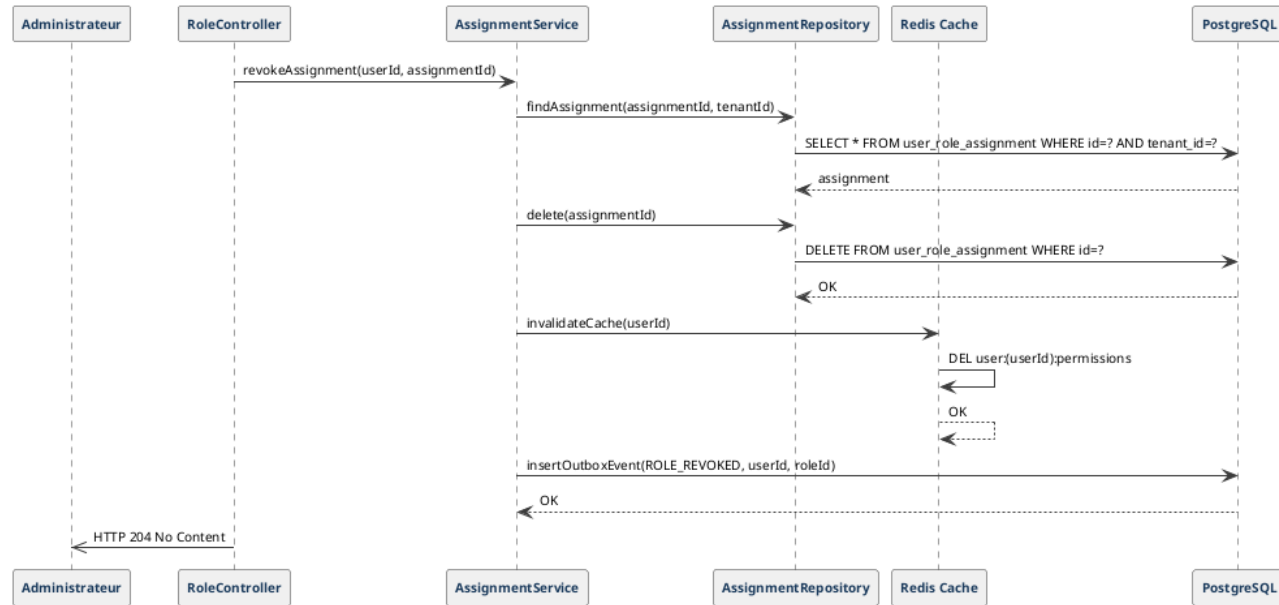
DS-R-03 — Résolution des permissions avec cache Redis



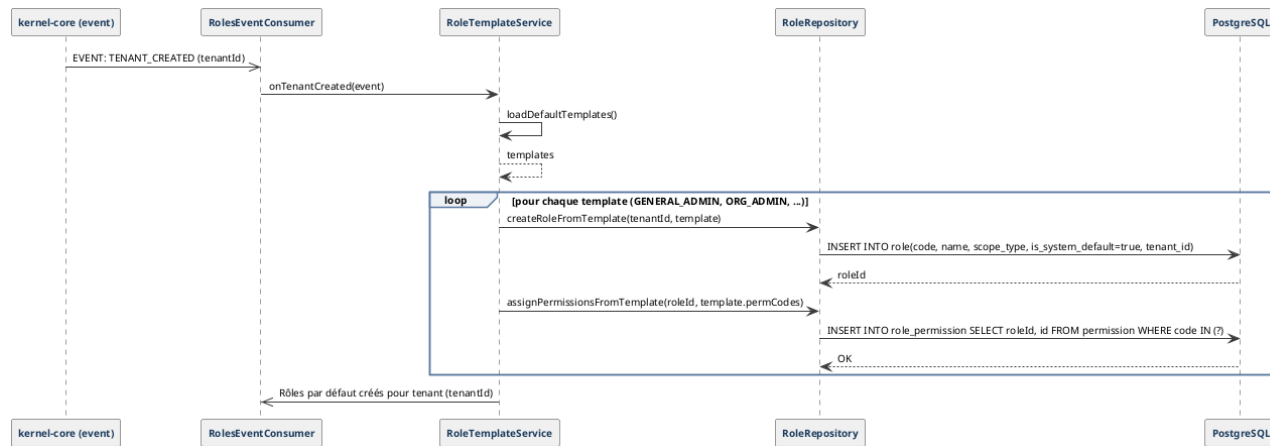
DS-R-04 — Clone d'un rôle existant



DS-R-05 — Révocation d'une affectation de rôle



DS-R-06 — Instanciation des rôles par défaut lors du provisionnement d'un tenant



6 organization-core

6.1 Description du core

L'**organization-core** modélise la structure organisationnelle des entités opérant sur la plateforme. Une *Organisation* est l'unité métier propriétaire des produits, des commandes et des stocks. Elle est composée d'une ou plusieurs *Agences* — des unités opérationnelles locales qui portent les horaires d'ouverture, les affiliations de personnel, et les points d'intérêt géographiques.

Ce core fournit également le modèle de domaine d'activité (*BusinessDomain*) qui classe les organisations selon leur secteur, ainsi que la gestion des certifications, des activités proposées, et des liens fonctionnels entre les acteurs humains et les entités organisationnelles.

6.2 Rôle principal

Rôle principal : Définir et maintenir la structure hiérarchique Organisation → Agence, gérer les liens fonctionnels entre acteurs et entités organisationnelles, et fournir aux autres cores une référence stable de l'identité et du contexte opérationnel de chaque unité.

6.3 Rôles secondaires

1. Gestion des agences opérationnelles

Chaque organisation possède une agence siège (*headquarter*) et peut avoir autant d'agences opérationnelles supplémentaires que nécessaire. L'agence est le niveau le plus fin d'exécution des opérations (stocks, horaires, affectations).

2. Horaires d'ouverture

Modèle d'horaires récurrents (*OpeningHoursRule*) et d'exceptions ponctuelles (*SpecialOpeningHours*) permettant de modéliser des calendriers complexes (jours fériés, plages horaires spécifiques).

3. Points d'intérêt géographique

Les agences peuvent déclarer des *PointOfInterest* : hubs logistiques, points de service, repères géographiques.

4. Domaines d'activité et certifications

Classification sectorielle des organisations via une taxonomie hiérarchique (**BusinessDomain**), et traçabilité des certifications professionnelles.

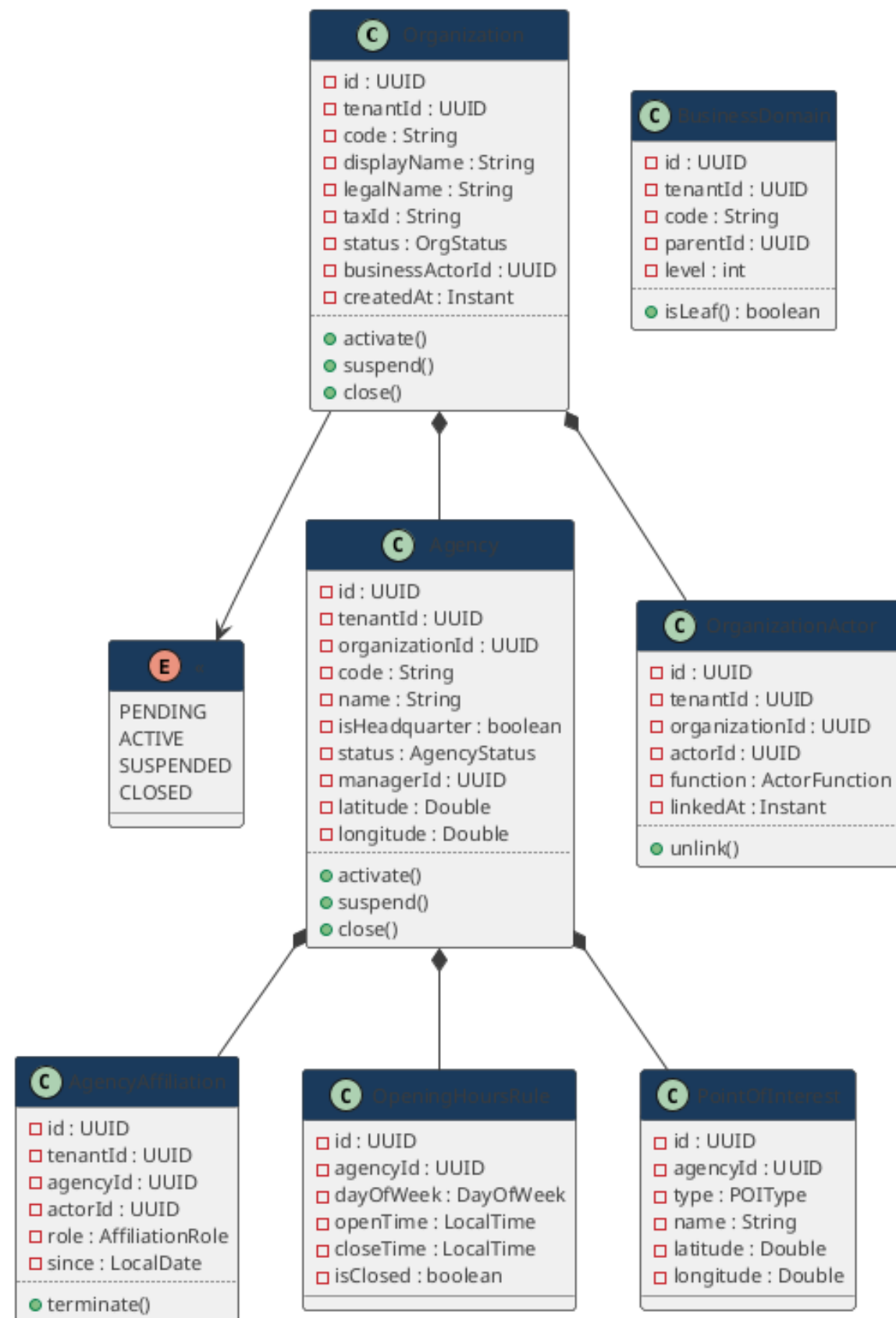
5. Source de vérité organisationnelle

Tous les cores référençant une organisation ou une agence utilisent uniquement leurs identifiants. L'organization-core est la seule source de vérité pour la dénomination, le statut et la structure.

6. Gouvernance et cycle de vie

Transitions de statut contrôlées pour les organisations et agences : création, activation, suspension, fermeture.

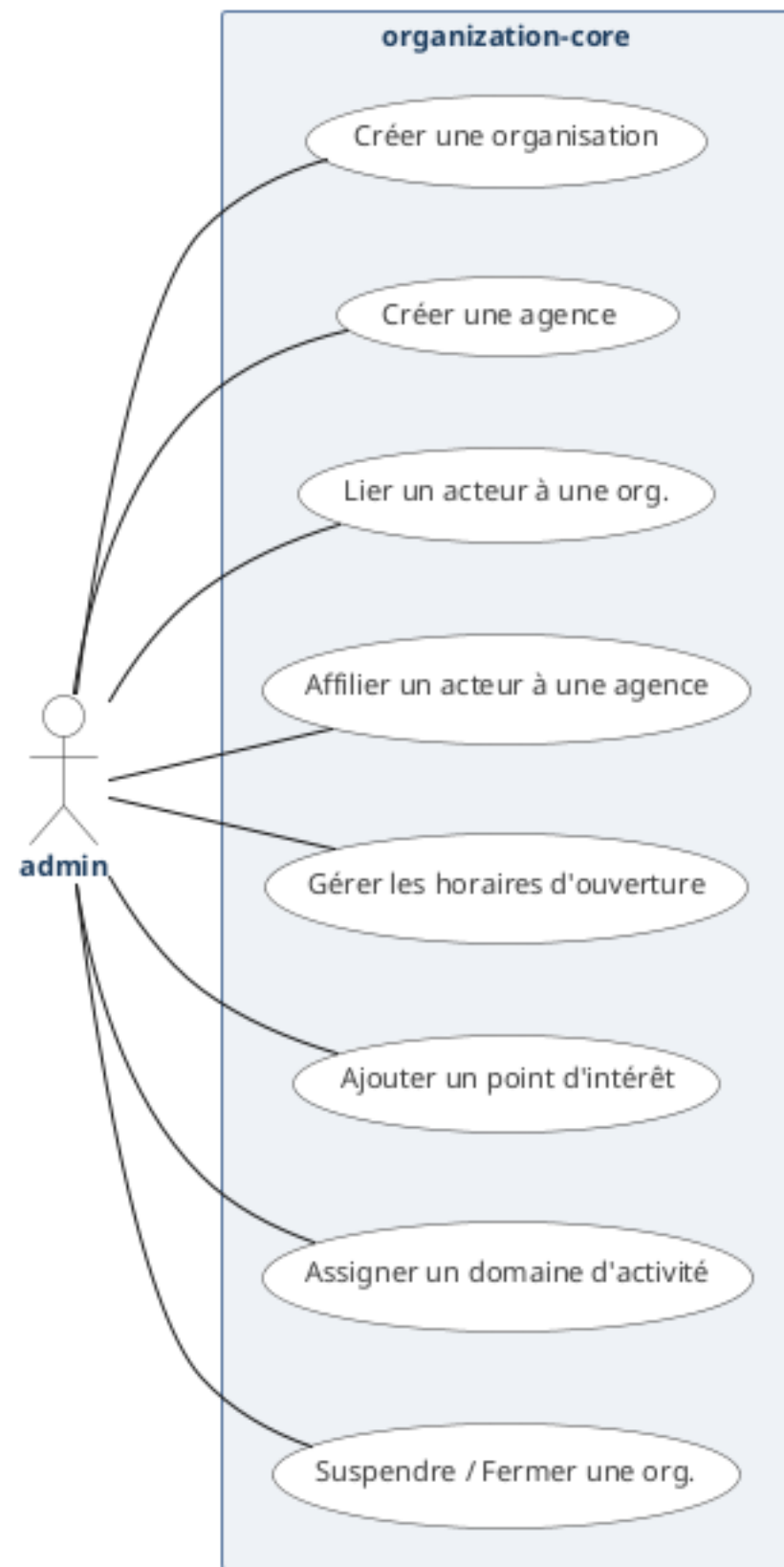
Diagramme de classes — organization-core



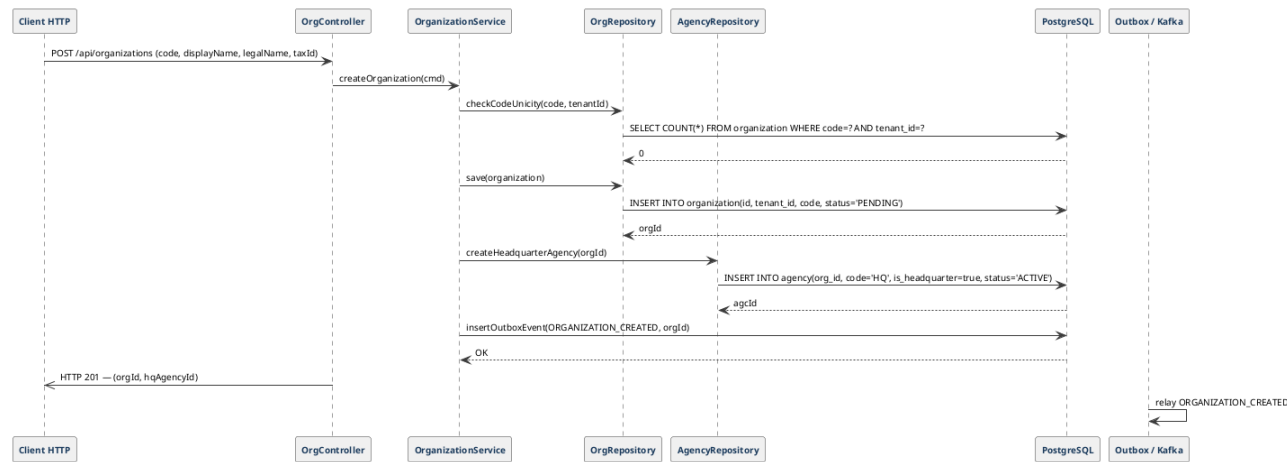
Invariants du organization-core :

- Le code d'une organisation est unique par tenant.
- Le code d'une agence est unique par organisation.
- Exactement une agence siège (`isHeadquarter=true`) par organisation.
- Une organisation `CLOSED` ne peut être réactivée.
- Le gestionnaire d'une agence doit être un acteur actif du même tenant.

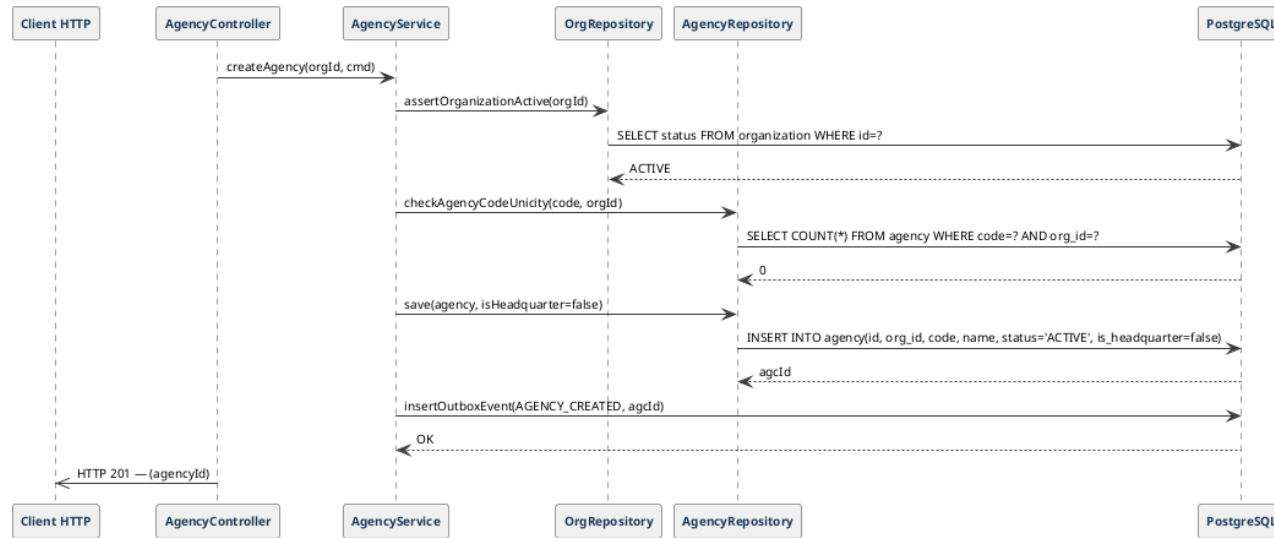
Diagramme des cas d'utilisation — organization-core



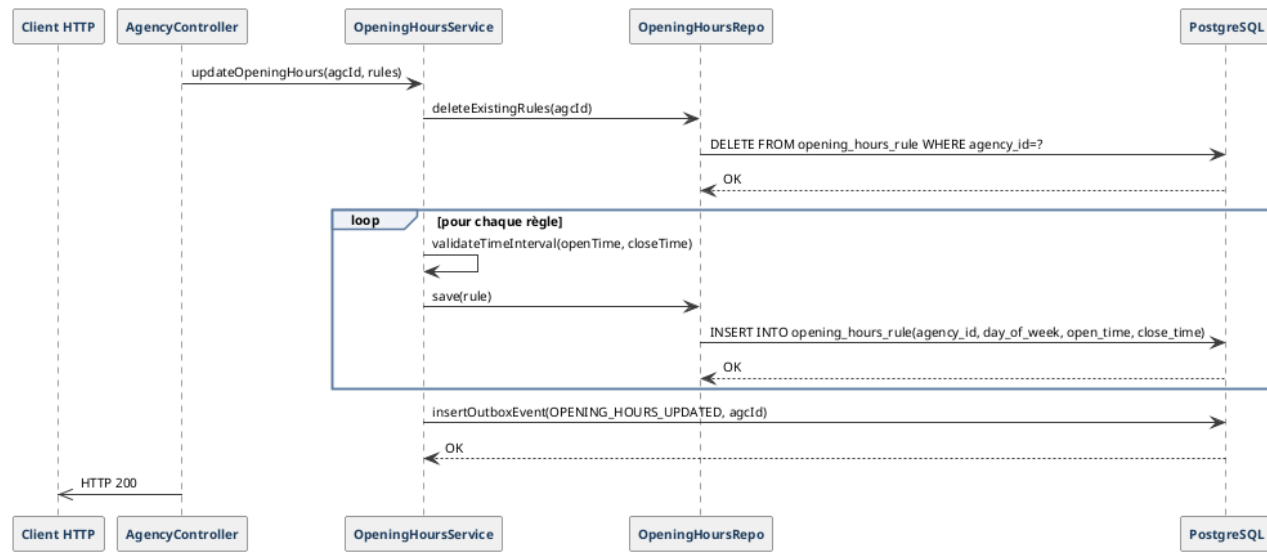
DS-O-01 — Création d'une organisation avec agence siège automatique



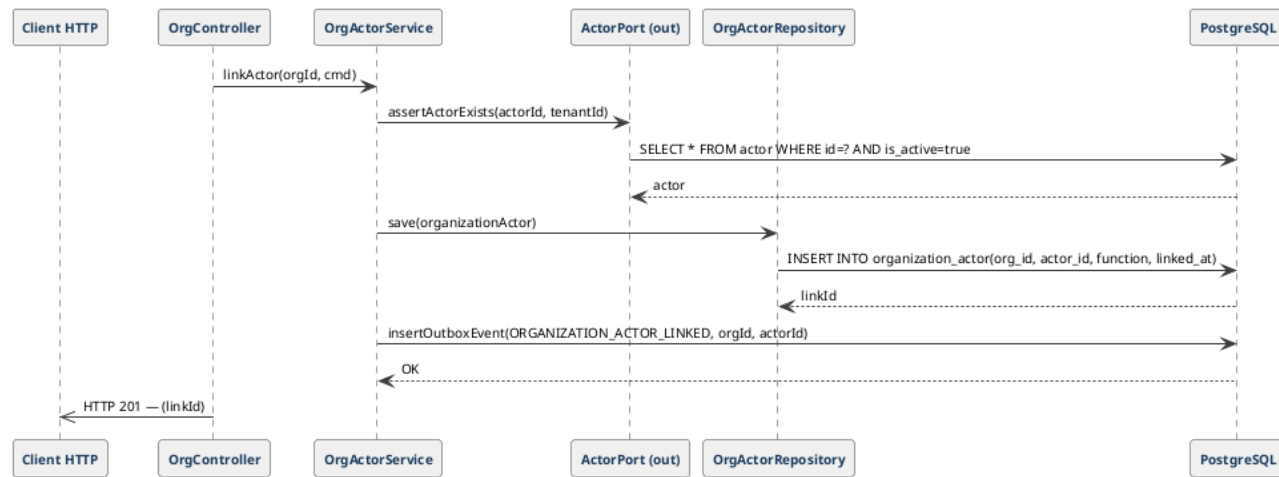
DS-O-02 — Création d'une agence opérationnelle



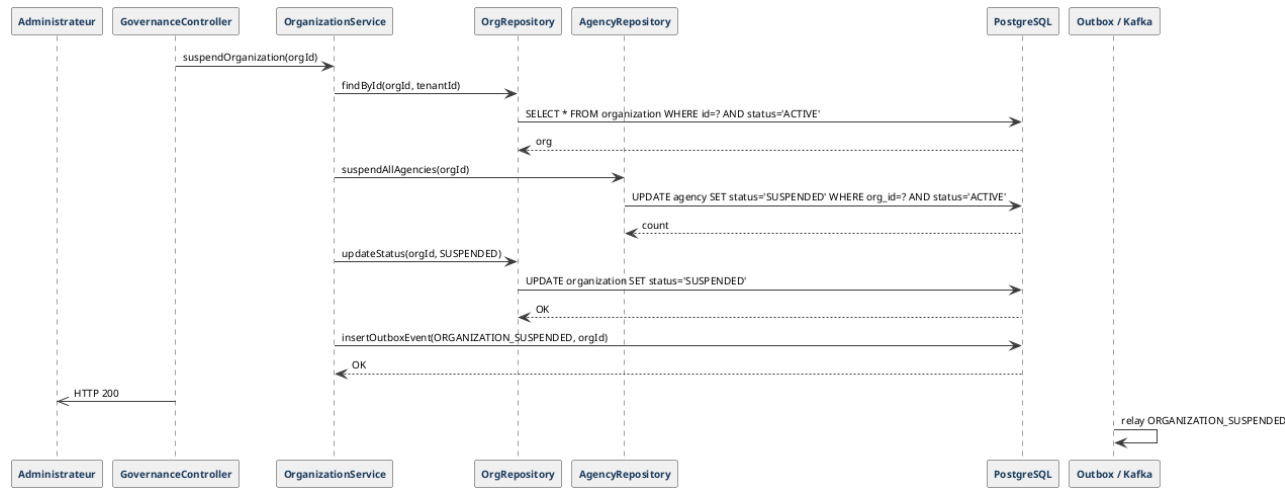
DS-O-03 — Mise à jour des horaires d'ouverture d'une agence



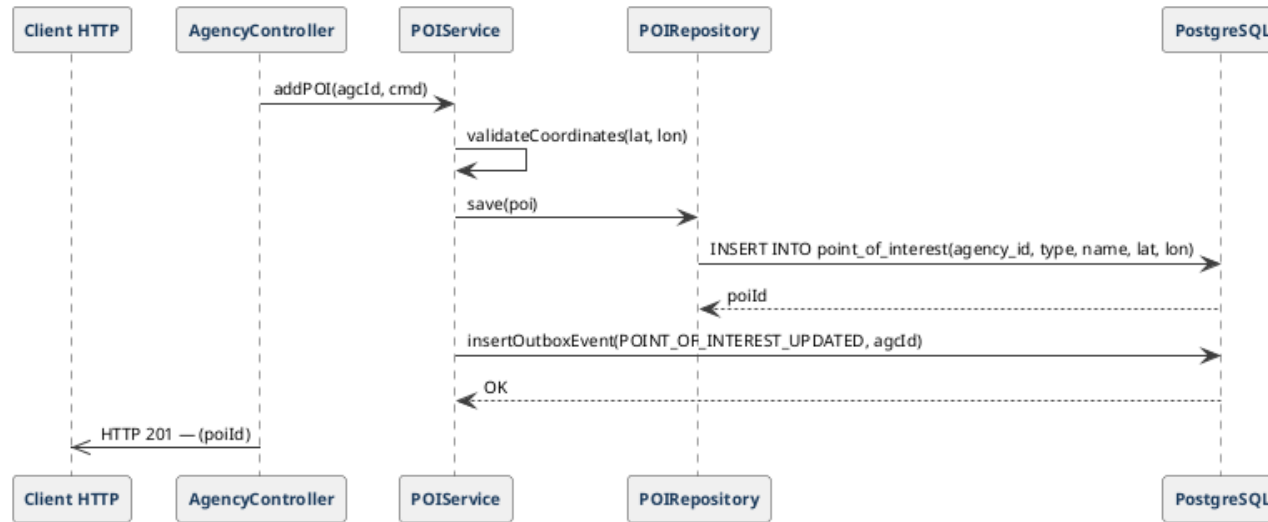
DS-O-04 — Lien fonctionnel entre un acteur et une organisation



DS-O-05 — Suspension d'une organisation et de ses agences



DS-O-06 — Ajout d'un point d'intérêt géographique à une agence



7 tp-core — Gestion des Tiers

7.1 Description du core

Le **tp-core** (Third-Parties Core) gère l'ensemble des entités externes avec lesquelles une organisation entretient des relations commerciales : clients, fournisseurs, agents commerciaux, et prospects. Un *ThirdParty* est une entité unifiée qui peut porter simultanément plusieurs qualifications (*rôles*) selon la nature de ses interactions avec l'organisation.

Ce core intègre également un module de gestion du cycle prospectif (*CRM léger*) : progression des prospects à travers les stades commerciaux, enregistrement des interactions, et conversion en client. Les données de qualification avancées (comptes bancaires, score de qualification, segment de marché, compte comptable) enrichissent le profil commercial des tiers.

7.2 Rôle principal

Rôle principal : Centraliser les informations sur les entités externes (clients, fournisseurs, prospects, agents), gérer leur cycle de qualification commerciale, et fournir les références stables utilisées par les cores opérationnels (ventes, comptabilité, trésorerie) pour identifier les contreparties.

7.3 Rôles secondaires

1. Multi-qualification d'un tiers

Un même tiers peut être simultanément client, fournisseur et agent commercial. Les rôles sont gérés comme une collection sur le tiers, sans duplication de données de base.

2. Gestion prospective (CRM)

Progression des prospects selon les stades **LEAD** → **QUALIFIED** → **NEGOTIATION** → **CUSTOMER**. Enregistrement des interactions (appels, e-mails, visites) avec date, type et résumé.

3. Conversion prospect → client

Opération atomique qui marque le prospect comme converti et ajoute le rôle

CLIENT au tiers.

4. **Gestion des comptes bancaires**

Enregistrement des coordonnées bancaires (IBAN, BIC, titulaire) pour les paiements et rapprochements, avec notion de compte principal.

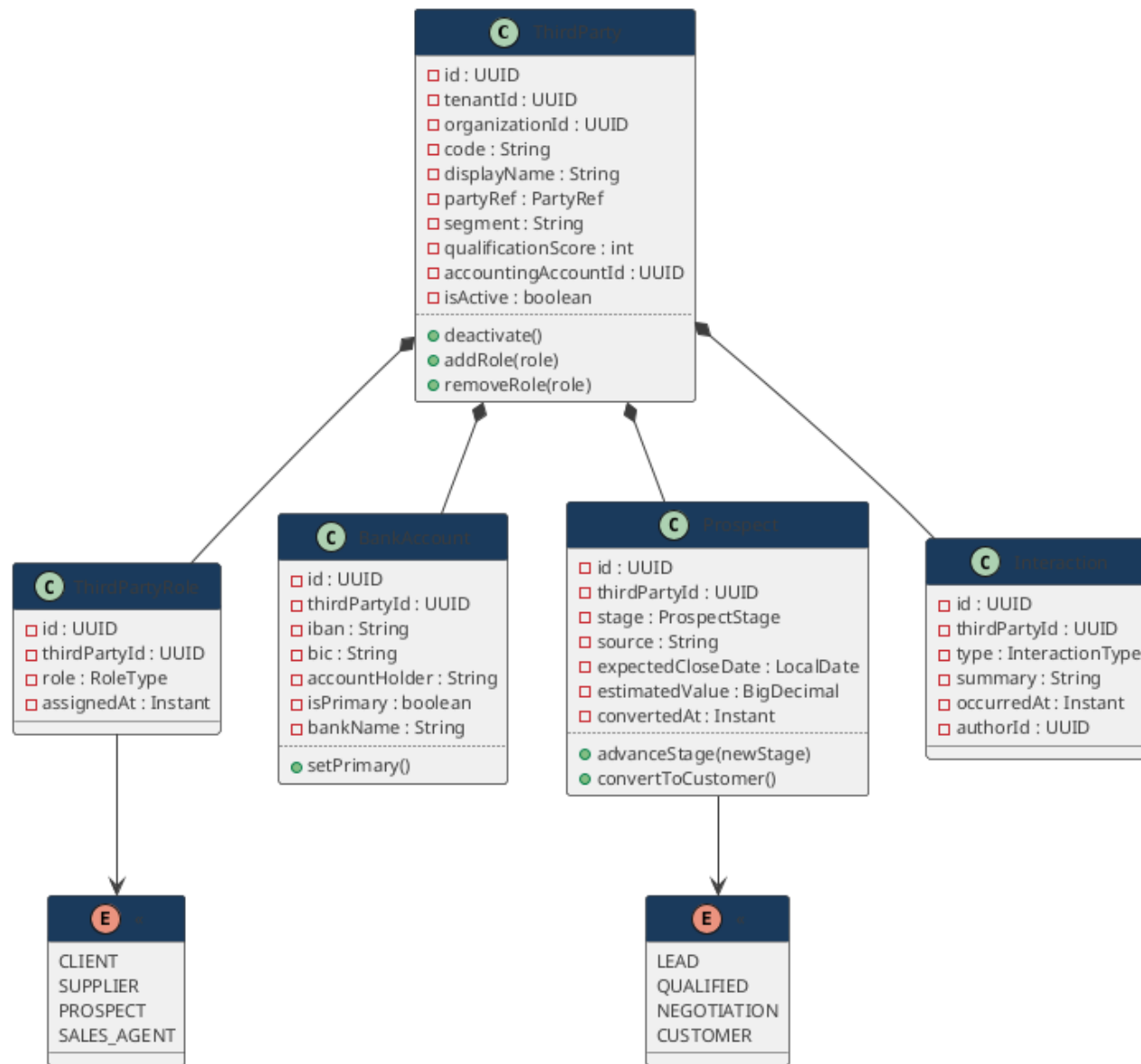
5. **Mapping comptable**

Association d'un compte du plan comptable (accounting-core) à chaque tiers pour l'imputation automatique lors de la facturation.

6. **Recherche plein-texte**

Projection Elasticsearch des tiers permettant une recherche par nom, code, rôle, segment, ou statut prospect.

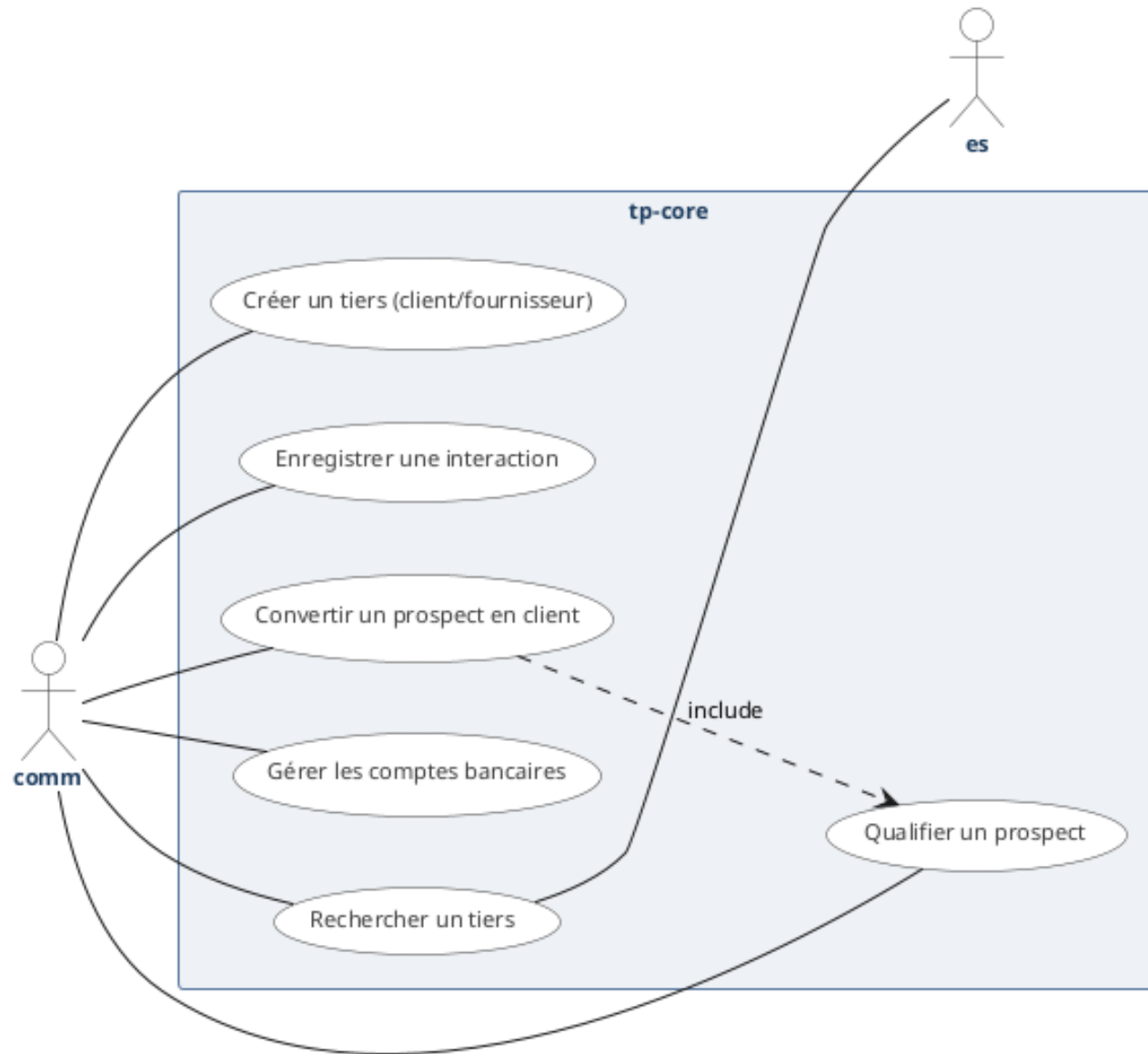
Diagramme de classes — tp-core



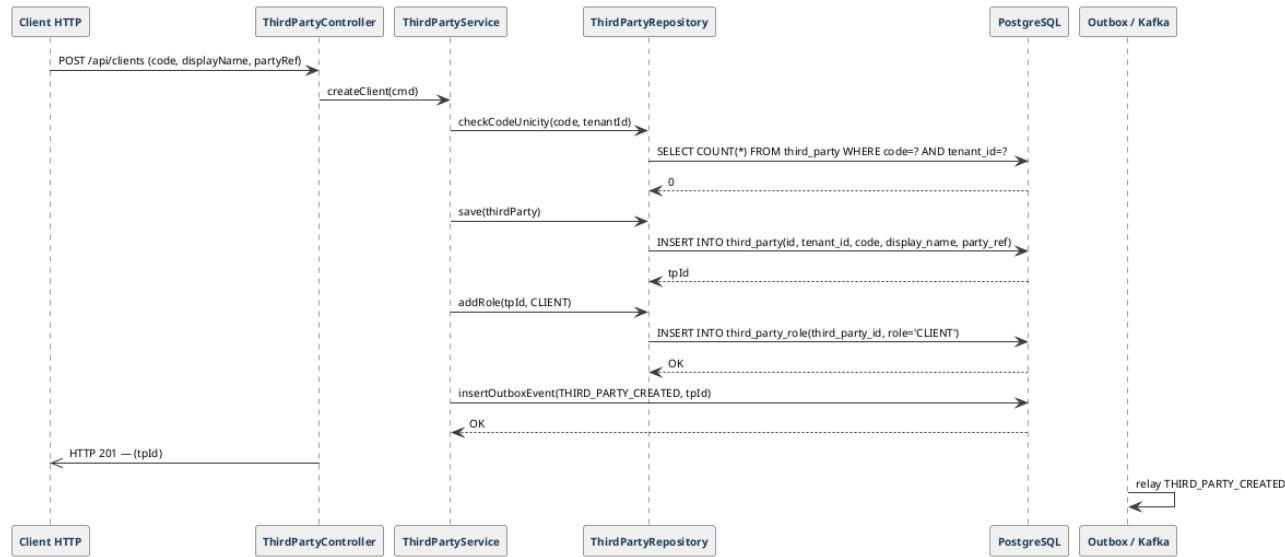
Invariants du tp-core :

- Le code d'un tiers est unique par tenant.
- Un tiers ne peut avoir qu'un seul **Prospect** actif.
- La progression de stade est strictement croissante (pas de retour en arrière).
- Un seul compte bancaire principal par tiers.
- Un tiers désactivé ne peut être référencé dans de nouvelles commandes.

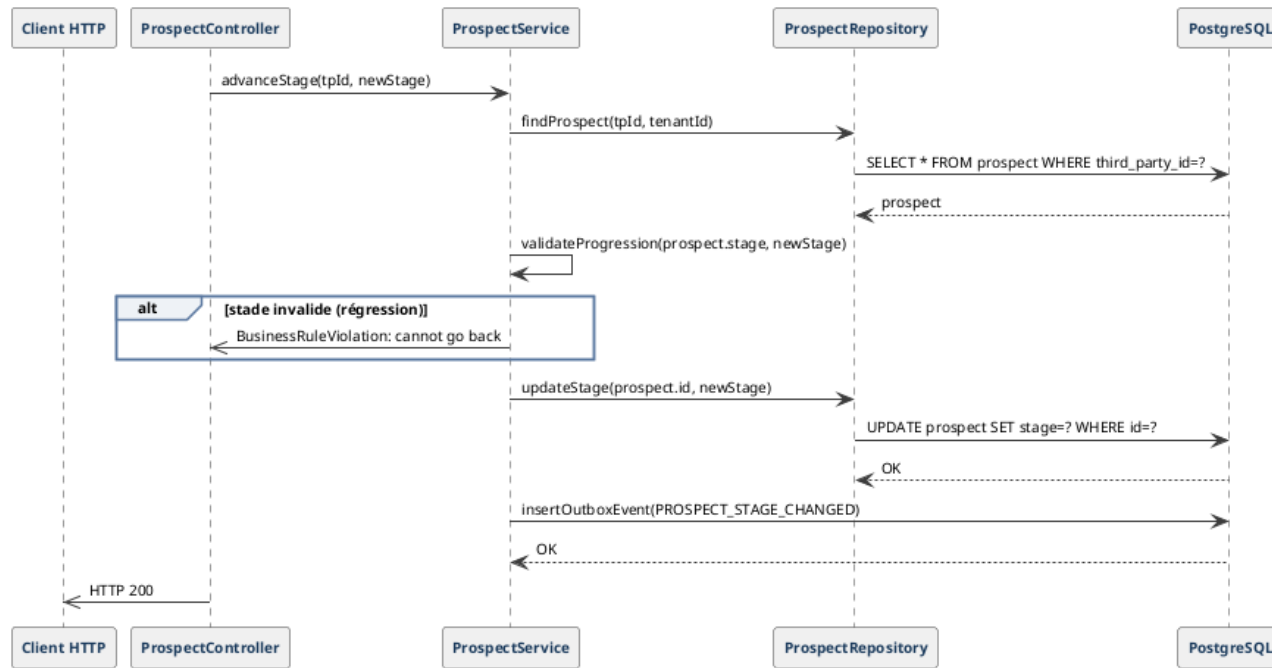
Diagramme des cas d'utilisation — tp-core



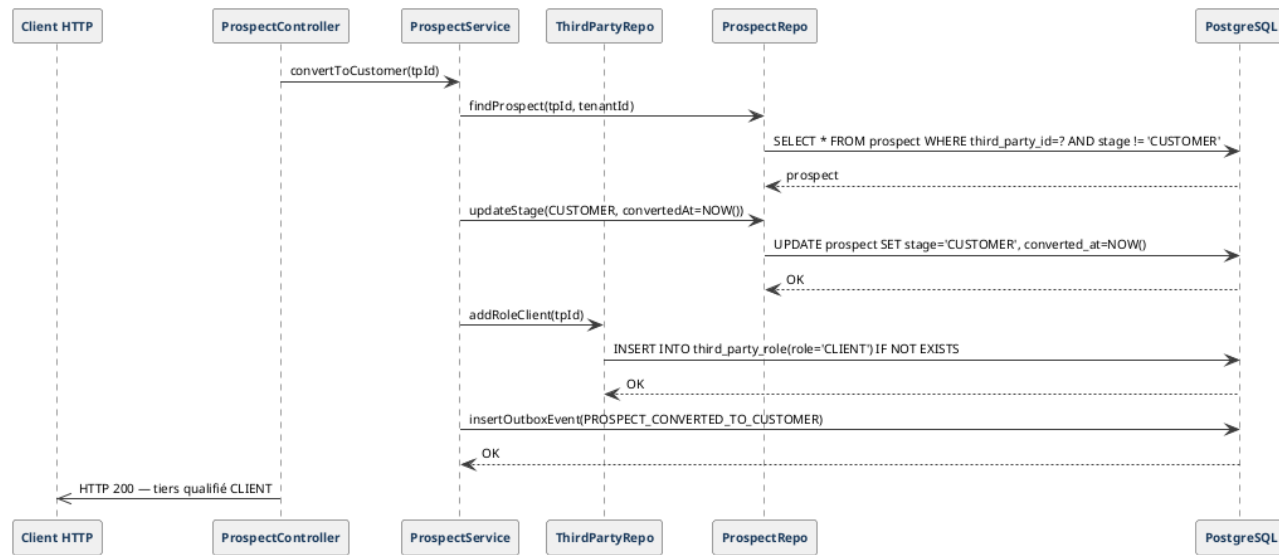
DS-TP-01 — Création d'un tiers avec rôle client



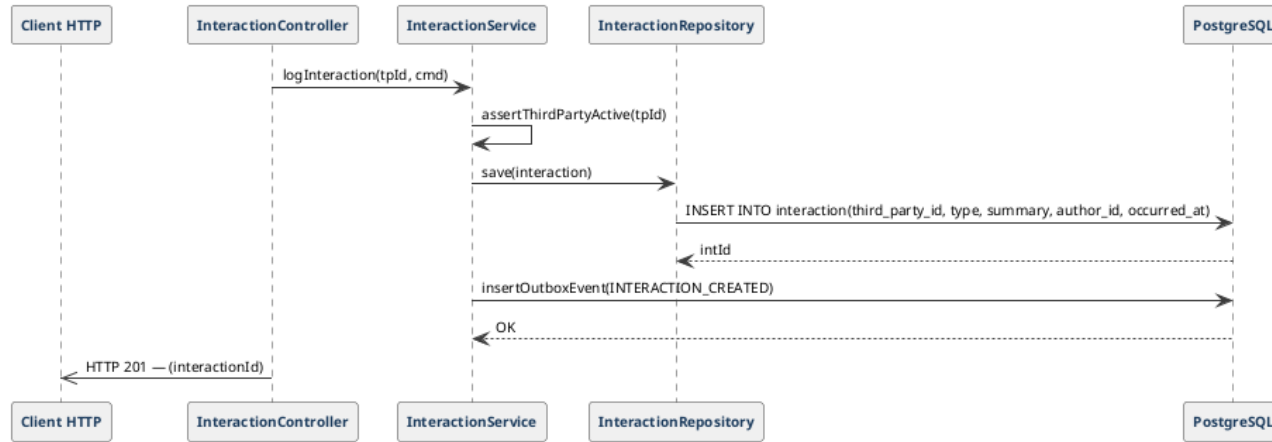
DS-TP-02 — Progression du stade commercial d'un prospect



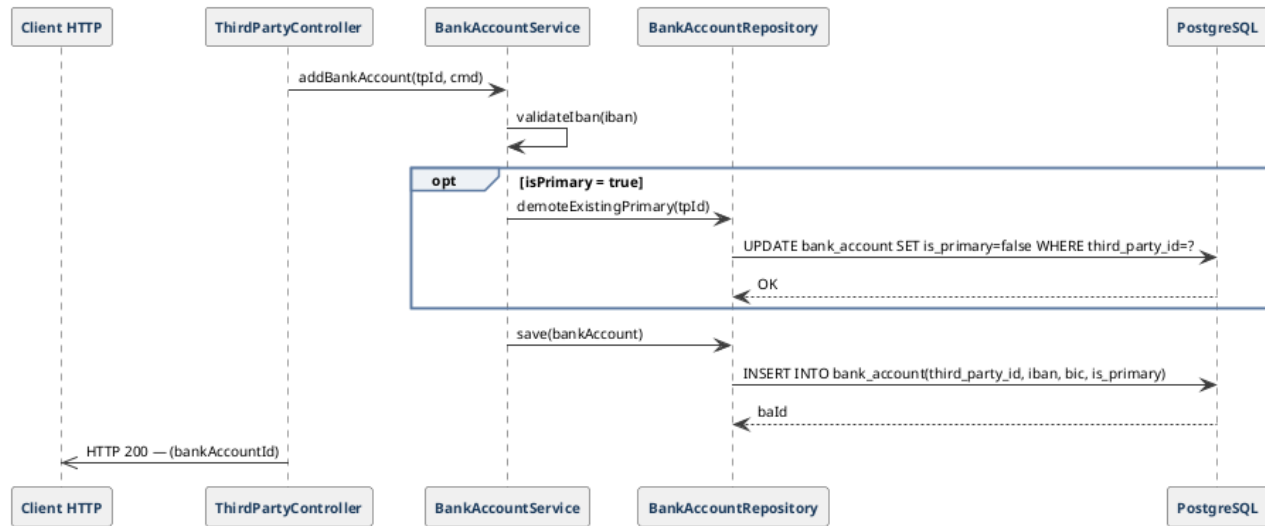
DS-TP-03 — Conversion d'un prospect en client



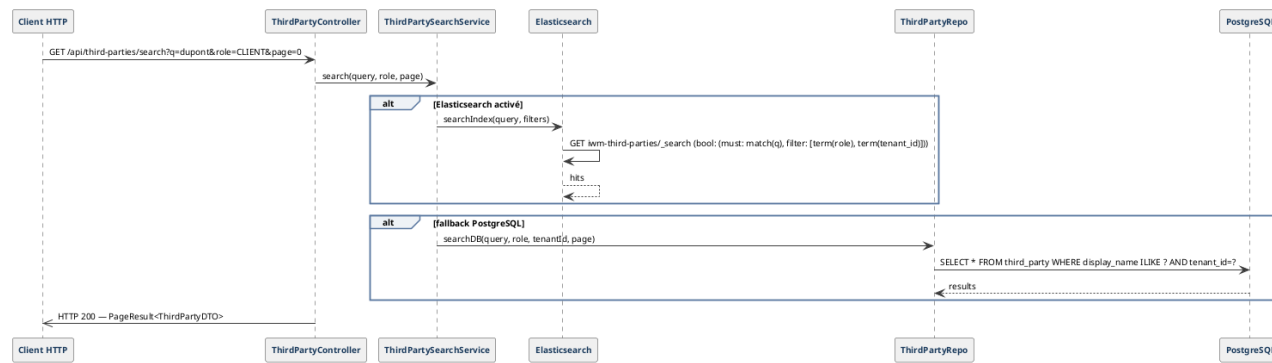
DS-TP-04 — Enregistrement d'une interaction commerciale



DS-TP-05 — Ajout d'un compte bancaire à un tiers



DS-TP-06 — Recherche plein-texte de tiers (Elasticsearch / fallback PostgreSQL)



8 settings-core

8.1 Description du core

Le **settings-core** centralise la configuration métier paramétrable et les séquences de numérotation documentaire. Il répond à deux besoins complémentaires : d'une part, stocker les paramètres fonctionnels propres à chaque organisation ou agence (devise par défaut, identité légale, politiques de validation) ; d'autre part, générer de manière atomique et sans collision les numéros de référence des documents métier (commandes, factures, mouvements de stock).

Le modèle d'héritage paramétrique est simple mais puissant : un paramètre défini au niveau de l'organisation s'applique à toutes ses agences, sauf si une agence surcharge explicitement ce paramètre.

8.2 Rôle principal

Rôle principal : Fournir un référentiel de configuration métier hiérarchique (organisation → agence) et un service de numérotation documentaire atomique garantissant l'unicité des références dans un contexte multi-tenant et multi-organisation.

8.3 Rôles secondaires

1. Configuration par organisation

Raison sociale, identifiant fiscal, devise de référence, et drapeaux de politiques (validation obligatoire des commandes, approbation des retours).

2. Override par agence

Une agence peut redéfinir certains paramètres sans écraser la configuration de son organisation. La résolution se fait en cascade : agence en priorité, organisation en repli.

3. Séquences documentaires atomiques

Chaque type de document (`ORDER`, `INVOICE`, `STOCK_MOVEMENT`, `CHECK`) dispose d'un compteur par (tenant, organisation, type, exercice fiscal). L'incrémentation est atomique — verrou applicatif Redis ou verrou de ligne PostgreSQL selon la configuration.

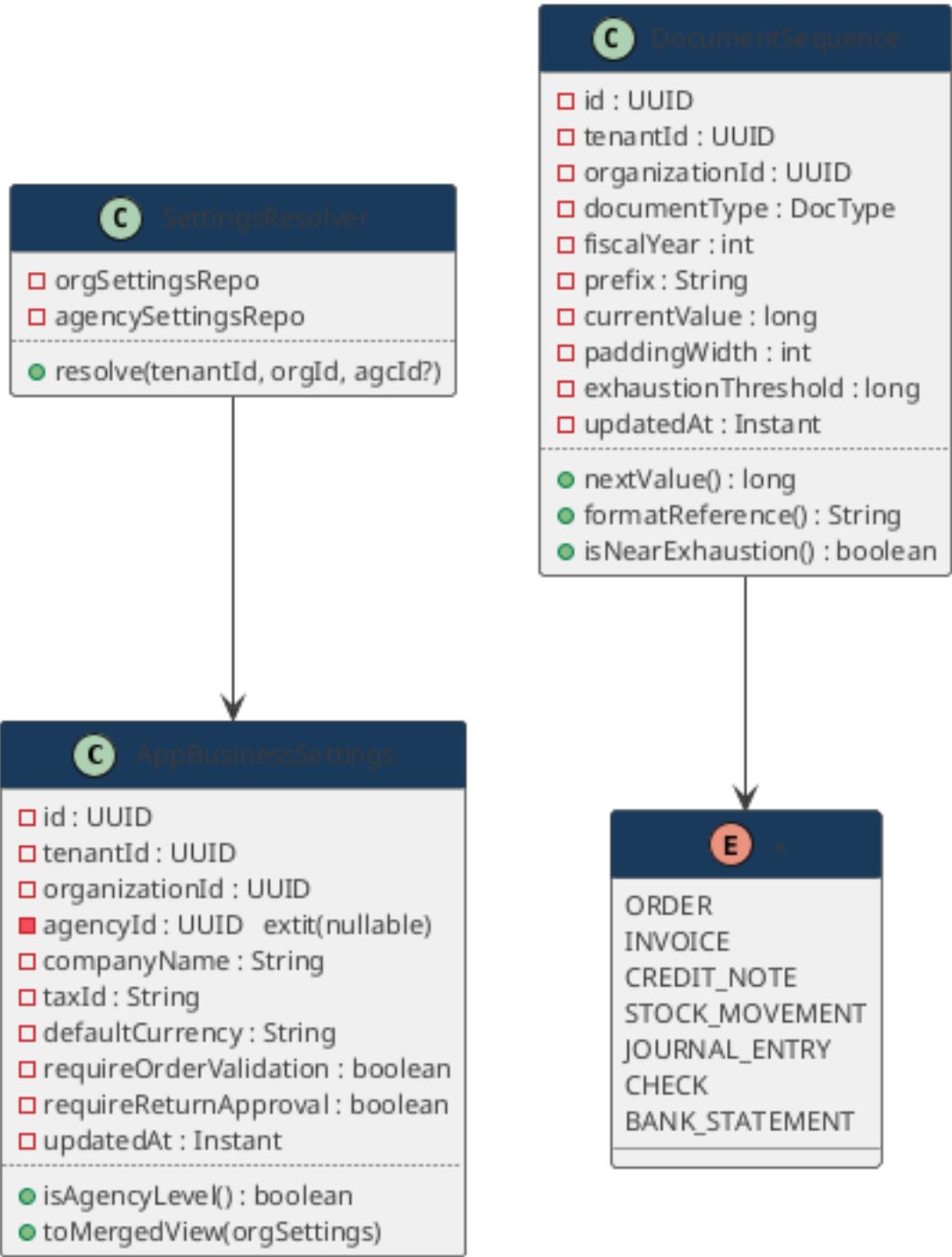
4. Format de numérotation configurable

Chaque séquence supporte un préfixe (ex. `CMD-`) et un rembourrage zéro (*zero-padding*, ex. `CMD-000042`).

5. Alertes d'épuisement

Lorsque le compteur d'une séquence dépasse un seuil configurable, un événement `SEQUENCE_EXHAUSTED` est émis pour avertir les administrateurs.

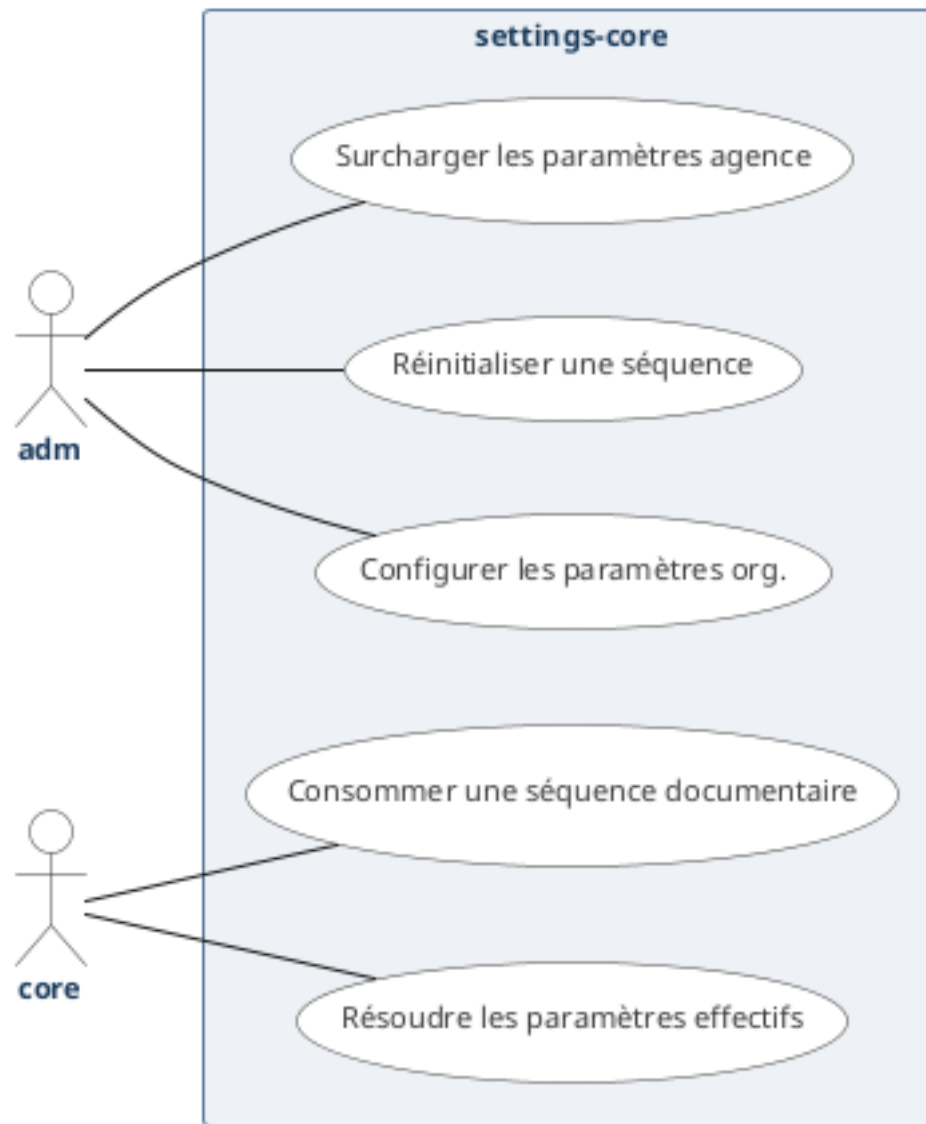
Diagramme de classes — settings-core



Invariants du settings-core :

- Paramètres organisation : unique par (tenantId, organizationId) avec agencyId NULL.
- Paramètres agence : unique par (tenantId, organizationId, agencyId).
- Séquence : unique par (tenantId, organizationId, documentType, fiscalYear).
- `currentValue` ≥ 0 et `paddingWidth` ≥ 0 .
- L'incrément de séquence est atomique (pas de doublon de référence possible).

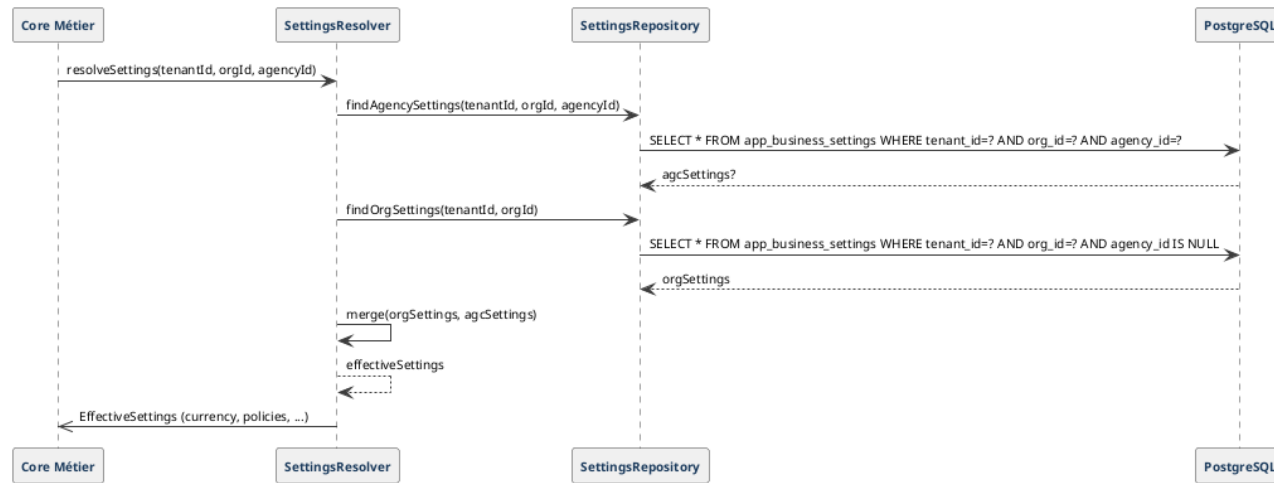
Diagramme des cas d'utilisation — settings-core



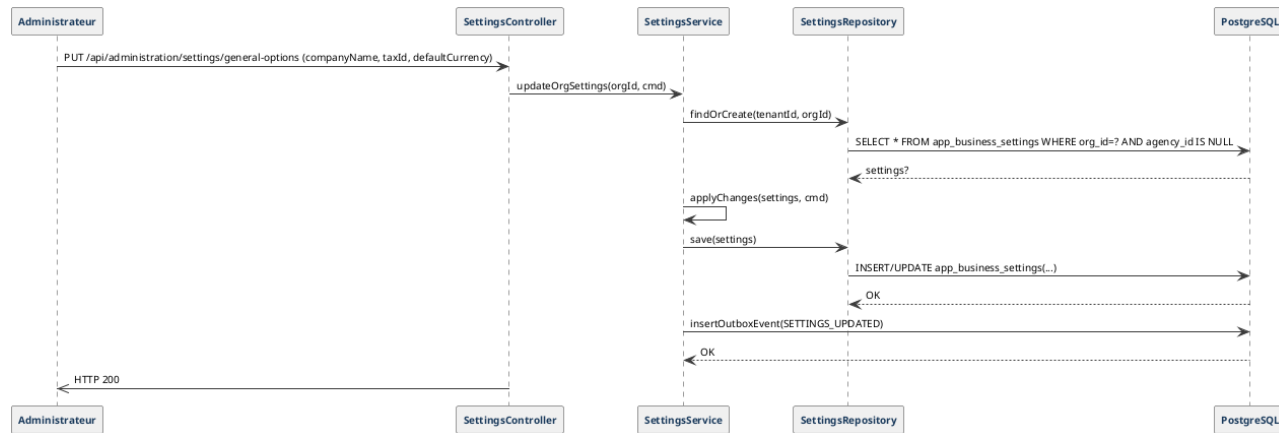
DS-S-01 — Consommation atomique d'une séquence documentaire



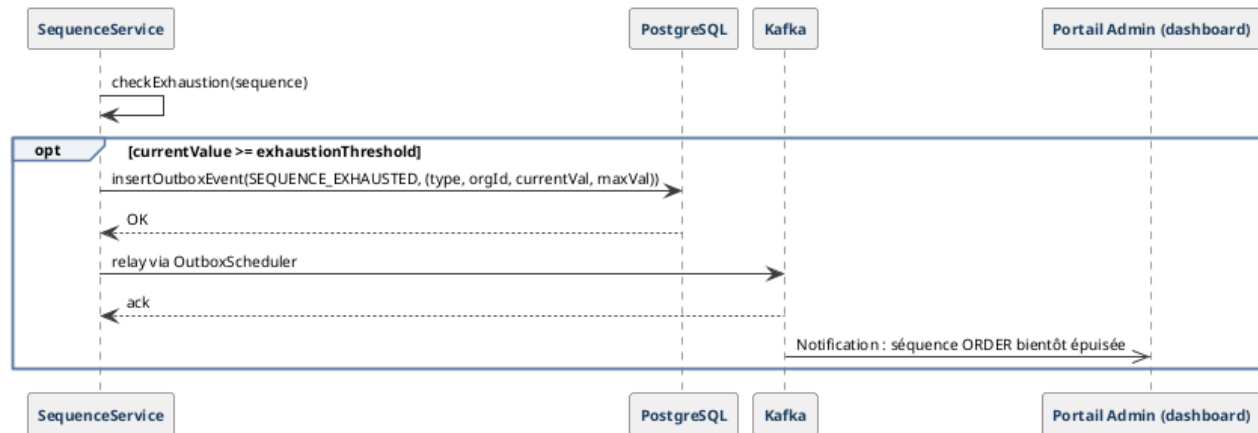
DS-S-02 — Résolution des paramètres par héritage organisation-agence



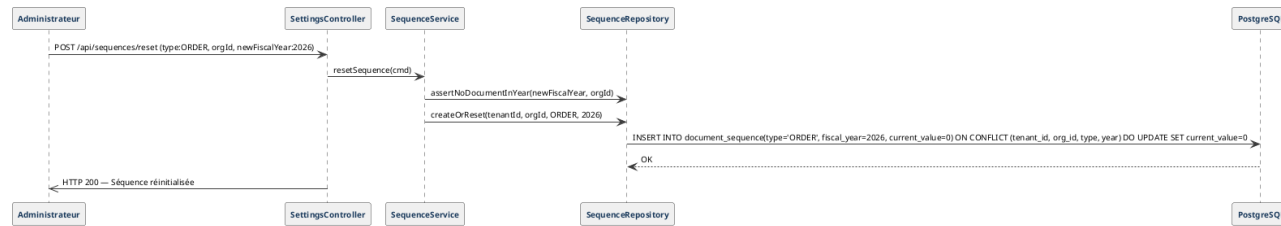
DS-S-03 — Configuration des paramètres d'une organisation



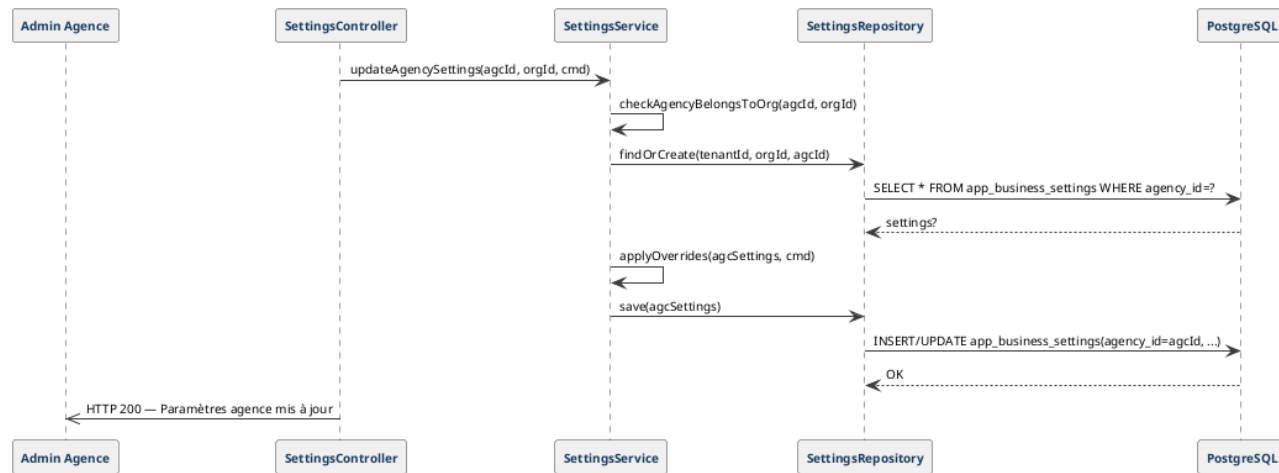
DS-S-04 — Alerte d'épuisement de séquence documentaire



DS-S-05 — Réinitialisation de séquence pour un nouvel exercice fiscal



DS-S-06 — Override de paramètres au niveau agence



9 product-core — Catalogue Produits

9.1 Description du core

Le **product-core** est le référentiel de catalogue de la plateforme. Il gère la définition des produits, leur organisation en catégories hiérarchiques, les variantes (SKU), les lots de fabrication, la tarification datée, et les médias associés. Par conception, ce core ne gère **aucun stock** : les quantités disponibles relèvent exclusivement de l'inventory-core.

Le modèle supporte les cas d'usage industriels les plus courants : produits multi-variantes (taille, couleur, matière), fiches nutritionnelles pour les produits alimentaires, spécifications techniques, gestion multi-langue des descriptions.

9.2 Rôle principal

Rôle principal : Maintenir le catalogue des produits commercialisés, de leurs variantes ordonnables et de leur tarification, et fournir aux cores aval (ventes, stocks) des snapshots stables des données produit au moment des transactions.

9.3 Rôles secondaires

1. Organisation en catégories hiérarchiques

Taxonomie arborescente de catégories avec support multi-langue (`CategoryI18n`). Les catégories peuvent être imbriquées à profondeur libre.

2. Gestion des variantes (SKU)

Chaque produit peut avoir plusieurs variantes caractérisées par des attributs (taille, couleur). Chaque variante possède un code SKU unique et constitue l'unité orderable atomique.

3. Traçabilité par lot (Batch)

Gestion des lots de fabrication avec numéro de lot, date de fabrication et date d'expiration. Essentiel pour les secteurs pharmaceutique et alimentaire.

4. Tarification datée

Les prix sont versionnés avec une date d'effet. Le prix effectif d'une variante à une date donnée est le prix le plus récent dont la date d'effet est antérieure ou égale à cette date.

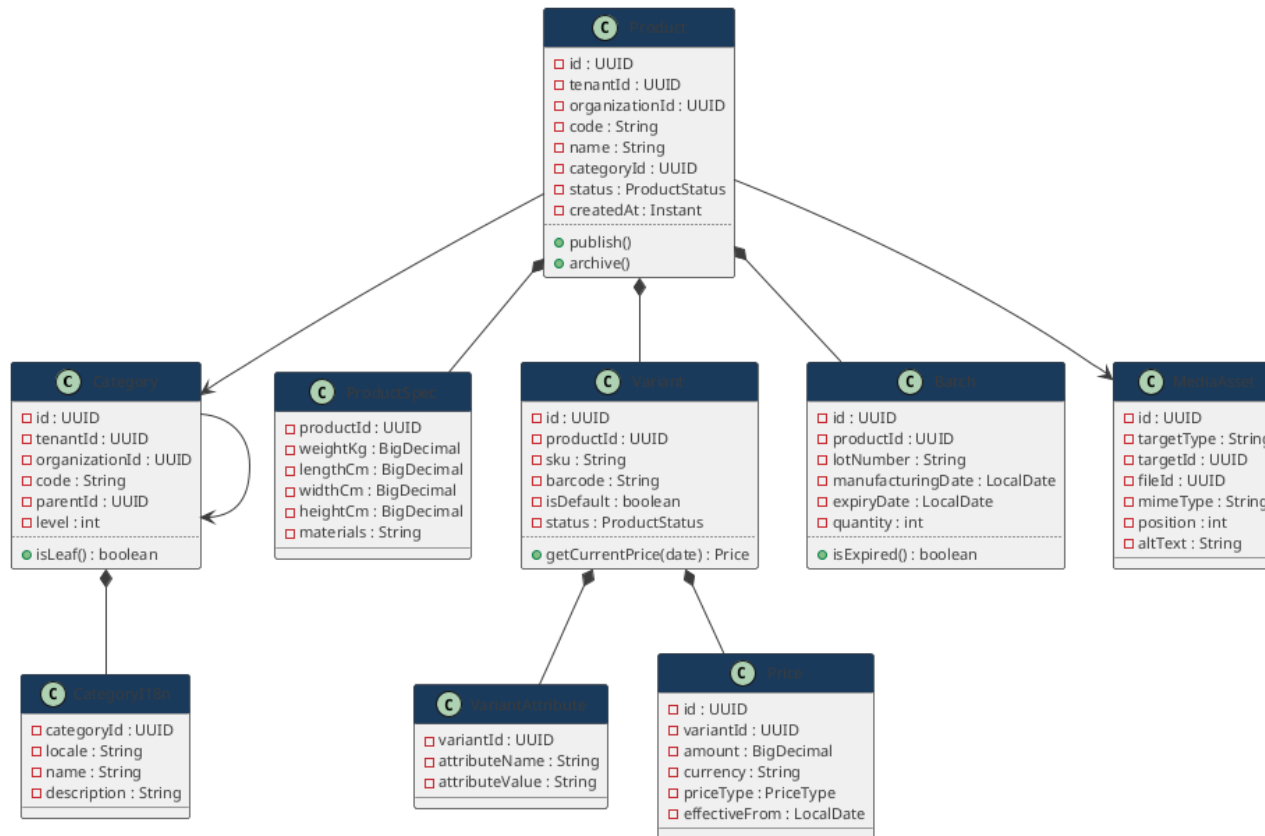
5. **Médias et assets**

Association d'images, fiches techniques, et autres documents à un produit ou une variante, avec gestion du type MIME et de la position d'affichage.

6. **Index de recherche Elasticsearch**

Projection des produits dans un index de recherche plein-texte pour les cas d'usage de catalogue avec filtres par famille, statut et attributs.

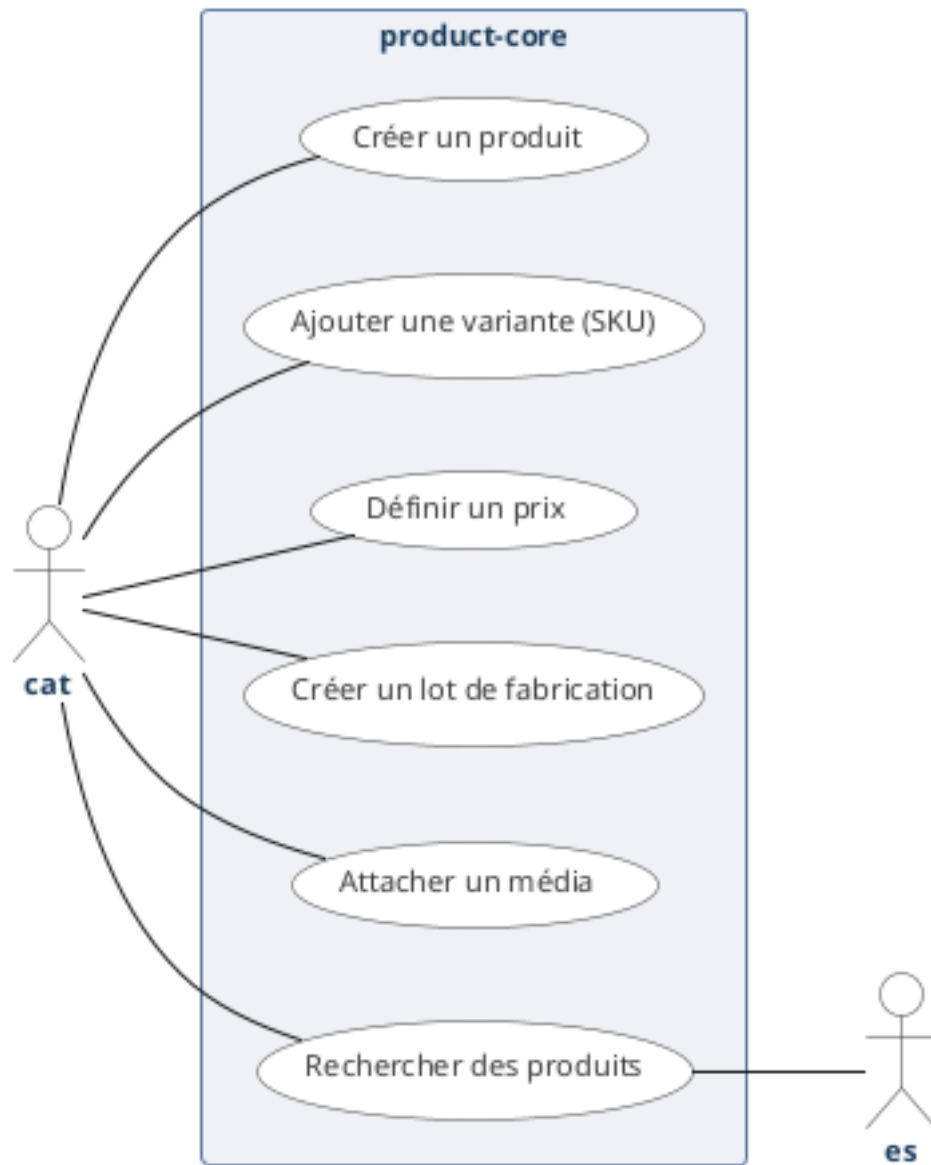
Diagramme de classes — product-core



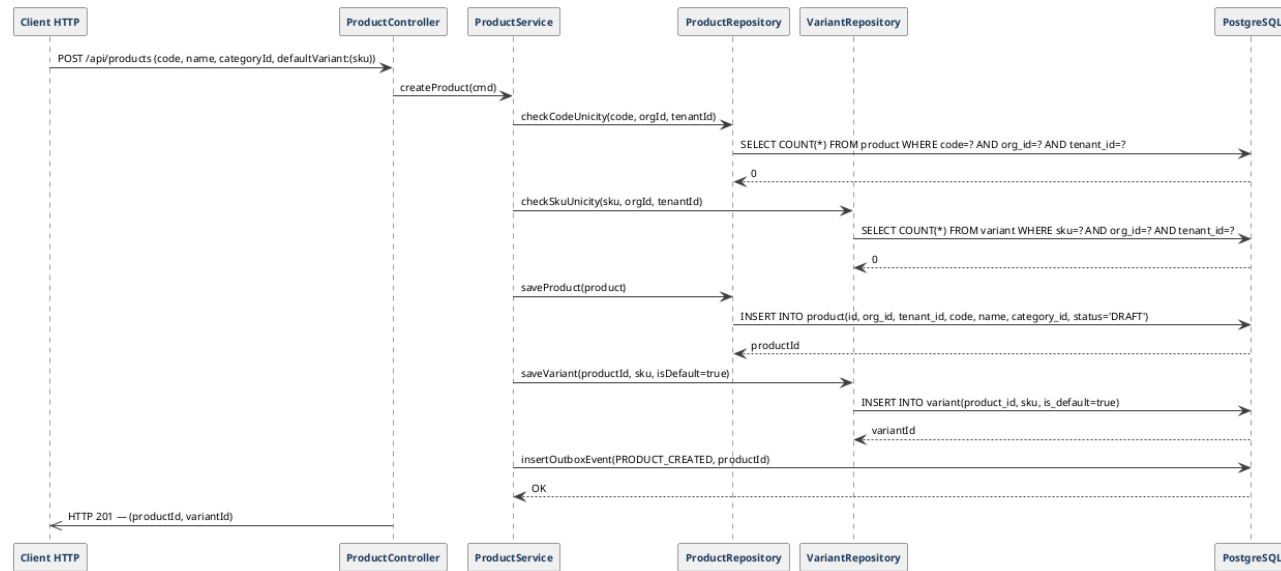
Invariants du product-core :

- Le code produit est unique par (tenantId, organizationId).
- Le SKU est unique par (tenantId, organizationId).
- Le numéro de lot est unique par produit.
- Chaque variante a exactement un prix effectif par type et par date d'évaluation.
- Le stock n'est **jamais** géré dans ce core.

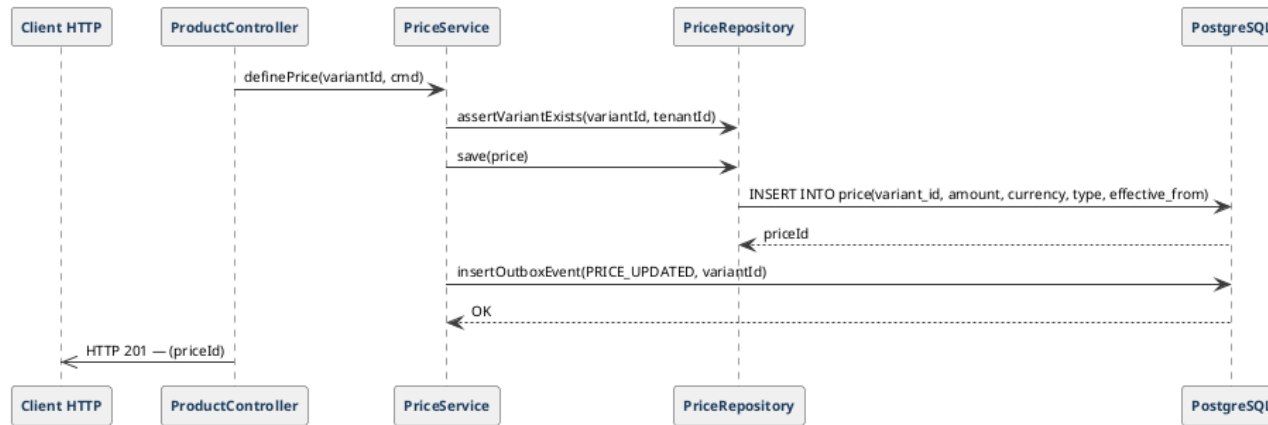
Diagramme des cas d'utilisation — product-core



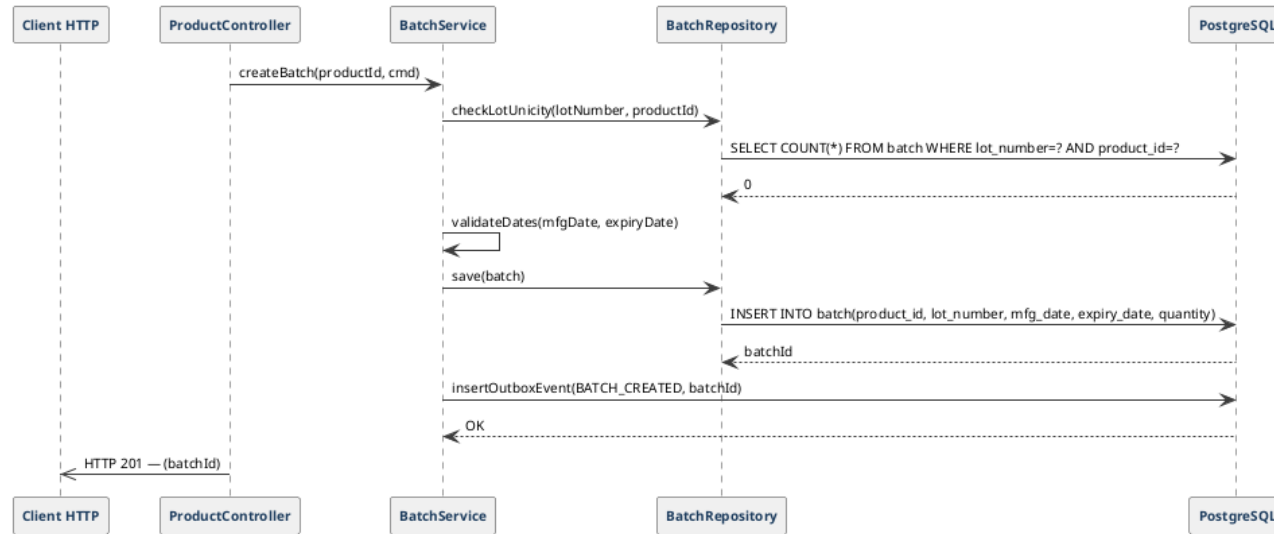
DS-P-01 — Création d'un produit avec variante par défaut



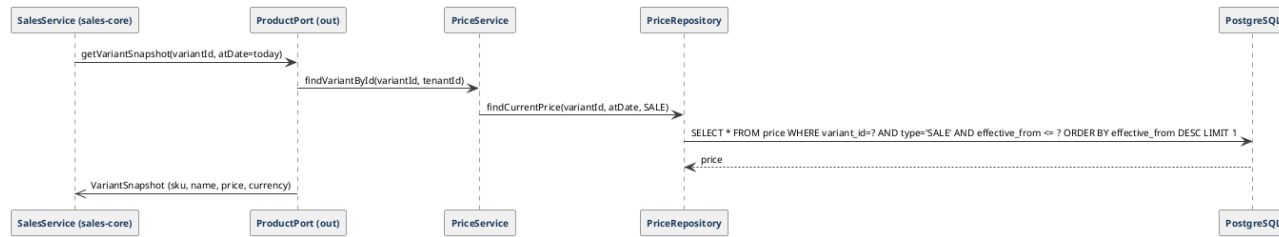
DS-P-02 — Définition d'un prix pour une variante



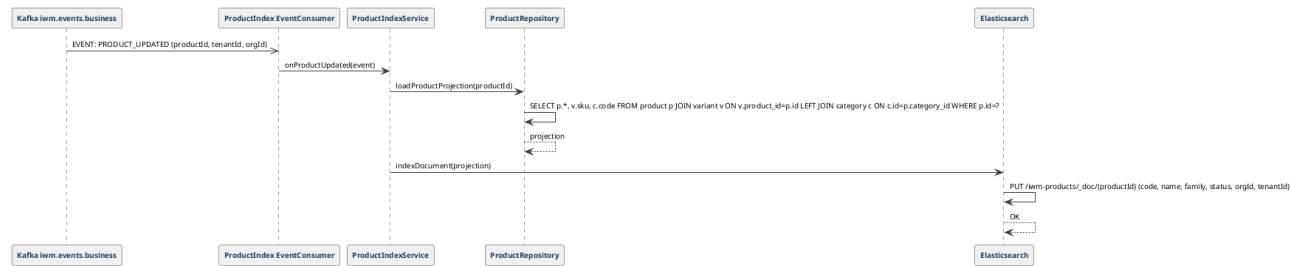
DS-P-03 — Création d'un lot de fabrication



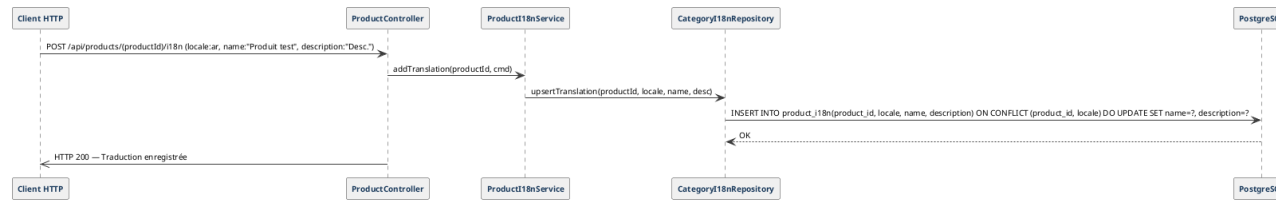
DS-P-04 — Résolution du prix effectif d'une variante à une date



DS-P-05 — Mise à jour de l'index Elasticsearch après modification produit



DS-P-06 — Ajout d'une traduction multi-langue à un produit



10 inventory-core — Gestion des Stocks

10.1 Description du core

L'**inventory-core** est le responsable unique de la vérité des quantités en stock. Il enregistre tous les mouvements de marchandises (entrées, sorties, transferts, ajustements) et maintient les niveaux de stock courants par combinaison (agence, variante, lot). Il reçoit des sollicitations des autres cores via des événements (confirmation de commande déclenche une dispatch stock) et peut émettre des alertes de stock bas.

Le core gère également les inventaires physiques (comptages) permettant de réconcilier le stock théorique avec le stock réel, ainsi que les transformations de produits (recettes : des variantes d'entrée produisent des variantes de sortie).

10.2 Rôle principal

Rôle principal : Maintenir en temps réel les niveaux de stock par agence, variante et lot, enregistrer tous les mouvements avec leur traçabilité documentaire, et garantir la cohérence des quantités disponibles lors des allocations de commandes.

10.3 Rôles secondaires

1. Traçabilité des mouvements

Chaque mouvement (entrée fournisseur, sortie client, ajustement) référence son document source (`sourceDocumentType`, `sourceDocumentNumber`) permettant une traçabilité complète de l'origine de chaque variation de stock.

2. Allocation à la confirmation de commande

À la réception de l'événement `SALES_ORDER_CONFIRMED`, le core vérifie la disponibilité stock, alloue les quantités et crée un mouvement de sortie de type commande.

3. Inventaires physiques

Processus de comptage permettant de corriger les écarts entre stock théorique et réel. Un inventaire validé est immuable et génère des mouvements d'ajustement.

4. Transformations de produits

Modélisation des recettes de production : consommation de variantes d'entrée pour produire des variantes de sortie. Chaque transformation validée génère les mouvements de stock correspondants.

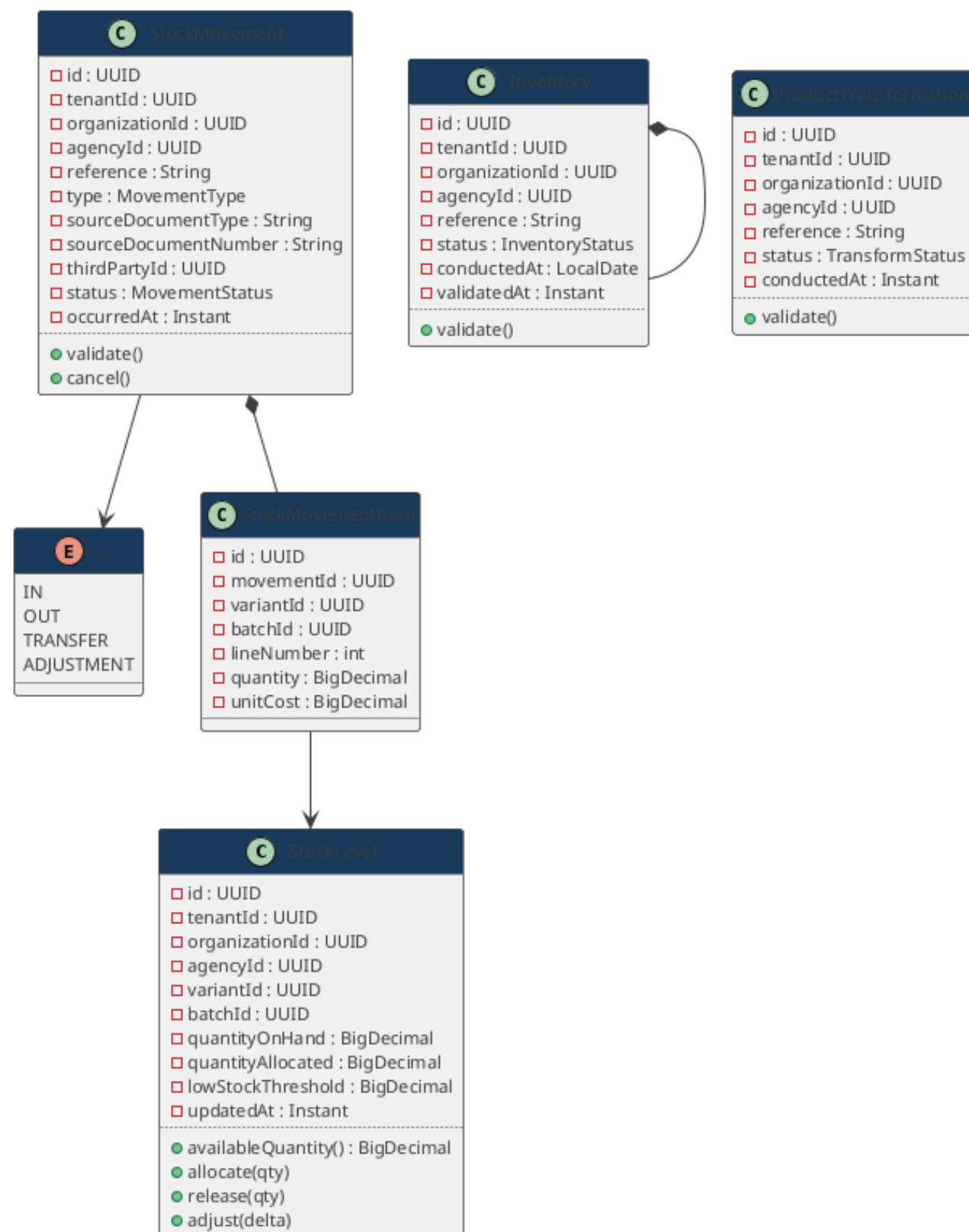
5. Transferts inter-agences

Mouvement de type `TRANSFER` générant simultanément une sortie de l'agence source et une entrée dans l'agence destination.

6. Alertes de stock bas

Émission d'un événement `LOW_STOCK_ALERT` lorsque le niveau de stock passe en dessous d'un seuil configurable.

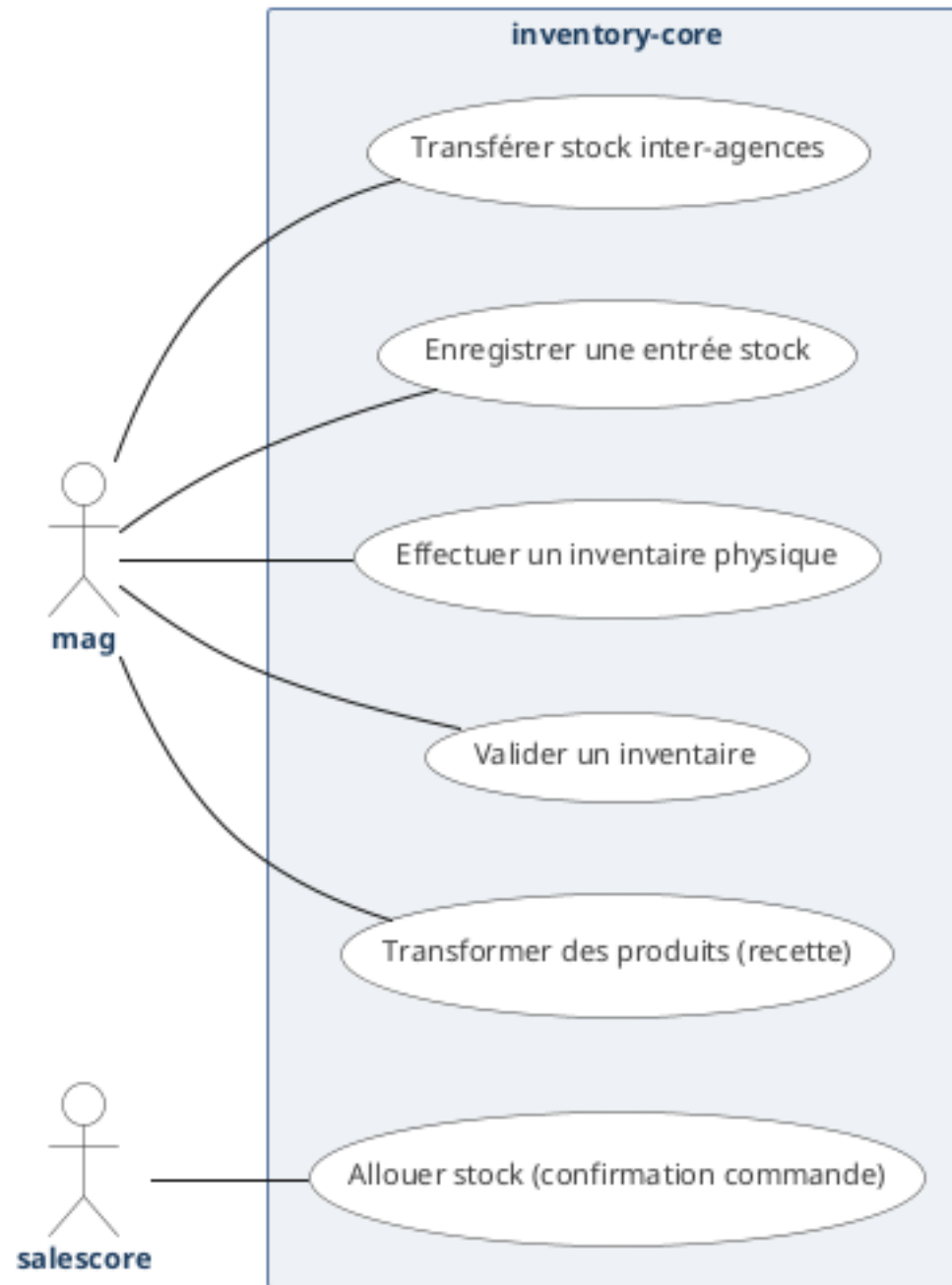
Diagramme de classes — inventory-core



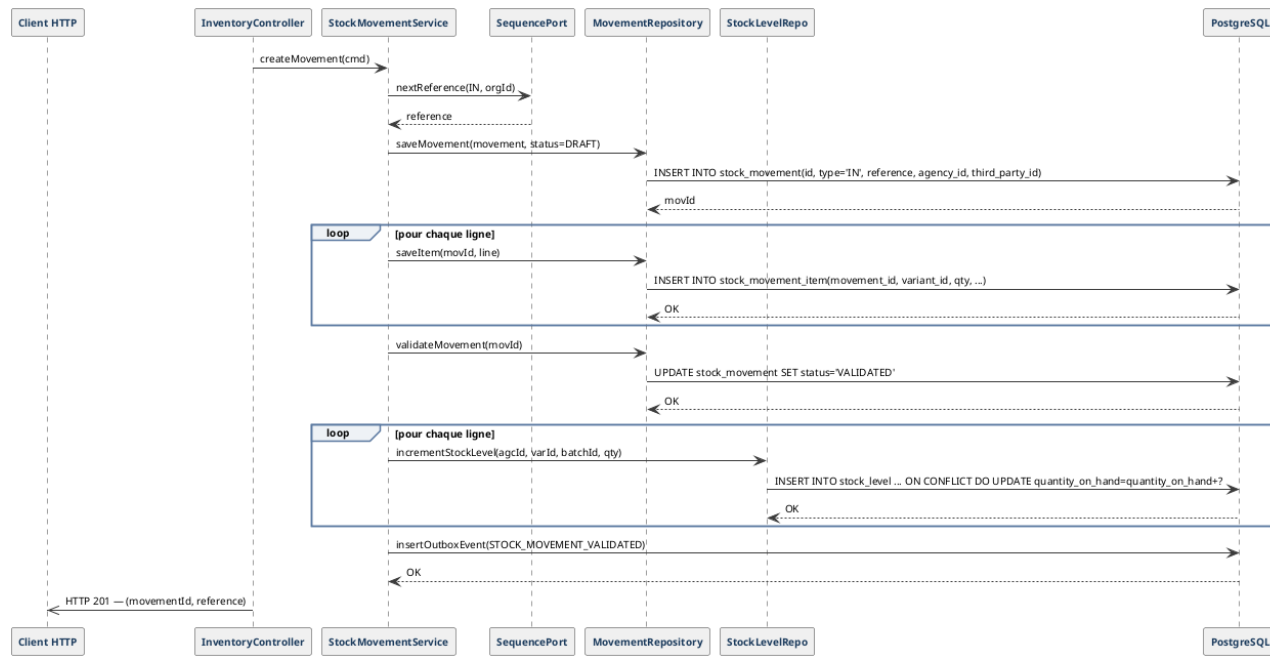
Invariants du inventory-core :

- `StockLevel` est unique par (`tenantId`, `organizationId`, `agencyId`, `variantId`, `batchId`).
- $\text{quantityOnHand} \geq \text{quantityAllocated} \geq 0$.
- Un mouvement validé est immuable.
- Un inventaire validé est immuable.
- Un numéro de ligne est unique au sein d'un mouvement.

Diagramme des cas d'utilisation — inventory-core



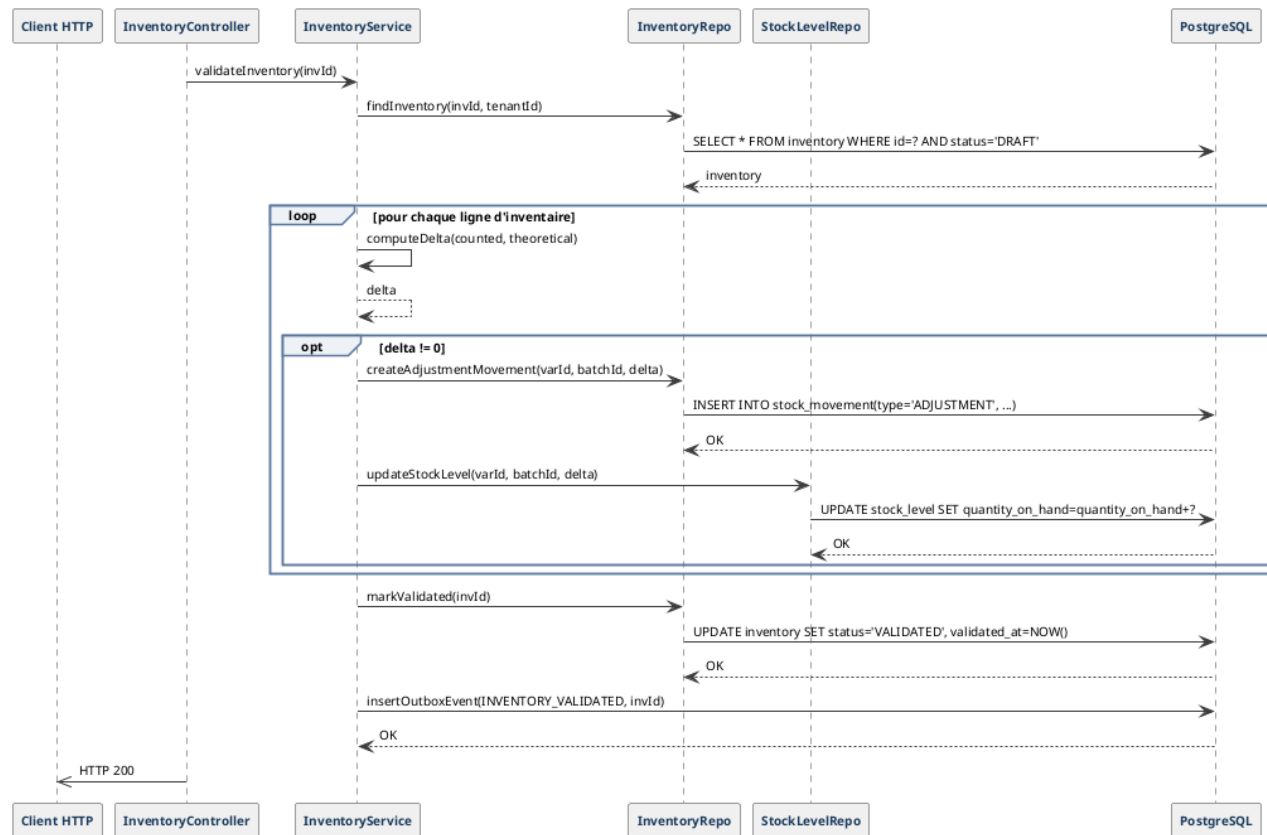
DS-I-01 — Enregistrement d'une entrée de stock (réception fournisseur)



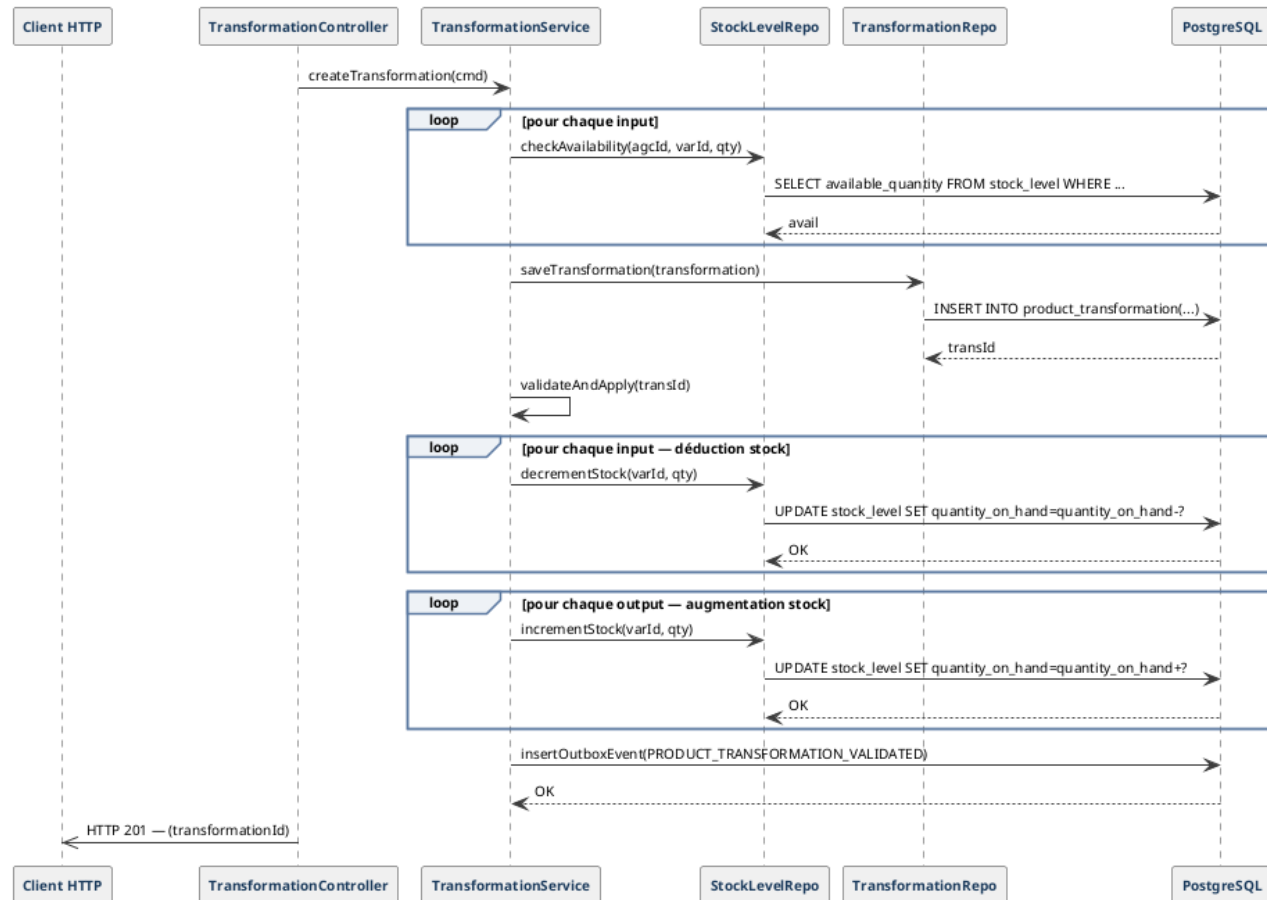
DS-I-02 — Allocation de stock à la confirmation de commande



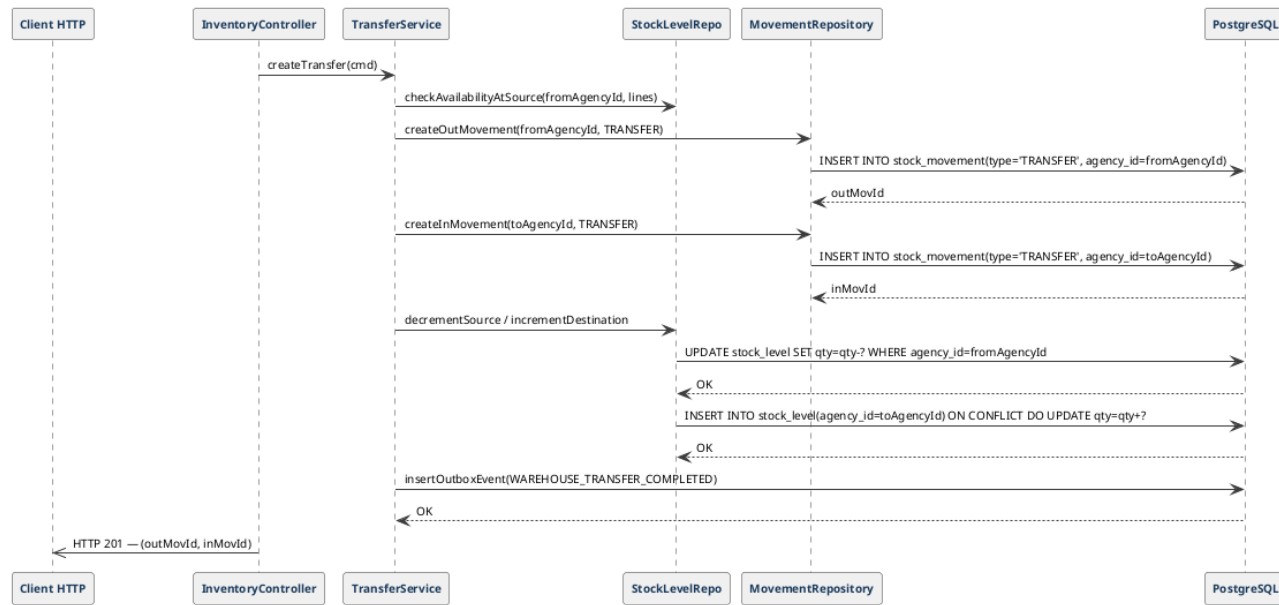
DS-I-03 — Validation d'un inventaire physique avec ajustements



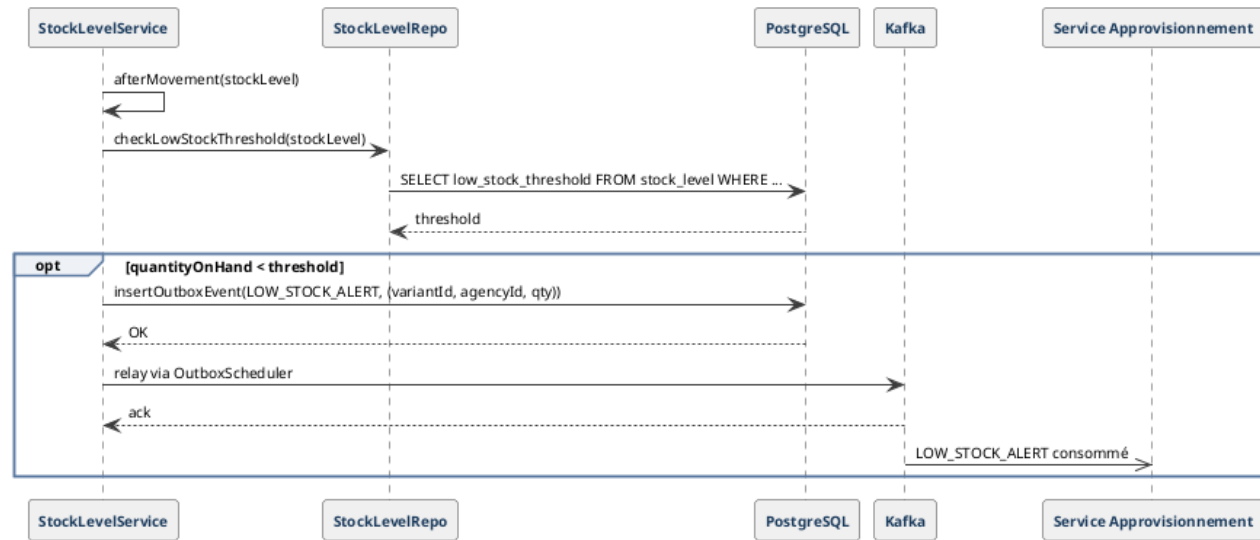
DS-I-04 — Transformation de produits (exécution d'une recette)



DS-I-05 — Transfert de stock entre deux agences



DS-I-06 — Alerte de stock bas après mouvement



11 resource-core — Ressources Matérielles

11.1 Description du core

Le **resource-core** gère les actifs matériels internes de la plateforme : équipements, véhicules, dispositifs informatiques, outillage. Chaque ressource possède un cycle de vie complet — opérationnelle, en maintenance, mise au rebut — et peut être affectée à un acteur, une agence ou une organisation.

Ce core fournit également un historique de localisation GPS (utile pour les flottes de véhicules), la gestion des interfaces réseau (adresses MAC et IP), et le journal de maintenance permettant de suivre les interventions techniques.

11.2 Rôle principal

Rôle principal : Gérer le cycle de vie complet des actifs matériels internes (création, affectation, maintenance, mise au rebut), maintenir leur localisation et leurs informations réseau, et assurer la traçabilité de chaque affectation dans un contexte multi-organisation.

11.3 Rôles secondaires

1. Affectation avec unicité d'affectation active

Une ressource ne peut avoir qu'une seule affectation active à la fois. L'affectation peut être à un acteur, une agence ou une organisation. Toutes les affectations historiques sont conservées pour la traçabilité.

2. Cycle de maintenance

Transition d'état contrôlée vers `IN_MAINTENANCE`, empêchant toute nouvelle affectation standard. À la fin de la maintenance, la ressource repasse à `OPERATIONAL`.

3. Traçabilité GPS

Enregistrement horodaté des positions géographiques permettant de reconstituer l'historique de déplacement d'un actif (flottes de véhicules, équipements mobiles).

4. Interfaces réseau

Association d'adresses MAC et IP à une ressource, avec notion d'interface principale — utile pour les inventaires réseau.

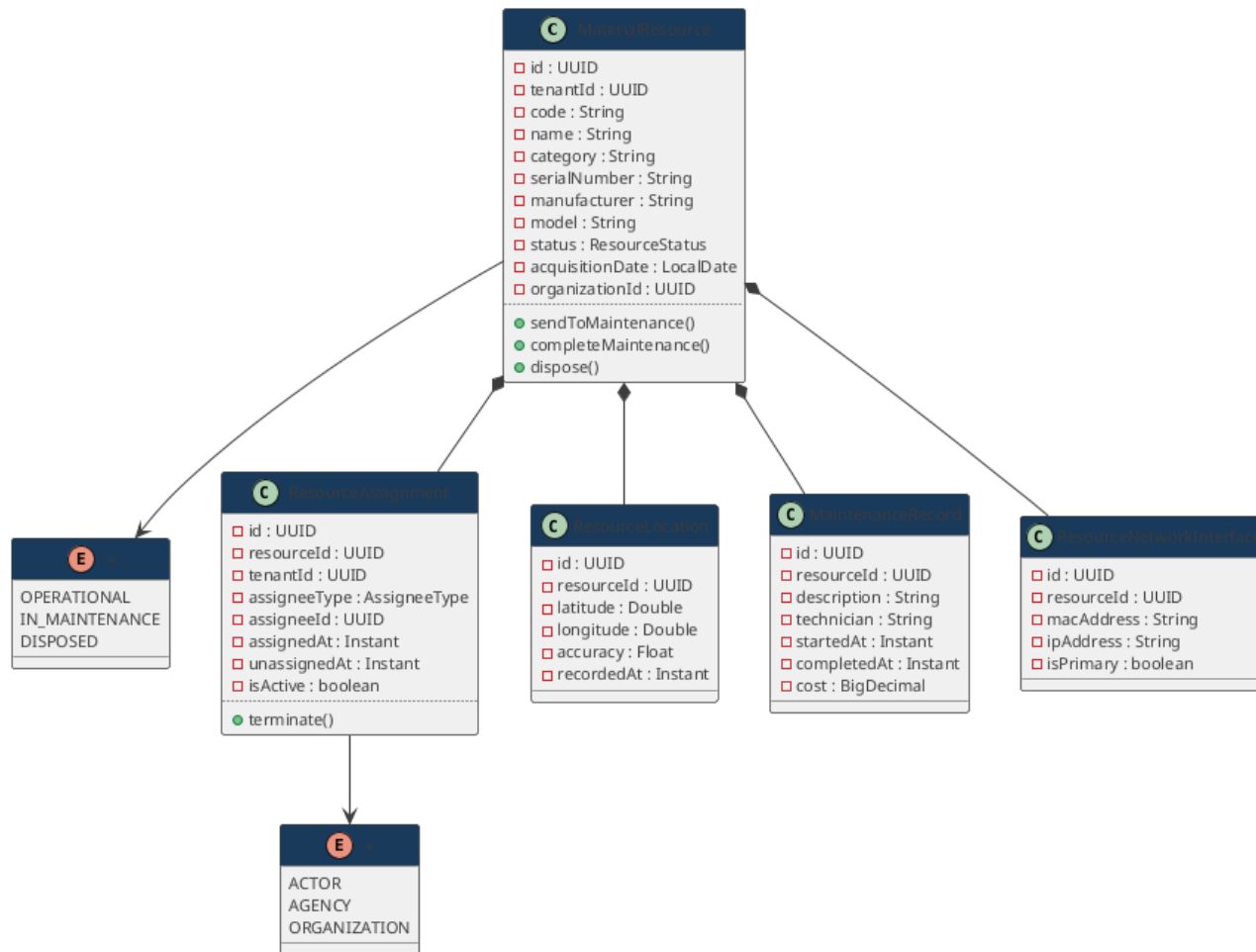
5. Recherche par critères

Recherche de ressources par catégorie, statut, agence d'affectation ou terme libre, avec projection Elasticsearch optionnelle.

6. Mise au rebut irréversible

L'état `DISPOSED` est terminal. Aucune nouvelle affectation ni maintenance ne peut être enregistrée pour une ressource mise au rebut.

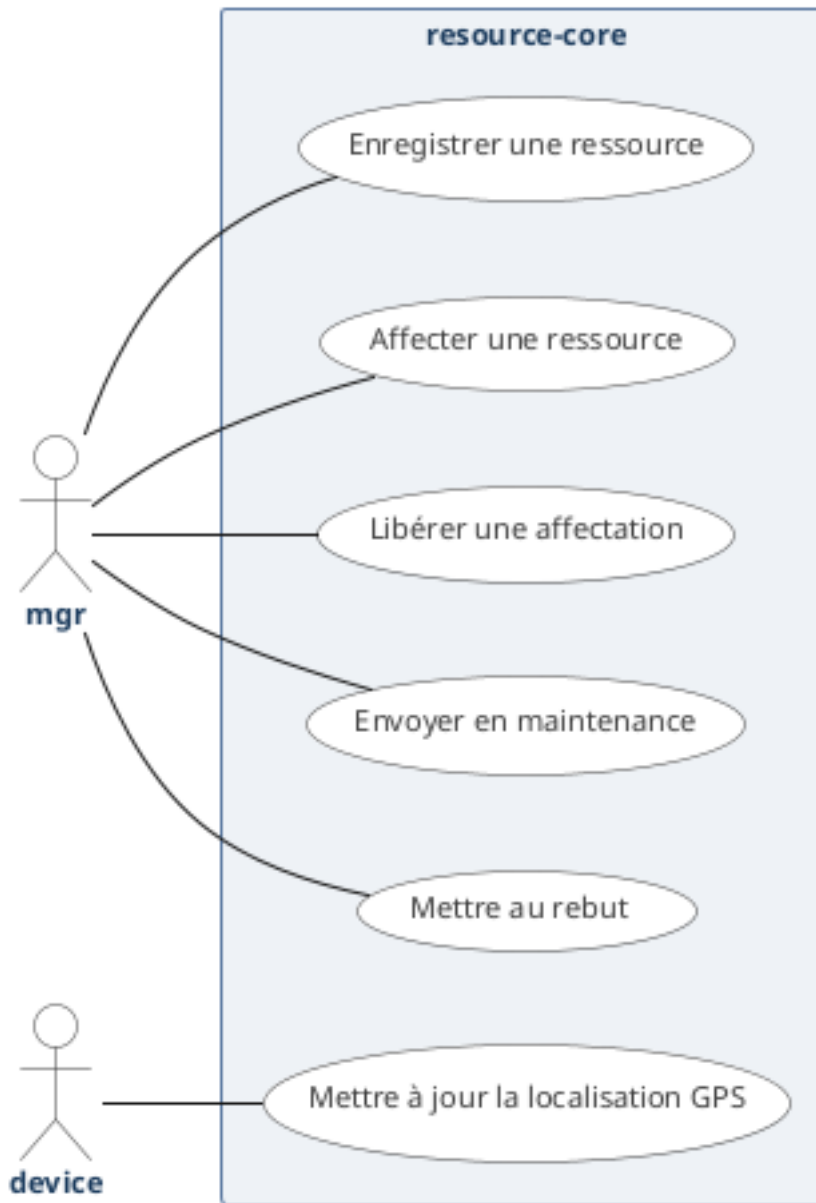
Diagramme de classes — resource-core



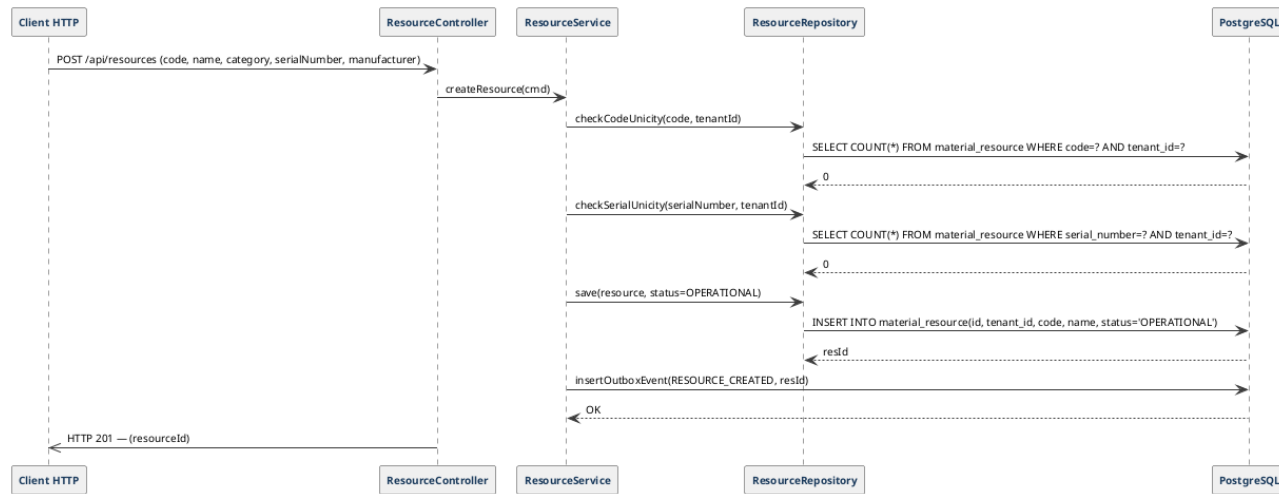
Invariants du resource-core :

- Le code d'une ressource est unique par tenant.
- Le numéro de série est unique par tenant (si renseigné).
- Une seule affectation active (`isActive=true`) par ressource à tout moment.
- Une ressource `DISPOSED` ne peut recevoir de nouvelles affectations.
- Une ressource `IN_MAINTENANCE` ne peut être affectée standard.
- Une seule interface réseau principale par ressource.

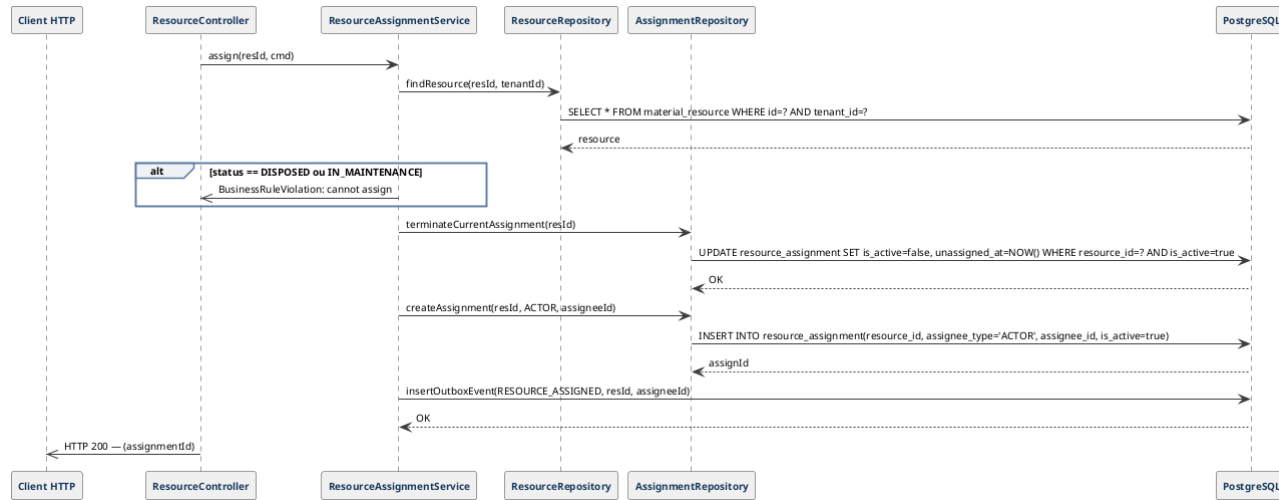
Diagramme des cas d'utilisation — resource-core



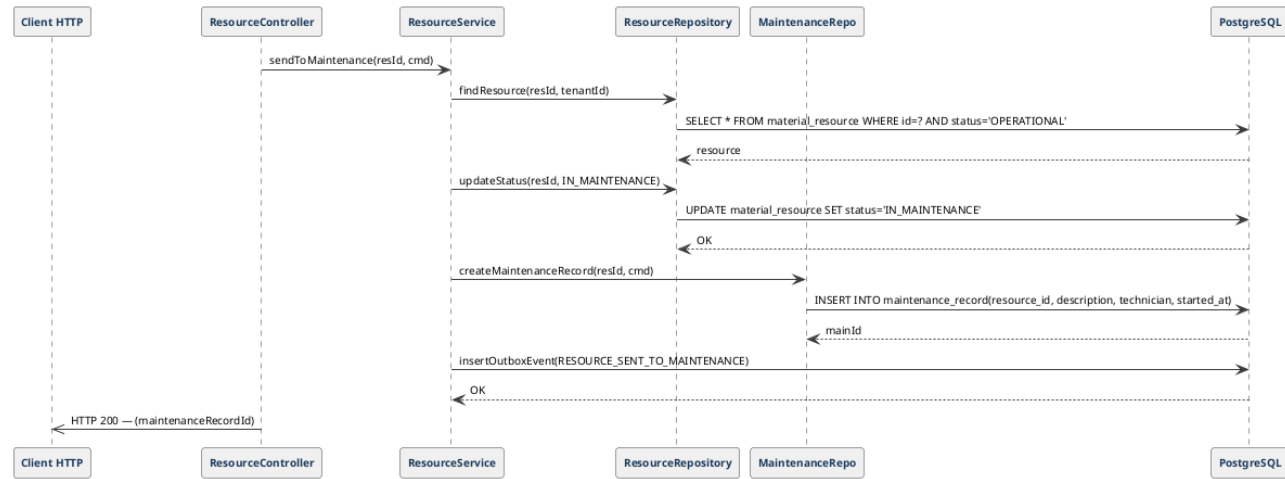
DS-RE-01 — Enregistrement d'une ressource matérielle



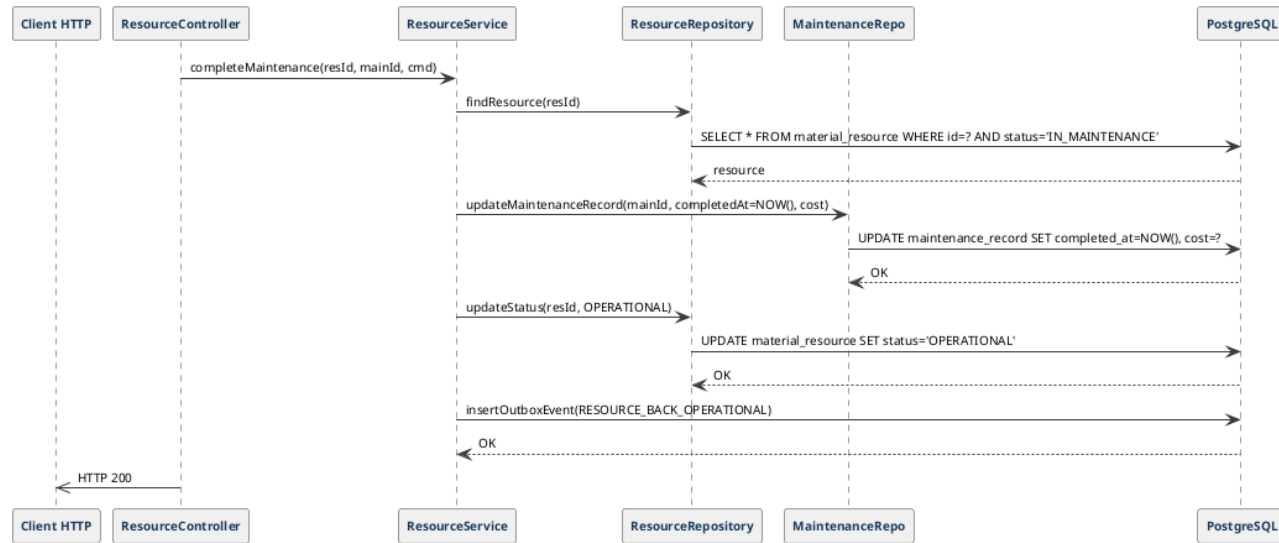
DS-RE-02 — Affectation d'une ressource à un acteur



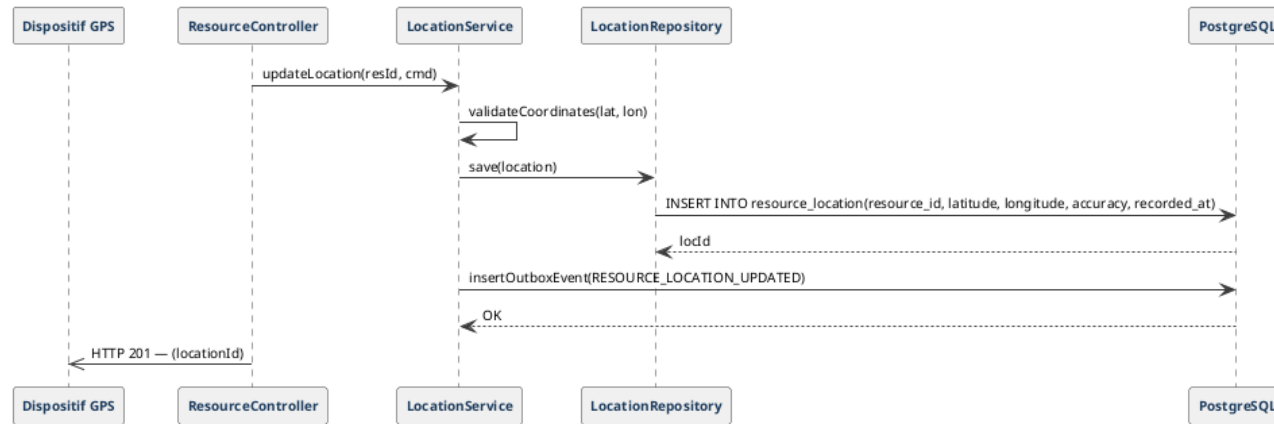
DS-RE-03 — Envoi d'une ressource en maintenance



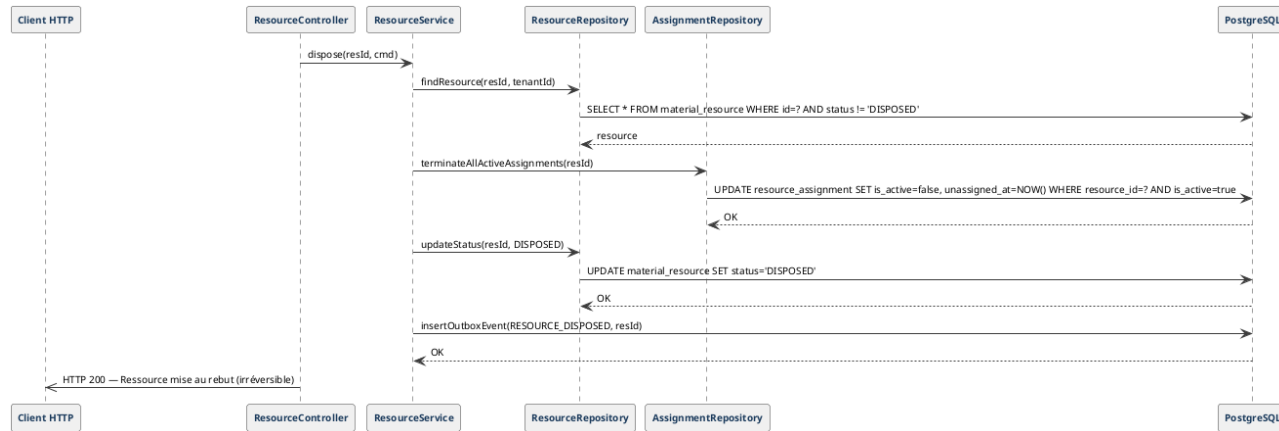
DS-RE-04 — Retour de maintenance vers l'état opérationnel



DS-RE-05 — Mise à jour de la localisation GPS d'une ressource



DS-RE-06 — Mise au rebut irréversible d'une ressource



12 sales-core — Commandes de Vente

12.1 Description du core

Le **sales-core** orchestre le cycle de vie complet des commandes de vente, depuis la création du brouillon jusqu'à la transmission à la comptabilité pour facturation. Il est le pivot central de la chaîne opérationnelle : il sollicite le **product-core** pour les snapshots tarifaires, déclenche l'allocation de stock dans l'**inventory-core** à la confirmation, et notifie l'**accounting-core** lorsqu'une commande est prête à être facturée.

La conception immutabilité-forte du domaine assure qu'une commande confirmée ou annulée ne peut plus être modifiée, garantissant l'intégrité de la piste d'audit comptable.

12.2 Rôle principal

Rôle principal : Gérer le cycle de vie des commandes de vente (création, modification, confirmation, annulation, préparation à la facturation), coordonner la vérification et l'allocation de stock, et transmettre les commandes confirmées à la comptabilité pour génération des factures.

12.3 Rôles secondaires

1. Capture de snapshots produit

À l'ajout de chaque ligne, le core capture un snapshot immuable de la variante (SKU, désignation, prix unitaire) depuis le **product-core**. Ce snapshot préserve les données tarifaires au moment de la saisie, indépendamment des modifications de catalogue ultérieures.

2. Validation de stock avant confirmation

La confirmation d'une commande est conditionnée à la disponibilité du stock pour chaque ligne. Si le stock est insuffisant, la confirmation est rejetée avec un message explicite par ligne.

3. Numérotation documentaire

Chaque commande reçoit une référence unique générée atomiquement par le **settings-core** à la confirmation (pas au brouillon).

4. Gestion des remises et taxes

Les lignes de commande supportent des remises en montant ou en pourcentage, ainsi que des taux de taxe. Le total de la commande est recalculé à chaque modification.

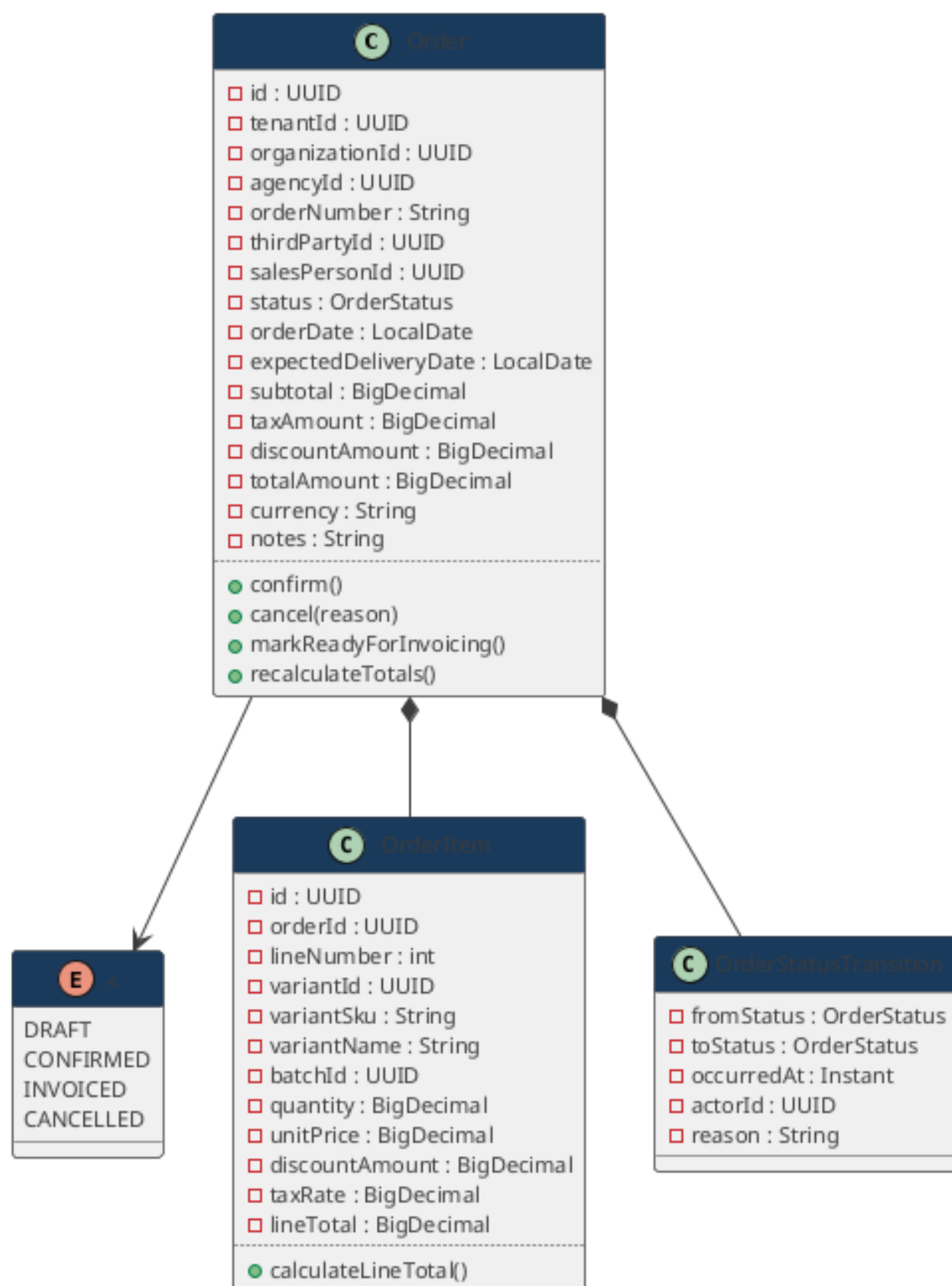
5. Libération de stock à l'annulation

L'annulation d'une commande confirmée émet un événement permettant à l'inventory-core de libérer le stock alloué.

6. Signal de facturation

La commande ne génère pas elle-même une facture. Elle émet un événement `SALES_ORDER_READY_FOR_INVOICING` consommé par l'accounting-core.

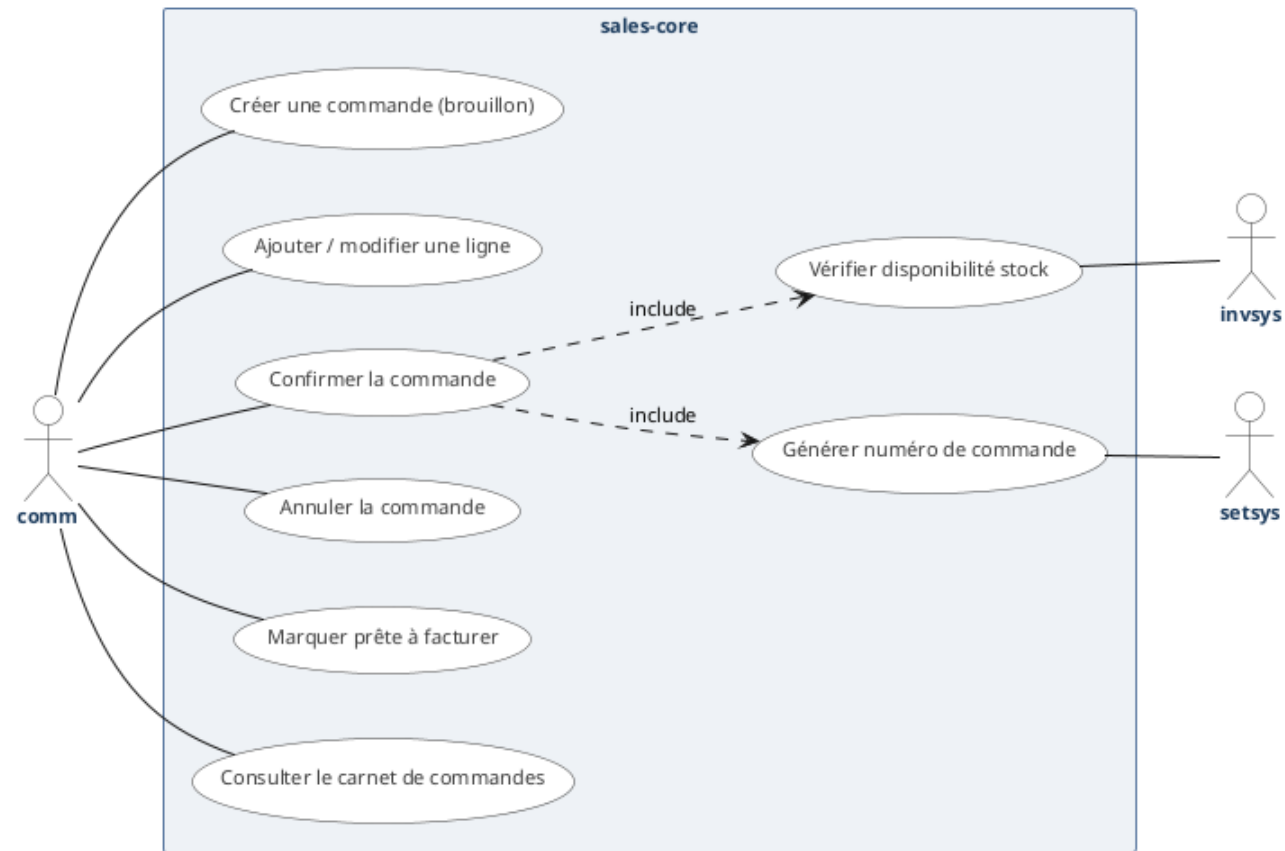
Diagramme de classes — sales-core



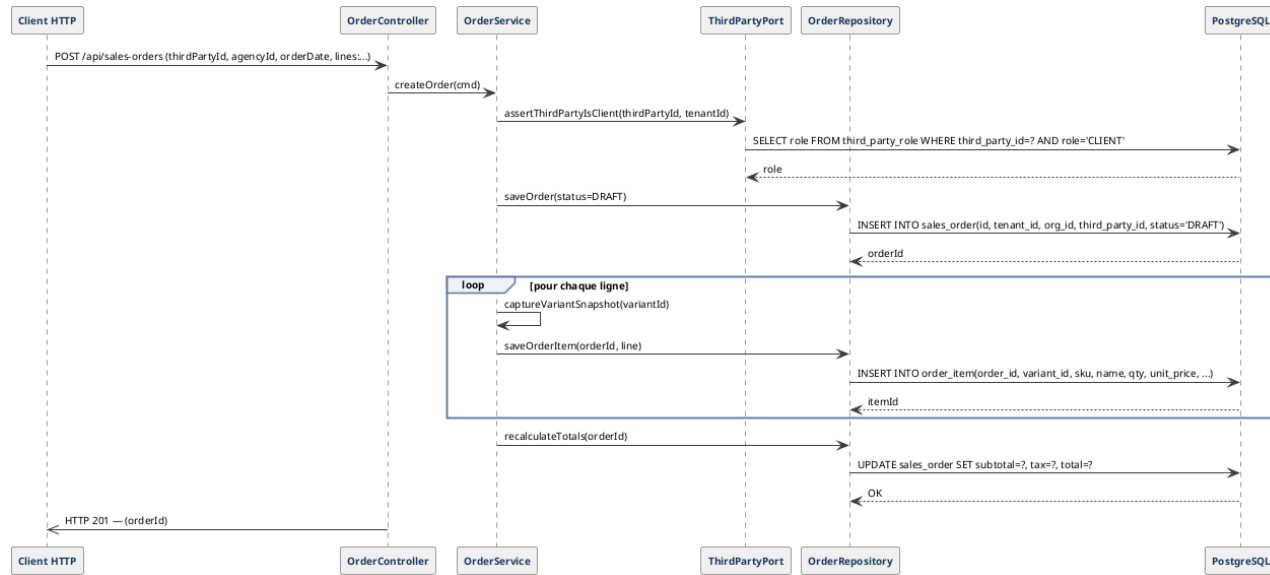
Invariants du sales-core :

- Le numéro de commande est unique par (tenantId, organizationId).
- Une commande **CONFIRMED** ou **CANCELLED** est immuable.
- Le numéro de ligne est unique au sein d'une commande.
- `totalAmount = subtotal + taxAmount - discountAmount`.
- La confirmation requiert la disponibilité stock vérifiée sur chaque ligne.

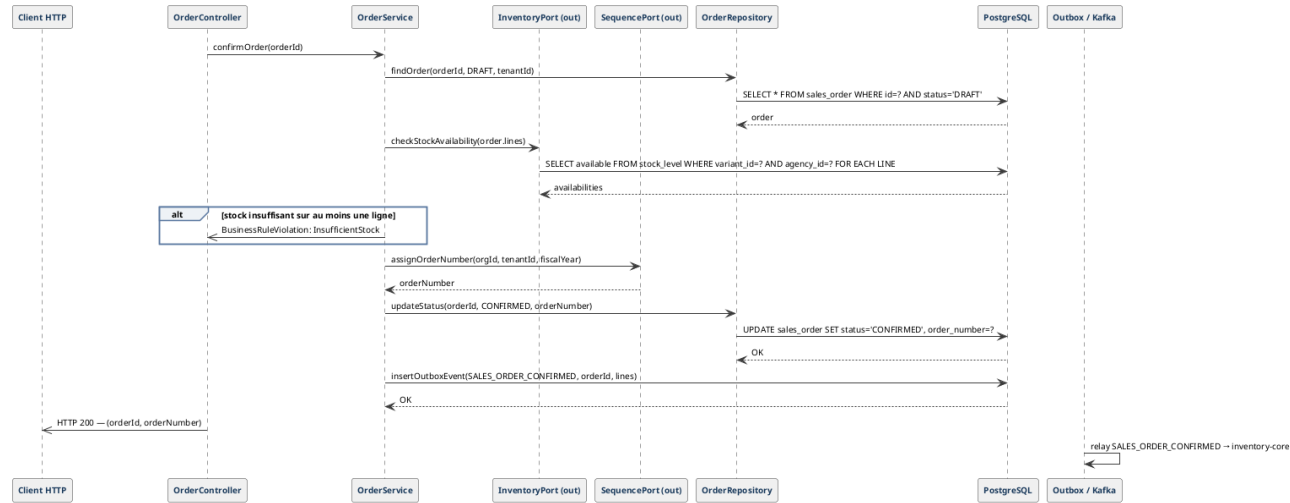
Diagramme des cas d'utilisation — sales-core



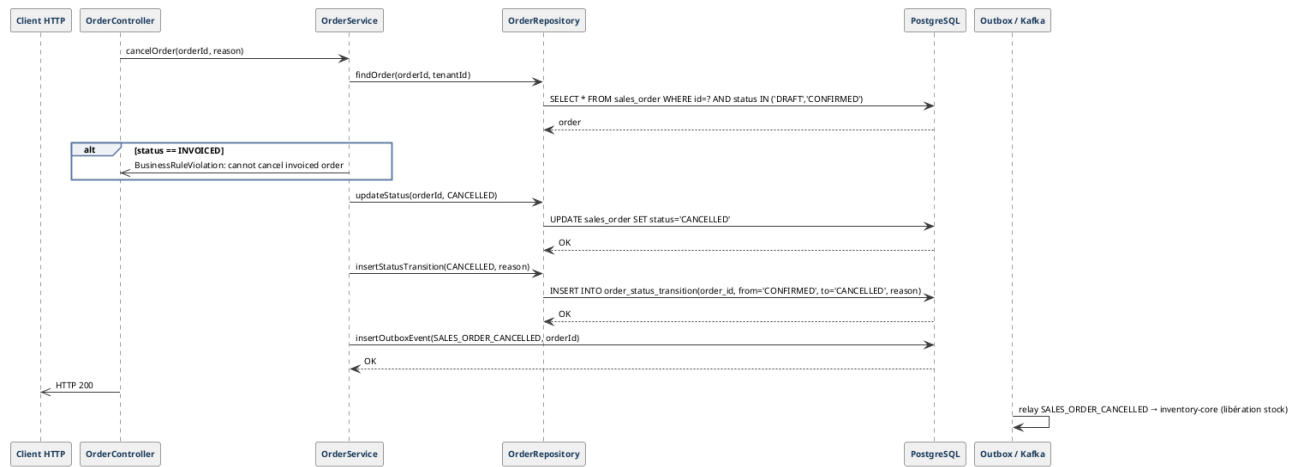
DS-SA-01 — Création d'une commande en brouillon



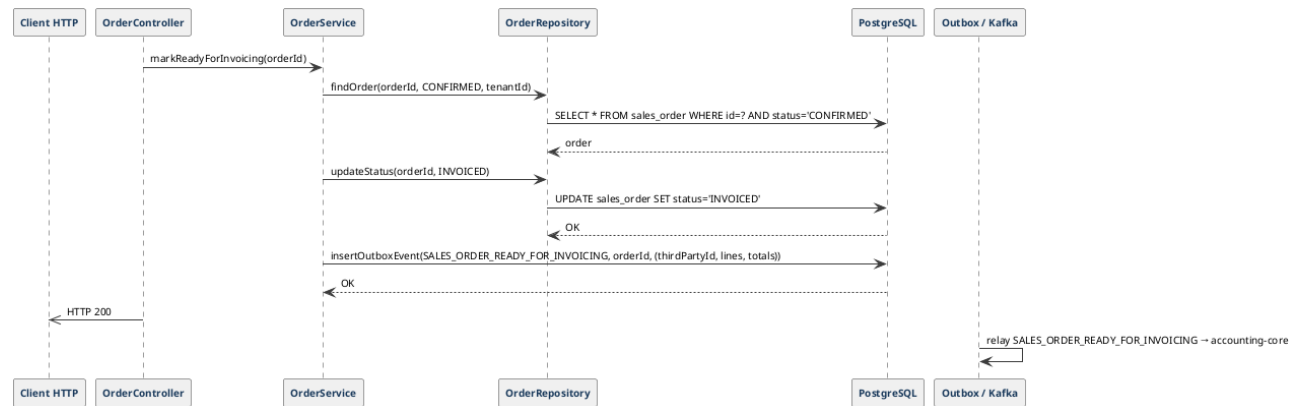
DS-SA-02 — Confirmation de commande avec vérification de stock



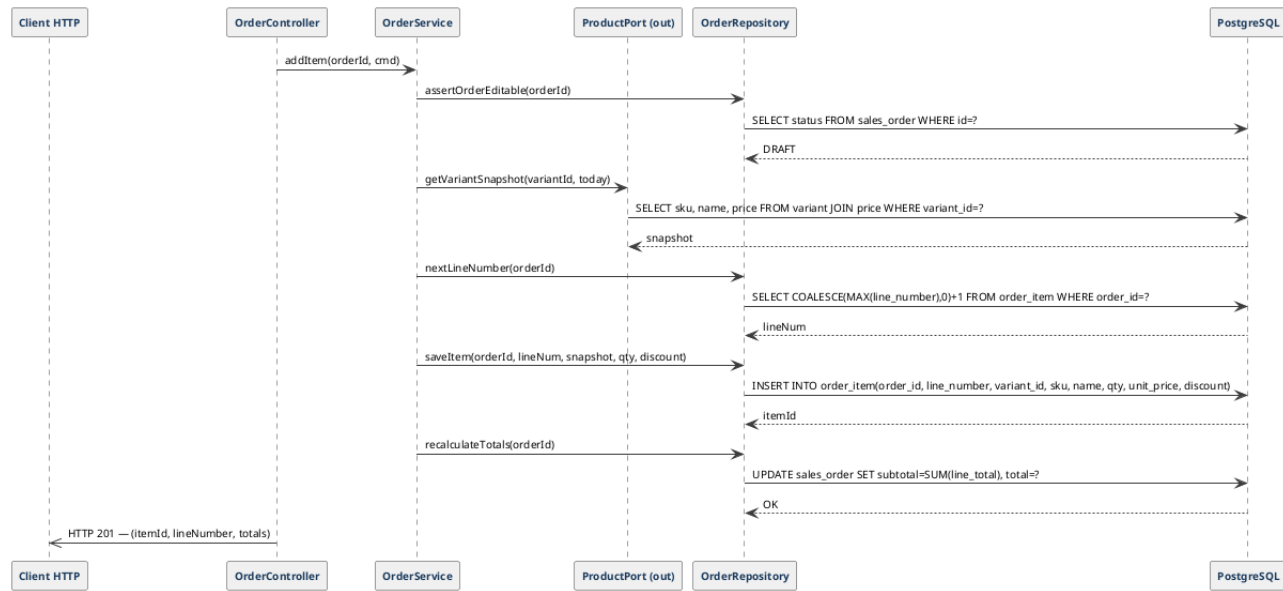
DS-SA-03 — Annulation d'une commande avec libération du stock alloué



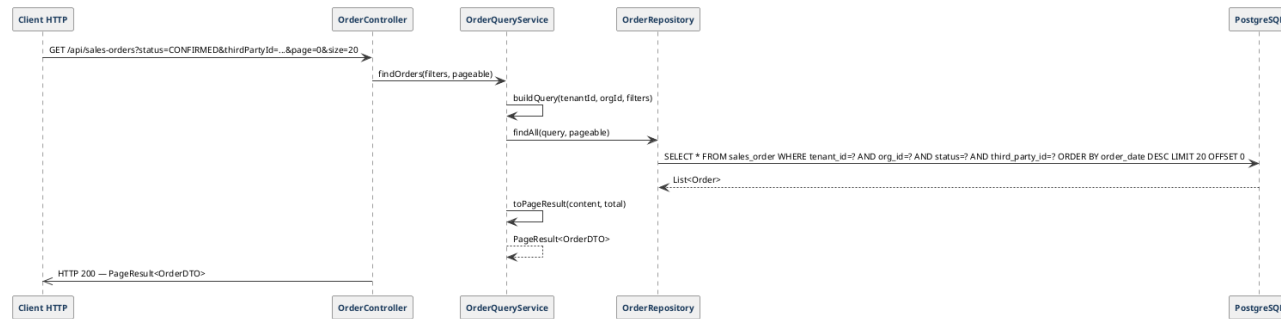
DS-SA-04 — Marquage prête à facturer et déclenchement de la facturation



DS-SA-05 — Ajout d'une ligne article à une commande en brouillon



DS-SA-06 — Consultation paginée et filtrée du carnet de commandes



13 accounting-core — Comptabilité

13.1 Description du core

L'**accounting-core** est le registre financier de la plateforme. Il maintient le plan comptable (*chart of accounts*), enregistre les écritures comptables en partie double, et gère le cycle de vie des factures clients et fournisseurs. Il est alimenté par les événements du sales-core (commandes prêtes à facturer) et déclenche les opérations de trésorerie lors du règlement des factures.

La règle fondamentale de la comptabilité en partie double est appliquée à la validation : la somme des débits doit strictement égaler la somme des crédits au sein d'une même écriture. Une écriture validée (*posted*) est définitivement immuable — la correction se fait par extourne.

13.2 Rôle principal

Rôle principal : Maintenir le plan comptable, enregistrer et valider les écritures en partie double, gérer le cycle de vie des factures (émission, validation, règlement partiel ou total), et coordonner avec la trésorerie pour le rapprochement des paiements.

13.3 Rôles secondaires

1. Plan comptable

Référentiel des comptes classés par type (ASSET, LIABILITY, EQUITY, REVENUE, EXPENSE). Chaque compte possède un numéro unique et un libellé.

2. Validation des écritures (partie double)

À la validation, l'écriture passe à l'état POSTED et ses lignes deviennent immuables. La règle *débit = crédit* est vérifiée atomiquement.

3. Extourne d'écriture

La correction d'une écriture POSTED passe par la création d'une écriture d'extourne (inversion des signes) avec référence croisée.

4. Génération automatique de factures

À la réception de l'événement `SALES_ORDER_READY_FOR_INVOICING`, une facture client est créée automatiquement avec les lignes correspondantes.

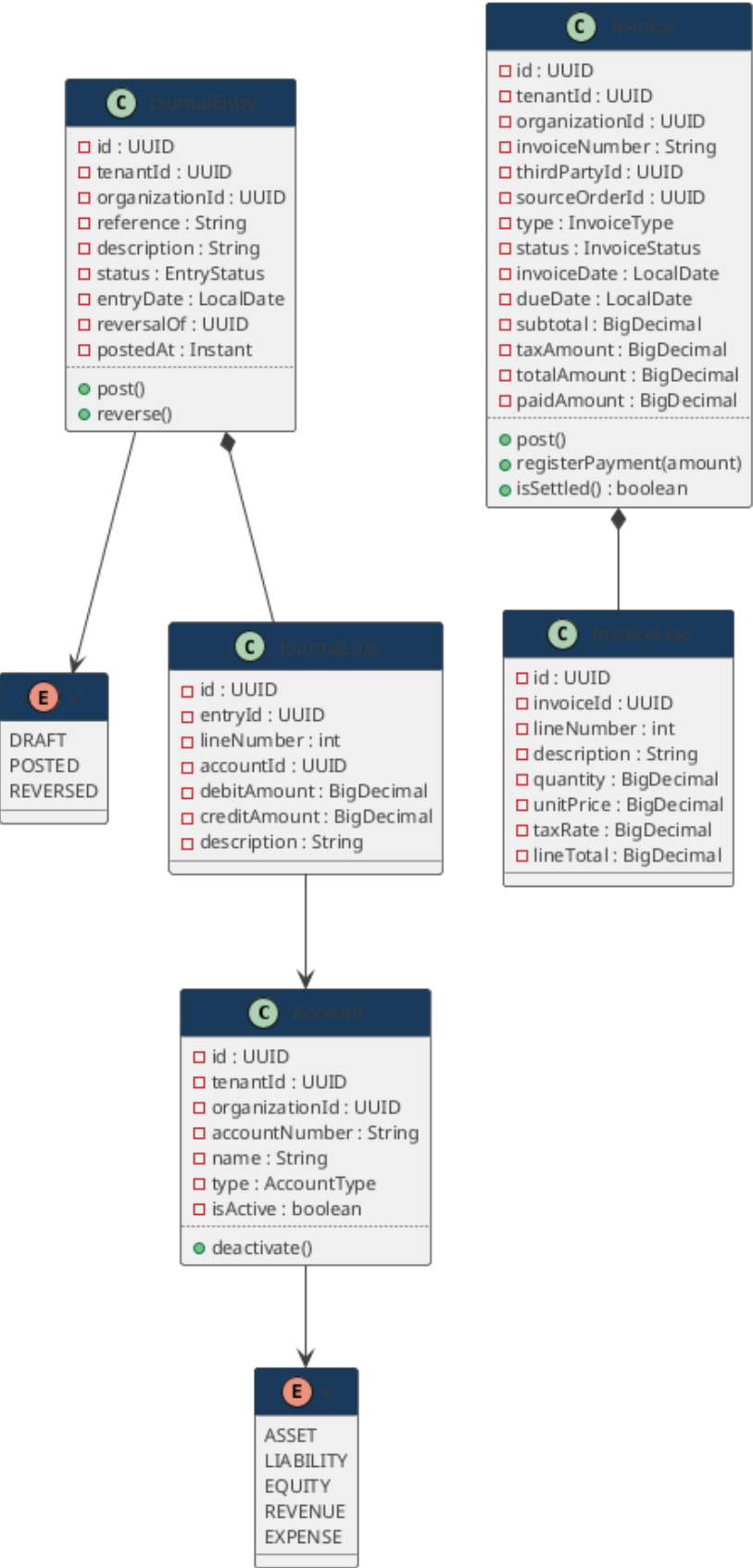
5. Règlement de factures

Enregistrement des paiements partiels ou totaux. Lors du règlement complet (`paidAmount >= totalAmount`), la facture passe à l'état `SETTLED` et un événement déclenche la trésorerie.

6. Coordination avec la trésorerie

L'événement `INVOICE_SETTLEMENT_APPLIED` est consommé par le `treasury-core` pour créer ou rapprocher la transaction bancaire correspondante.

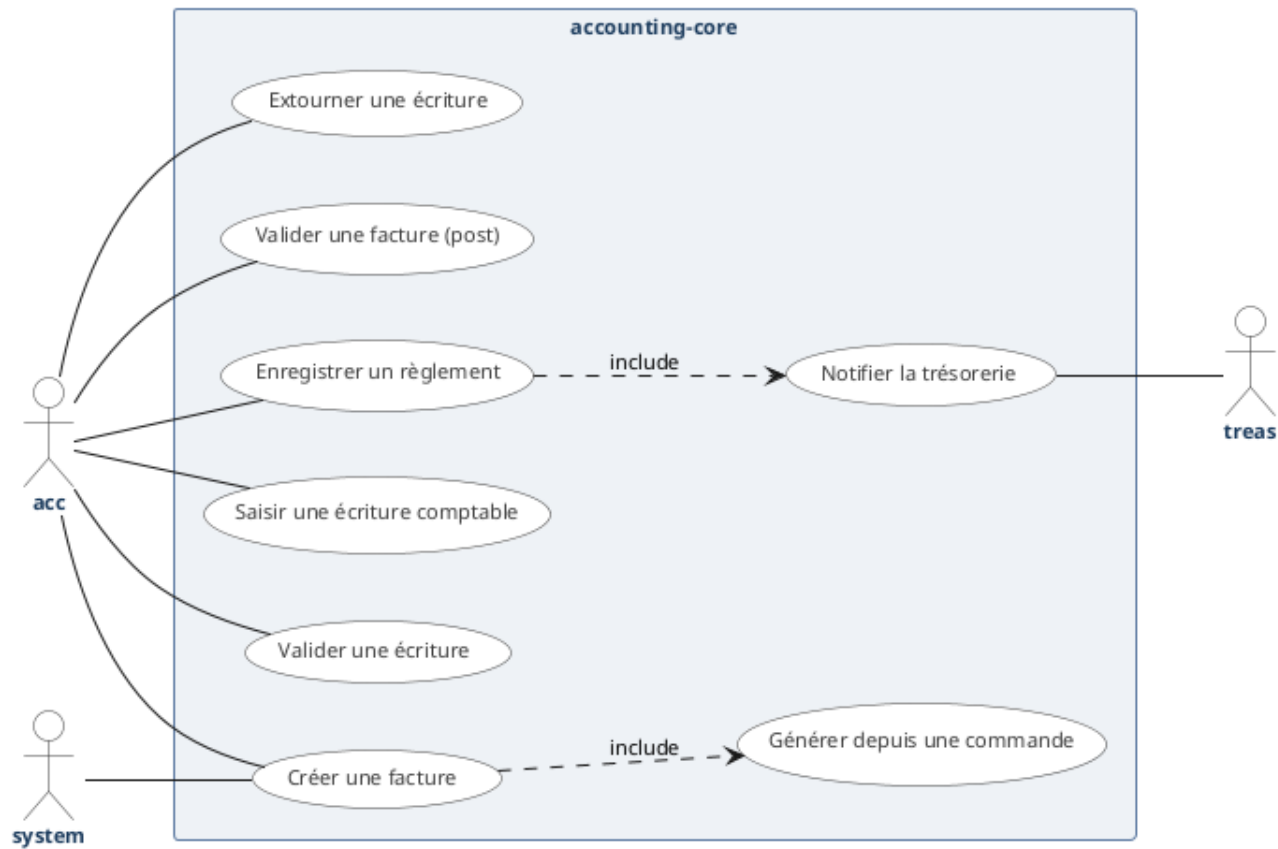
Diagramme de classes — accounting-core



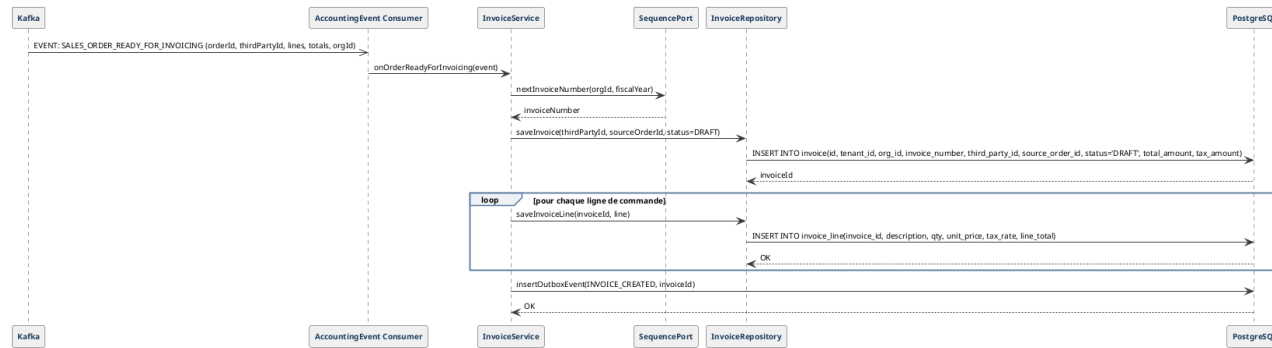
Invariants du accounting-core :

- $\sum \text{débits} = \sum \text{crédits}$ sur chaque écriture validée.
- Une écriture POSTED est immuable — correction par extourne uniquement.
- `paidAmount` $\leq$ `totalAmount`.
- Le numéro de ligne est unique au sein d'une écriture ou d'une facture.
- `totalAmount` = `subtotal` + `taxAmount`.

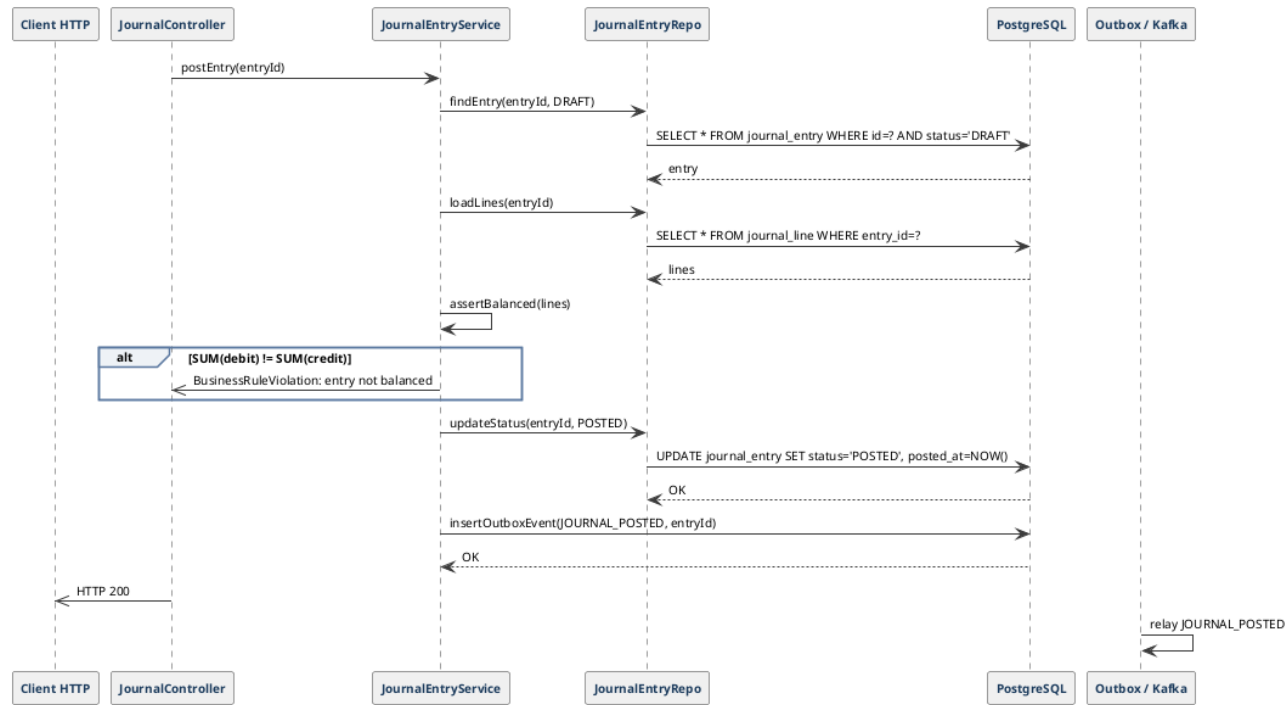
Diagramme des cas d'utilisation — accounting-core



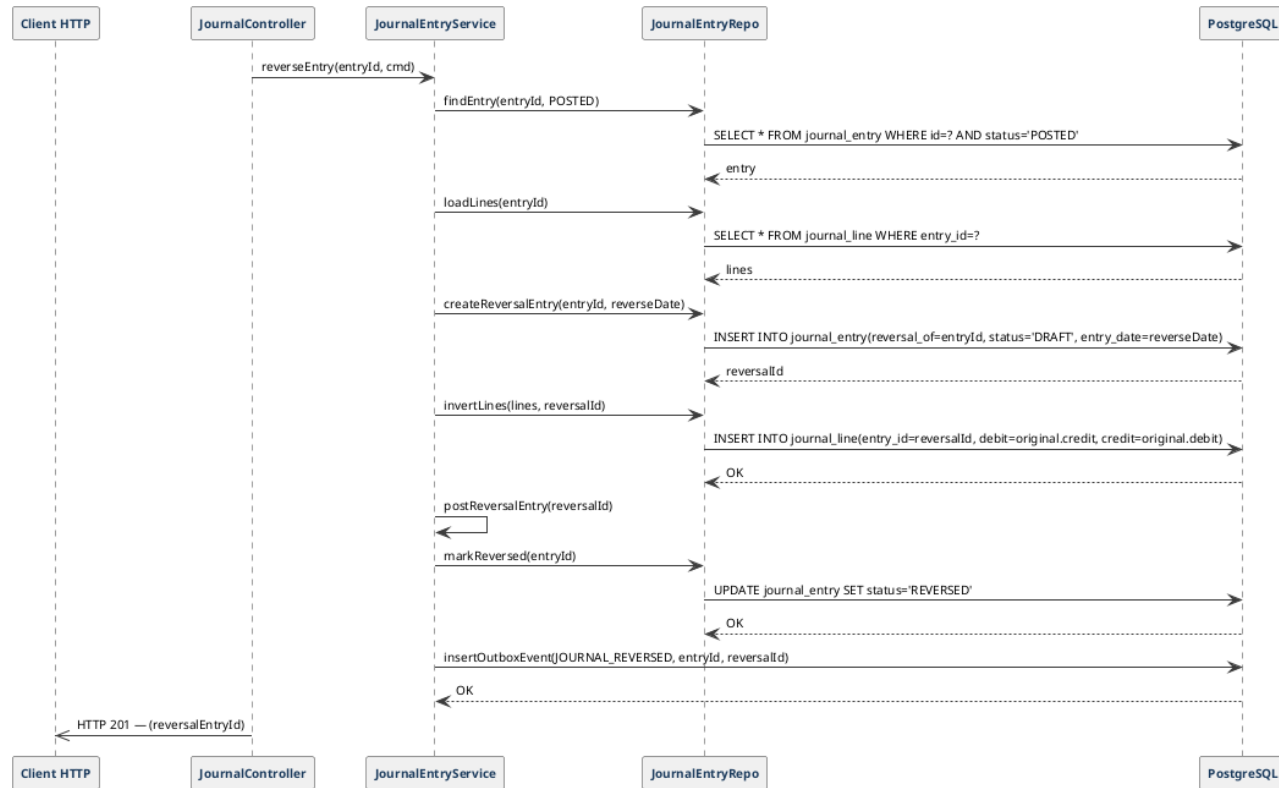
DS-AC-01 — Génération automatique d'une facture depuis un événement commande



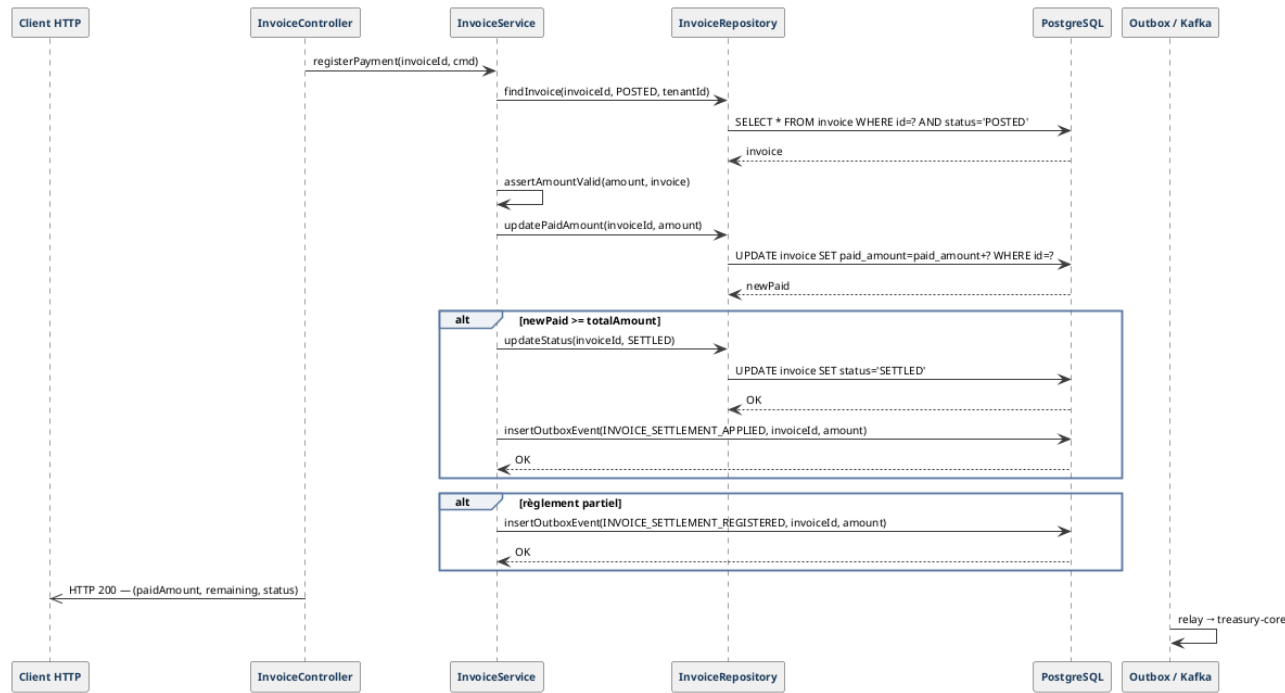
DS-AC-02 — Validation (post) d'une écriture comptable en partie double



DS-AC-03 — Extourne d'une écriture comptable validée



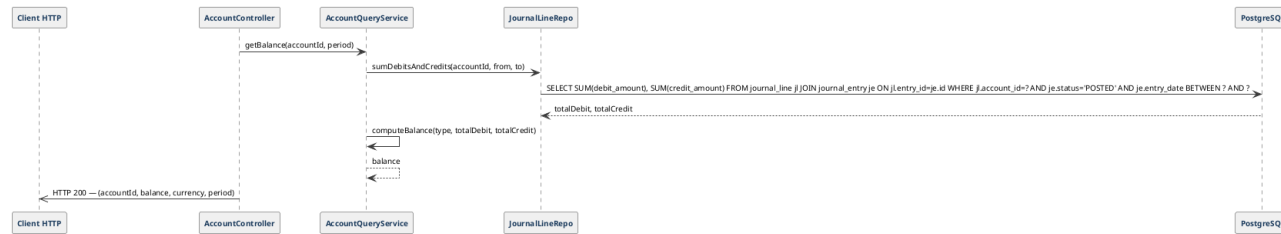
DS-AC-04 — Règlement partiel ou total d'une facture



DS-AC-05 — Validation comptable d'une facture avec génération d'écritures



DS-AC-06 — Consultation du solde d'un compte sur une période



14 treasury-core — Trésorerie

14.1 Description du core

Le **treasury-core** gère les flux de trésorerie et les opérations bancaires de l'organisation. Il maintient les comptes bancaires, enregistre les transactions, importe les relevés bancaires, gère les chèques émis, et assure le rapprochement entre les mouvements comptables et les opérations réelles sur les comptes.

Ce core est en position finale dans la chaîne financière : il reçoit les événements de règlement de l'accounting-core et les traduit en transactions bancaires concrètes. Le rapprochement bancaire constitue l'opération d'audit ultime qui valide la cohérence entre les livres comptables et les relevés bancaires.

14.2 Rôle principal

Rôle principal : Gérer les comptes bancaires de l'organisation, enregistrer toutes les transactions financières, importer et rapprocher les relevés bancaires, et clôturer les rapprochements après validation de la cohérence avec les opérations comptables.

14.3 Rôles secondaires

1. Gestion des établissements et comptes bancaires

Référentiel des banques partenaires (SWIFT, pays) et des comptes de l'organisation (IBAN, BIC, titulaire).

2. Enregistrement des transactions

Chaque entrée ou sortie de fonds (virement, prélèvement, paiement par chèque) est enregistrée comme une **BankTransaction**.

3. Import de relevés bancaires

Un relevé peut être importé manuellement ou par flux automatisé. Un relevé remplace le précédent pour la même période et le même compte.

4. Gestion des chèques

Suivi du cycle de vie des chèques émis : **ISSUED** → **CLEARED** (encaissement confirmé) ou **BOUNCED** (rejet).

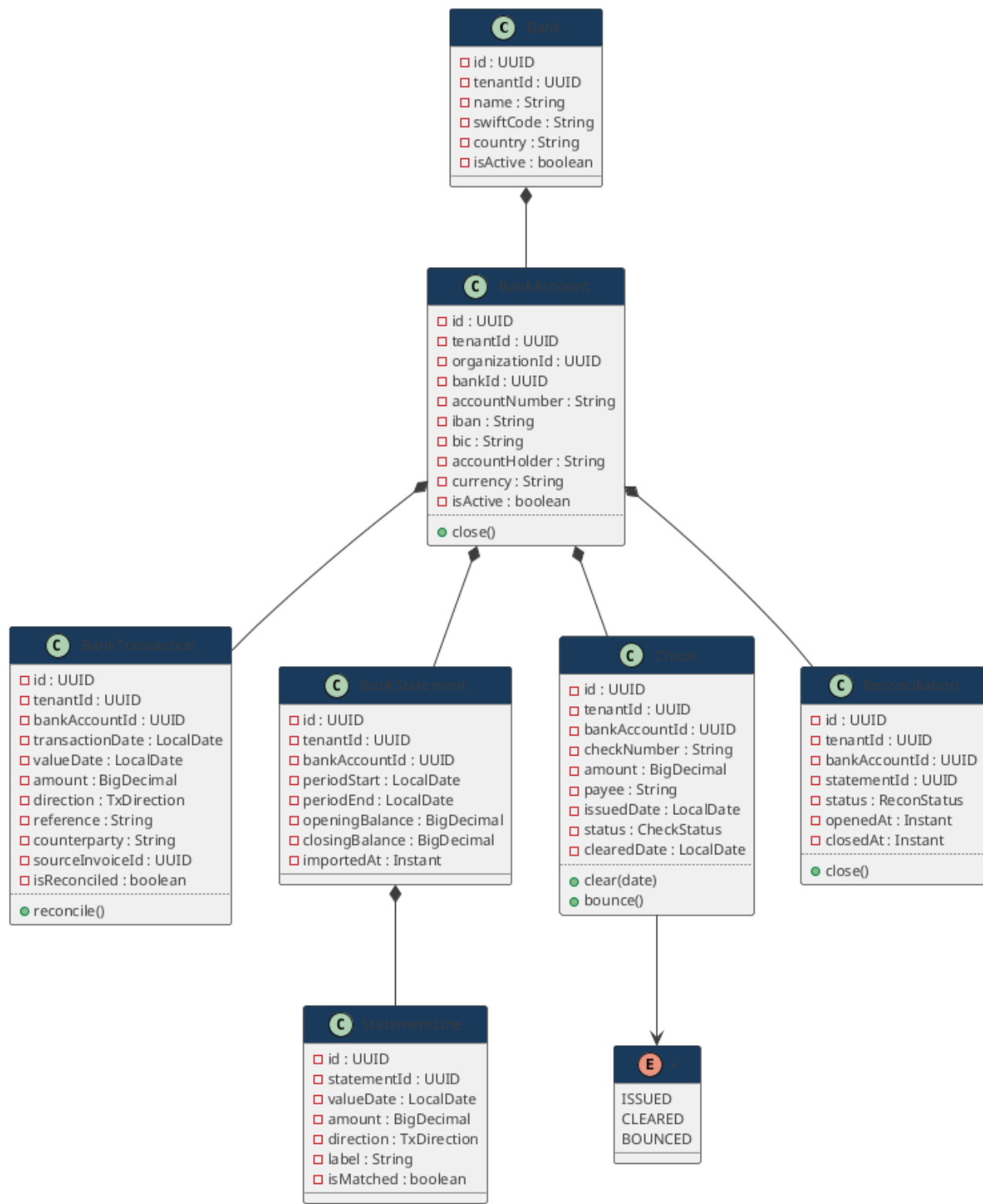
5. Rapprochement bancaire

Association des transactions enregistrées aux lignes du relevé bancaire. Un rapprochement clôturé est immuable et constitue une pièce d'audit.

6. Coordination avec la comptabilité

Réception de l'événement `INVOICE_SETTLEMENT_APPLIED` pour créer automatiquement la transaction bancaire correspondante et l'associer au rapprochement en cours.

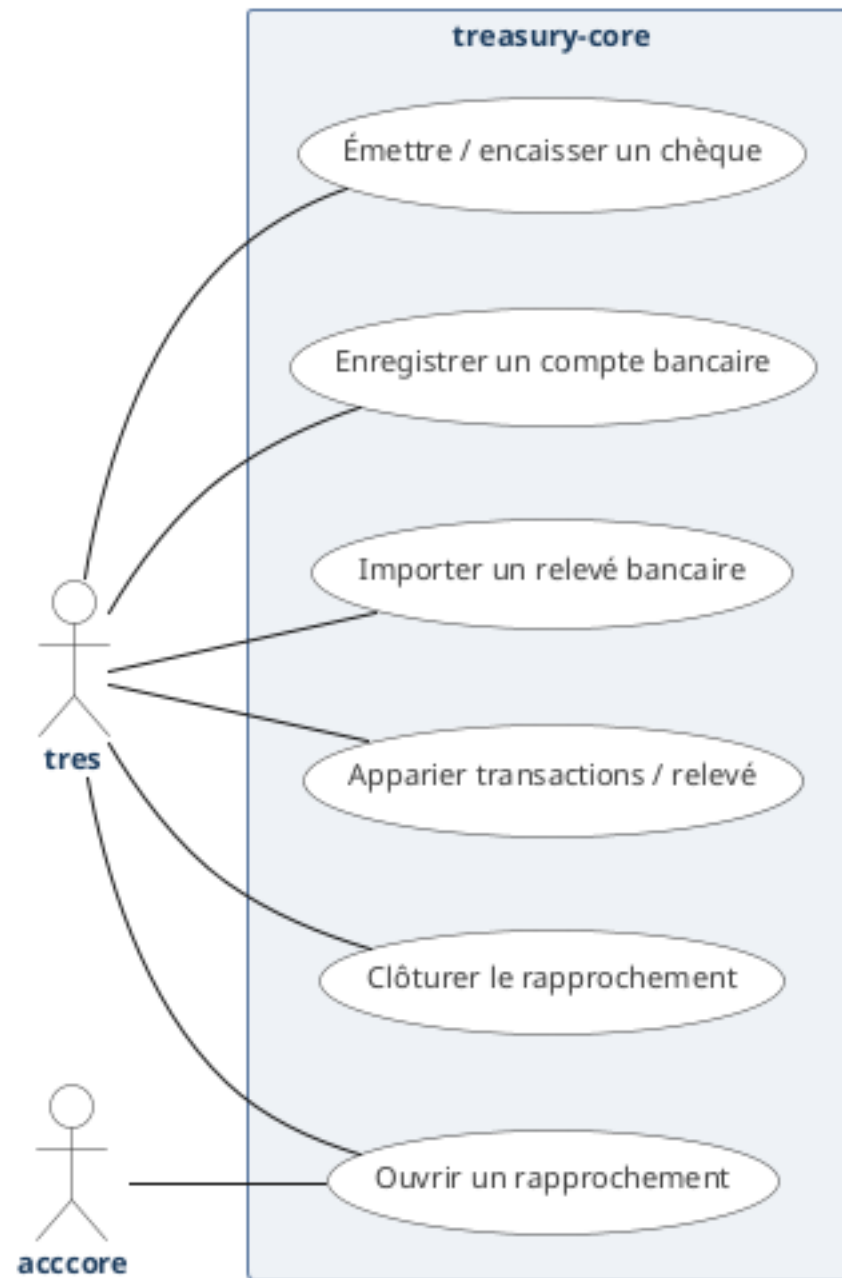
Diagramme de classes — treasury-core



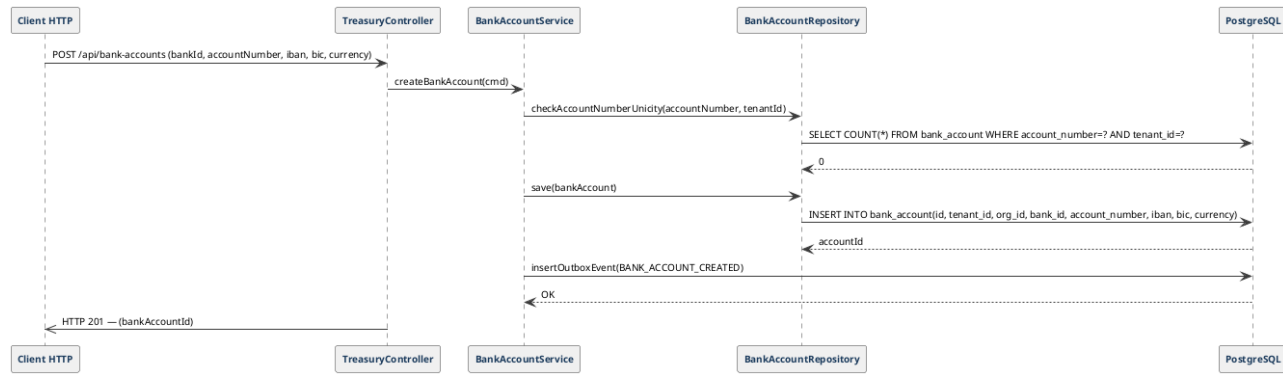
Invariants du treasury-core :

- Le numéro de compte bancaire est unique par tenant.
- Un relevé est unique par (compteId, périodeStart, périodeEnd).
- Un chèque ne peut être encaissé (CLEARED) qu'une seule fois.
- Un rapprochement CLOSED est immuable.
- $\text{closingBalance} = \text{openingBalance} + \text{SUM}(\text{transactions sur la période})$.

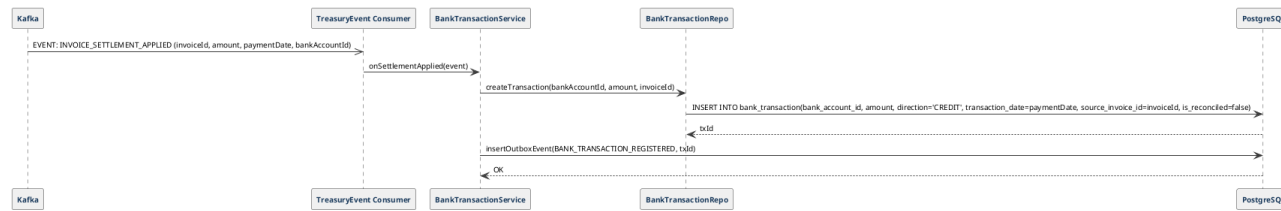
Diagramme des cas d'utilisation — treasury-core



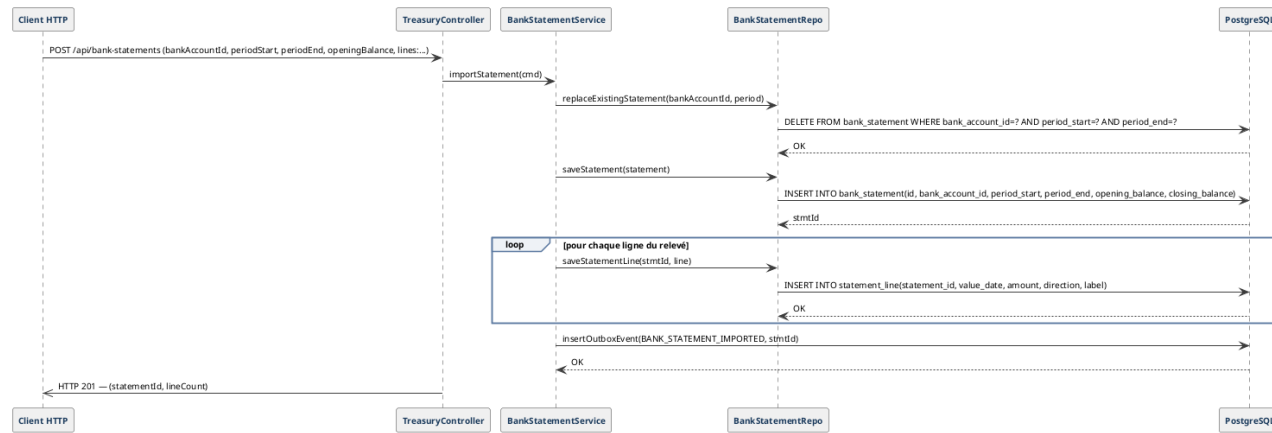
DS-TR-01 — Enregistrement d'un compte bancaire



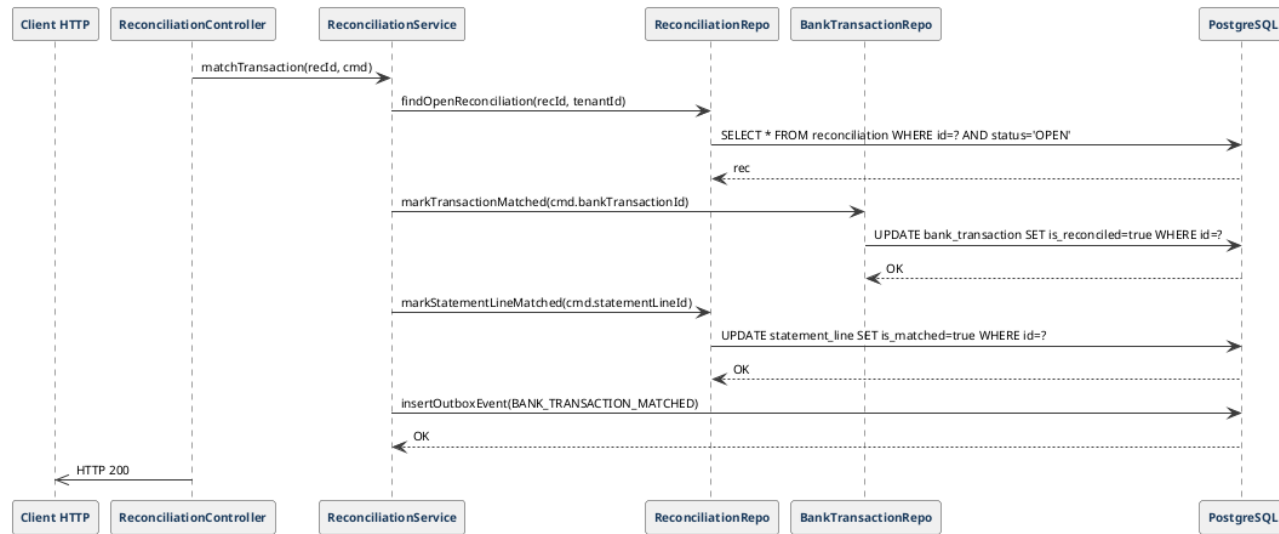
DS-TR-02 — Création automatique d'une transaction bancaire depuis un règlement



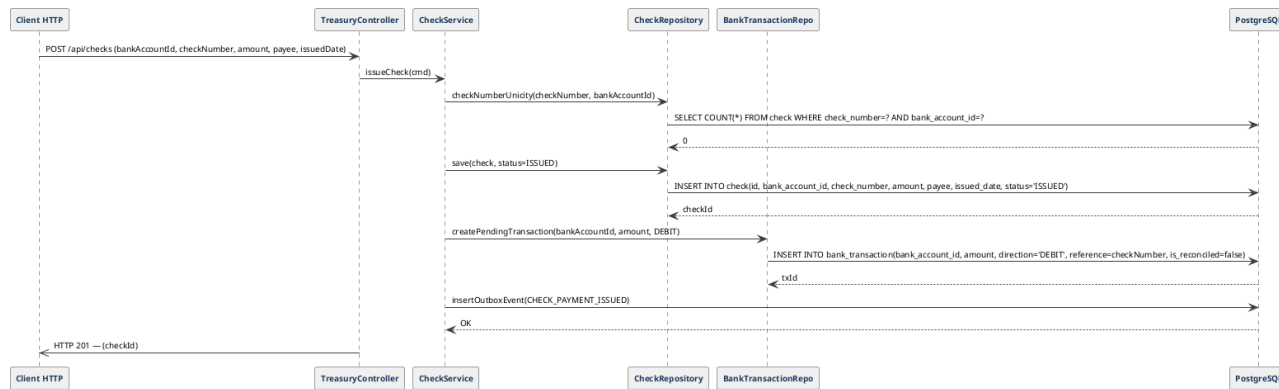
DS-TR-03 — Import d'un relevé bancaire



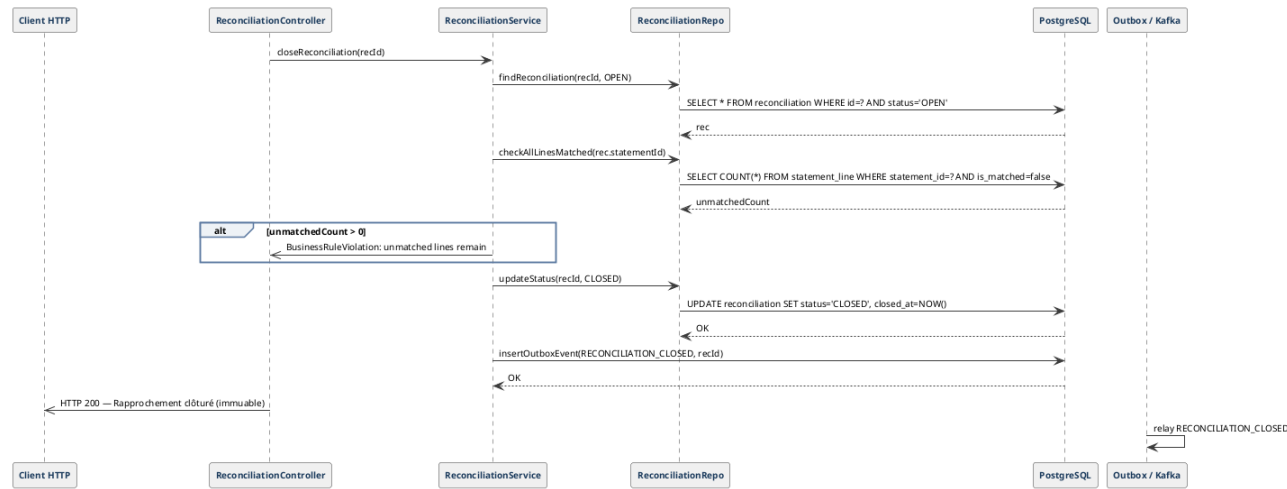
DS-TR-04 — Appariement d'une transaction bancaire à une ligne de relevé



DS-TR-05 — Émission d'un chèque avec création de la transaction en attente



DS-TR-06 — Clôture définitive d'un rapprochement bancaire



15 administration-core — Gouvernance et Administration

15.1 Description du core

L'**administration-core** est le module de gouvernance transversale de la plateforme. Il regroupe les fonctions d'administration système qui ne relèvent pas d'un domaine métier précis : approbation des entités soumises à validation (*BusinessActors*, *Organisations*, *Agences*), gestion des options de plateforme, consultation de la piste d'audit, et orchestration des rôles et permissions à l'échelle tenant.

Ce core opère avec les permissions les plus élevées et expose des routes protégées par des rôles d'administration. Il coordonne avec les autres cores via des ports sortants et des événements, sans implémenter de logique métier opérationnelle.

15.2 Rôle principal

Rôle principal : Orchestrer la gouvernance du tenant (approbation des entités, configuration des politiques plateforme, supervision de l'audit) et fournir les interfaces d'administration centralisées pour la gestion des rôles, des utilisateurs et des paramètres globaux.

15.3 Rôles secondaires

1. Approbation des entités soumises à validation

Flux de gouvernance pour les *BusinessActors* (soumission, approbation/rejet), les *Organisations* et les *Agences* lorsque les options de plateforme exigent une validation manuelle.

2. Options de plateforme

Paramètres globaux du tenant contrôlant les comportements de self-service : `allowOrganizationSelfServiceCreation`, `requireBusinessActorApproval`, `allowRoleCloning`, etc.

3. Consultation de la piste d'audit

Interface de lecture de l'*AuditTrail* du kernel-core, exposée aux administrateurs

avec filtres par acteur, type d'action, date et cible.

4. Gestion centralisée des rôles et utilisateurs

Surface d'API unifiée pour les opérations d'administration des rôles (création, clonage, affectation) réexposées depuis le roles-core avec vérifications de politique supplémentaires.

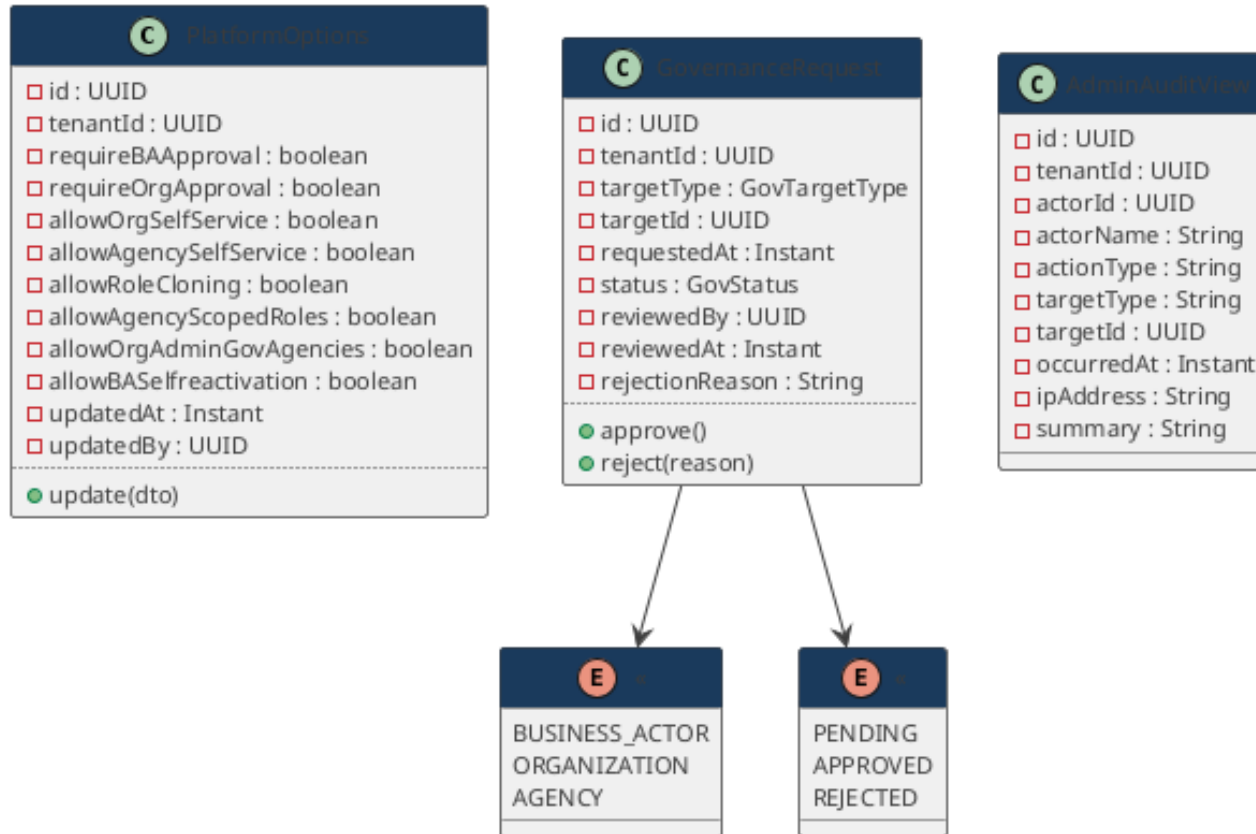
5. Supervision des métriques opérationnelles

Accès aux indicateurs Actuator (santé, métriques, état de l'outbox) via le port de management sécurisé.

6. Initialisation du bootstrap tenant

En mode bootstrap temporaire, permet la création des premiers comptes et rôles système avant que le tenant ne soit pleinement opérationnel.

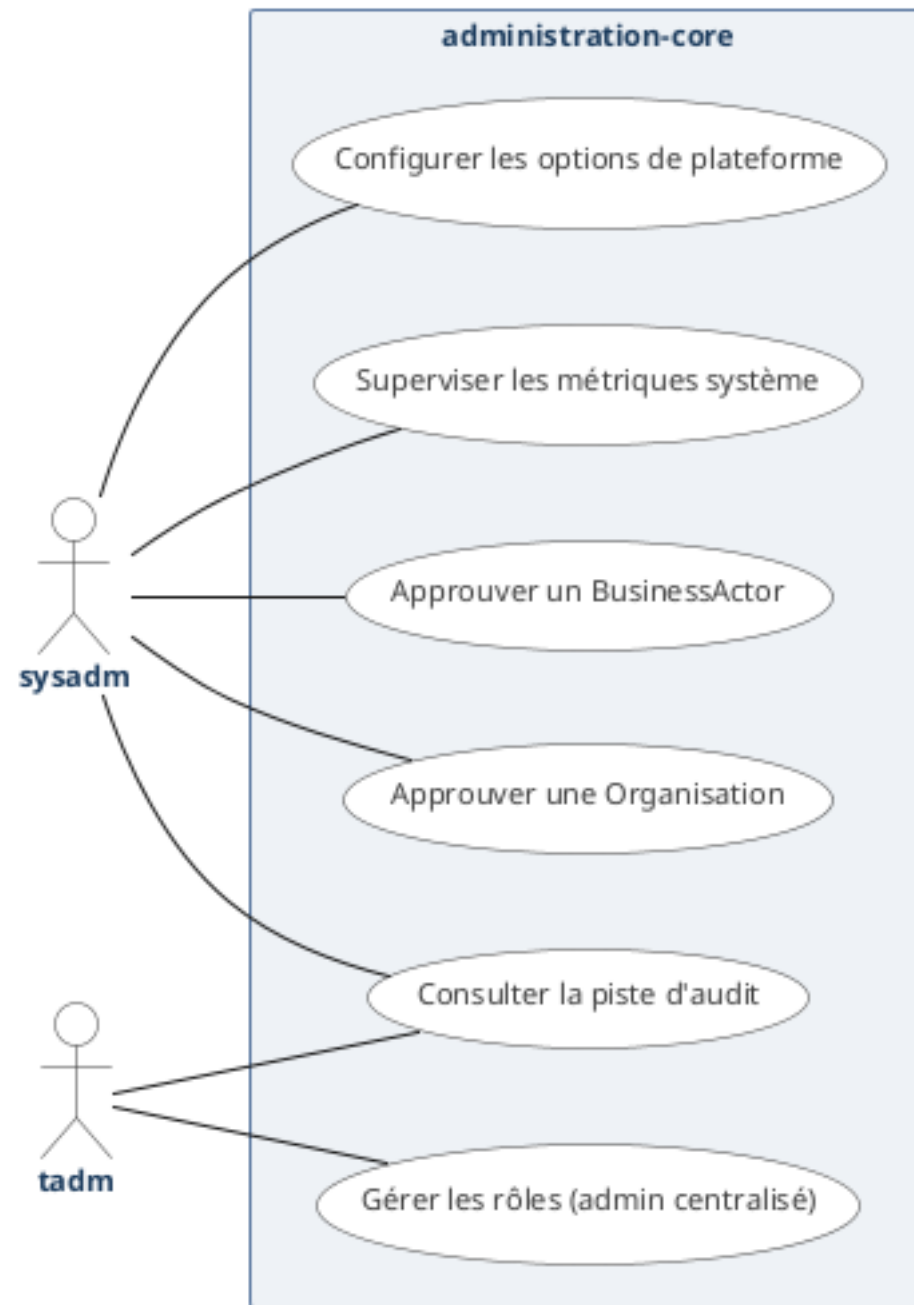
Diagramme de classes — administration-core



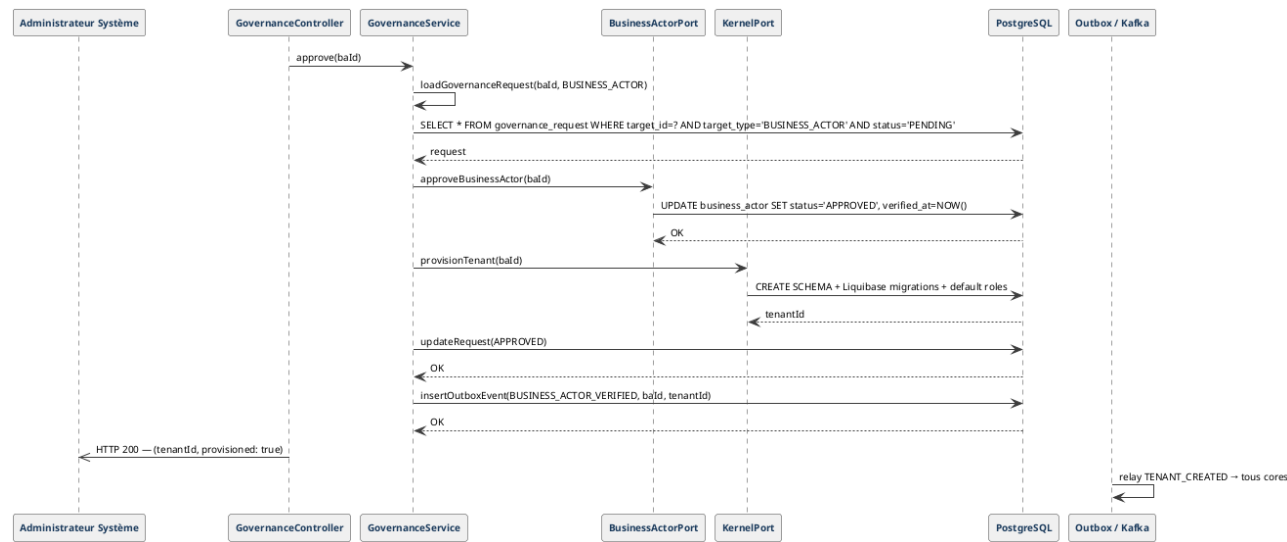
Invariants du administration-core :

- Les `PlatformOptions` sont uniques par tenant (une seule ligne de configuration).
- Un `GovernanceRequest` approuvé ou rejeté est immuable.
- La modification des options de plateforme est tracée dans l'`AuditTrail`.
- L'`AdminAuditView` est en lecture seule — jamais modifiée par ce core.

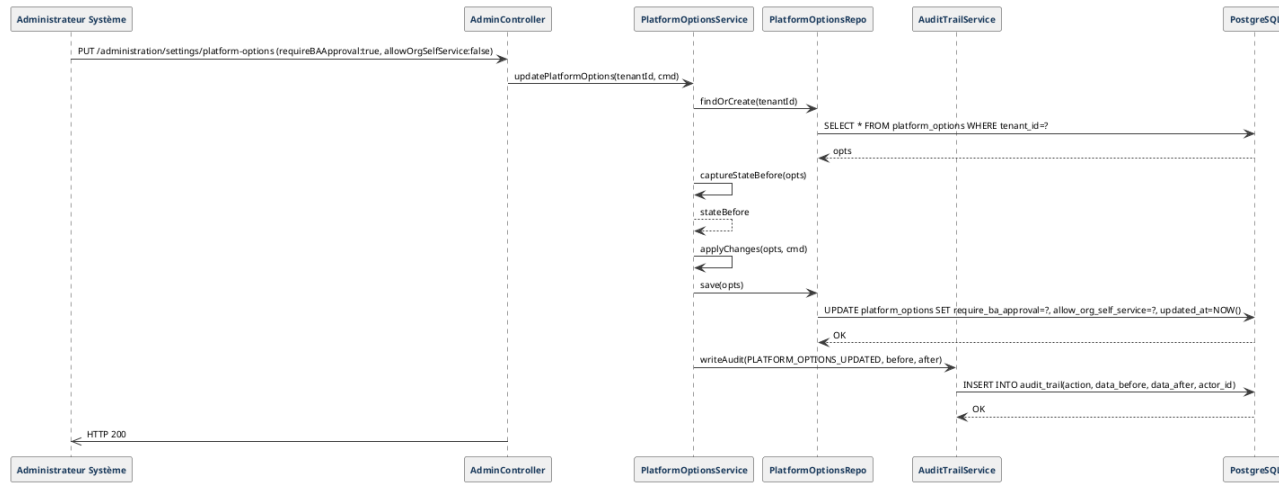
Diagramme des cas d'utilisation — administration-core



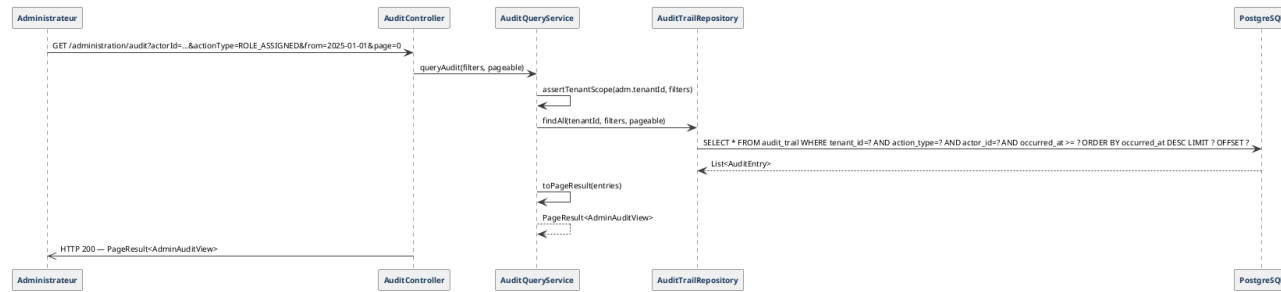
DS-AD-01 — Approbation d'un BusinessActor et provisionnement du tenant



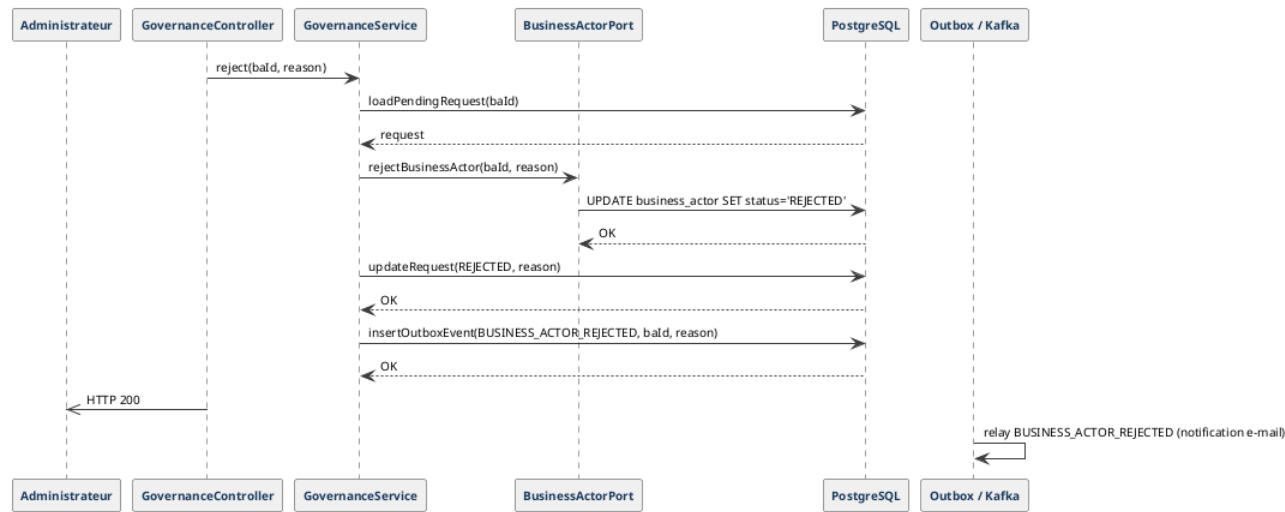
DS-AD-02 — Mise à jour des options de plateforme avec audit



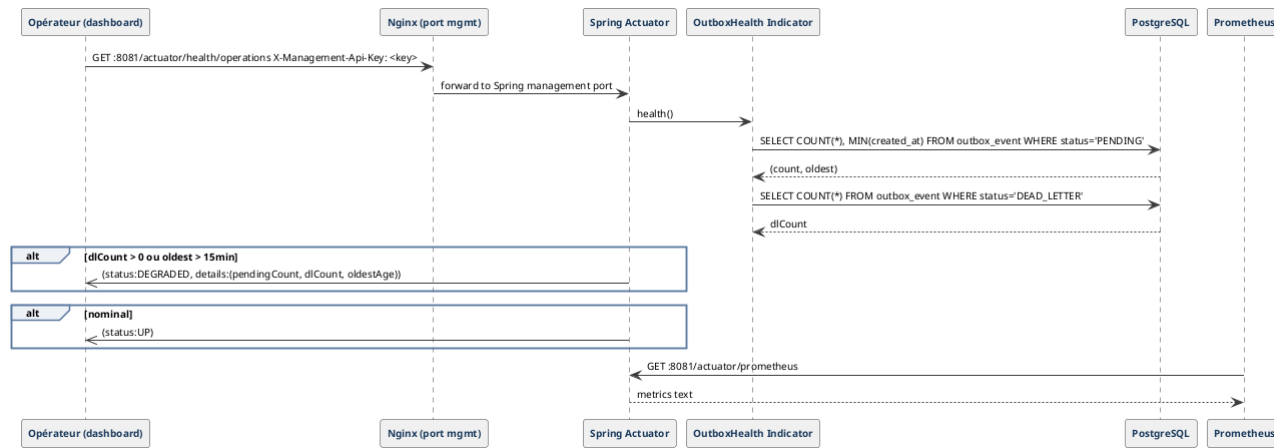
DS-AD-03 — Consultation filtrée de la piste d'audit



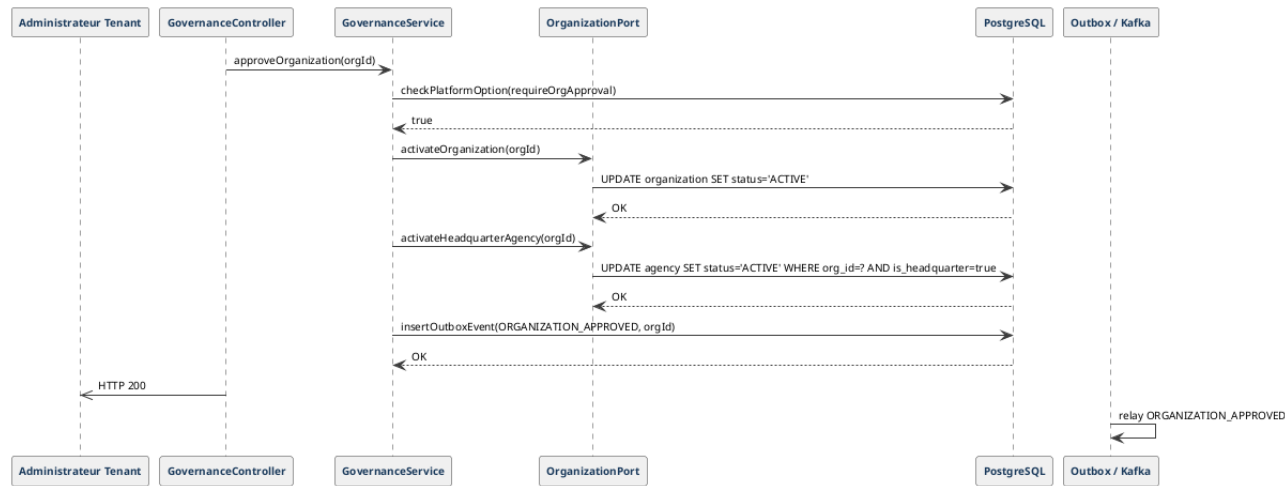
DS-AD-04 — Rejet d'une demande de gouvernance avec notification



DS-AD-05 — Supervision de l'état du système via le port de management



DS-AD-06 — Approbation d'une organisation soumise à validation



16 file-core — Gestion des Fichiers

16.1 Description du core

Le **file-core** est le service de stockage et de gestion des fichiers binaires de la plateforme. Il offre un point d'accès unifié pour l'upload, le téléchargement et l'association de fichiers aux entités métier (produits, tiers, ressources, documents). Les métadonnées sont stockées en PostgreSQL tandis que les binaires sont stockés localement ou dans un service de stockage objet configurable.

La conception sécurisée du core prévient les vulnérabilités classiques liées à la gestion de fichiers : traversée de répertoire (*path traversal*), injection de type MIME, et dépassement de quota.

16.2 Rôle principal

Rôle principal : Fournir un service centralisé et sécurisé pour l'upload, le stockage, la récupération et l'association de fichiers binaires aux entités de la plateforme, avec validation du type MIME, contrôle de taille et isolation par tenant.

16.3 Rôles secondaires

1. Validation du type MIME

Validation par liste blanche des types MIME autorisés. La validation s'effectue sur le contenu réel du fichier (magic bytes), pas uniquement sur l'extension déclarée.

2. Contrôle de taille

Limite de taille configurable par type de fichier et par tenant. Les fichiers dépassant la limite sont rejetés avant stockage.

3. Isolation par tenant

Les fichiers sont stockés dans un répertoire racine dédié par tenant, avec validation stricte du chemin pour prévenir tout accès cross-tenant.

4. Association à des entités

Un fichier peut être associé à une entité cible (produit, tiers, ressource) via un

enregistrement `FileAssociation`. Une entité peut avoir plusieurs fichiers.

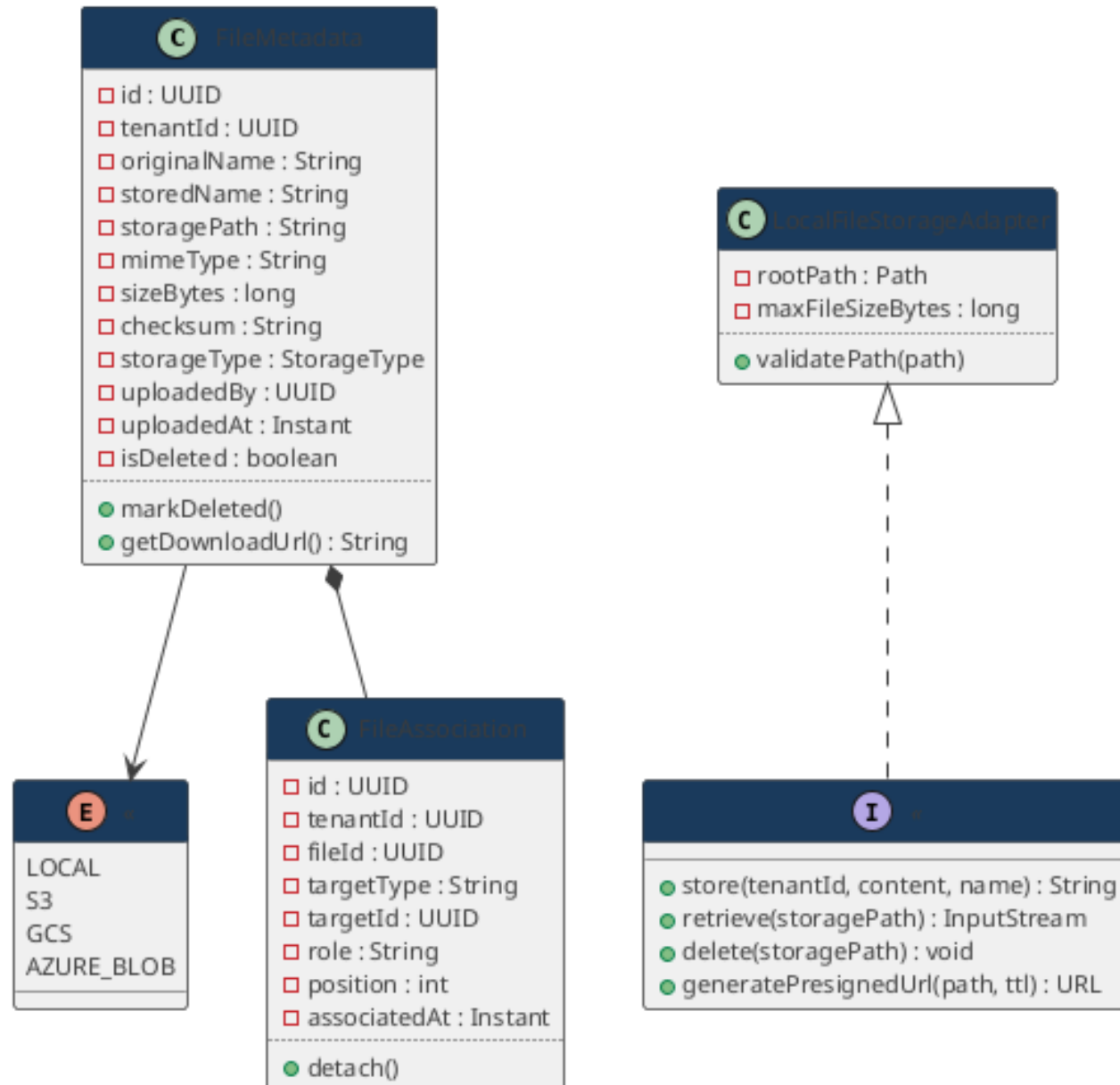
5. Suppression sécurisée

La suppression d'un fichier vérifie d'abord qu'il n'est plus référencé par aucune entité active avant de supprimer le binaire et les métadonnées.

6. Génération d'URLs temporaires

Pour les fichiers stockés dans un service objet, génération d'URLs signées à durée de vie limitée (*presigned URLs*), évitant l'exposition directe des credentials de stockage.

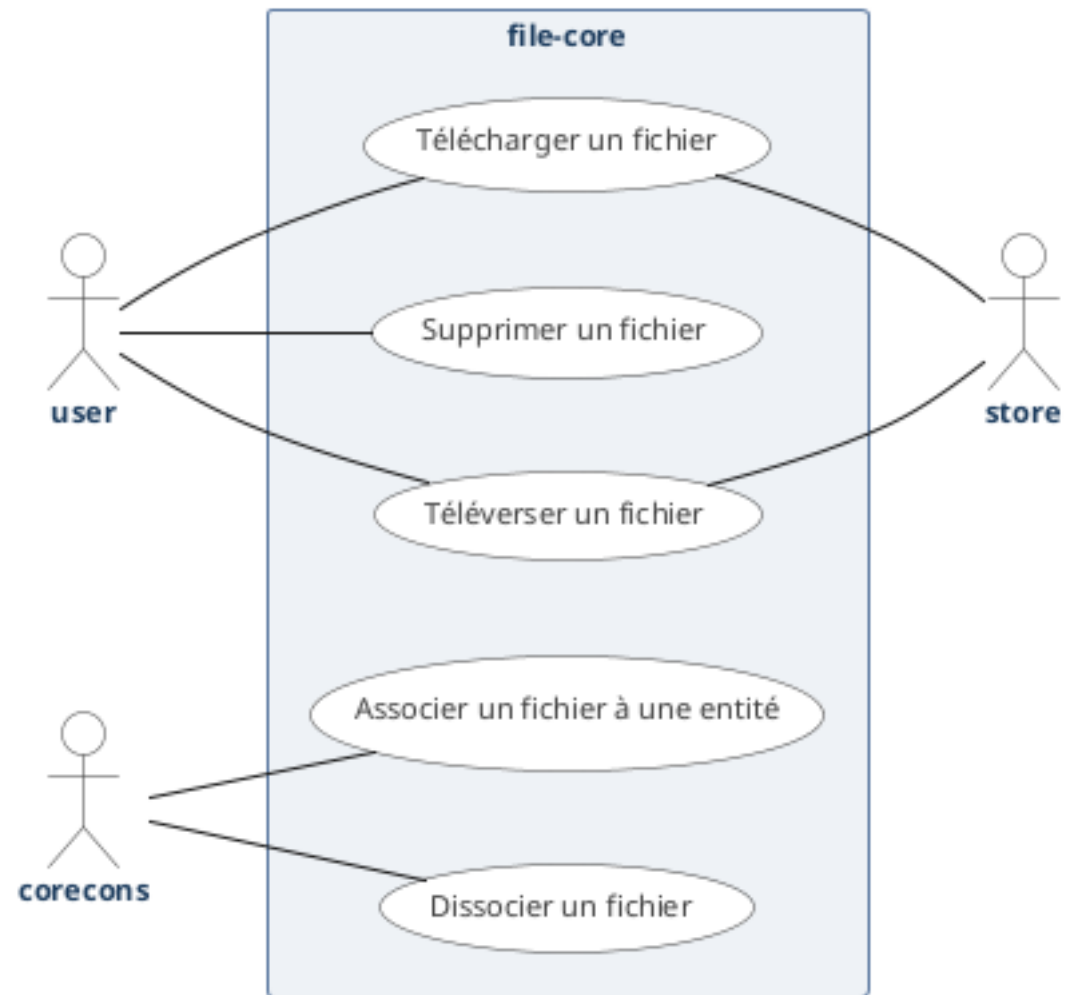
Diagramme de classes — file-core



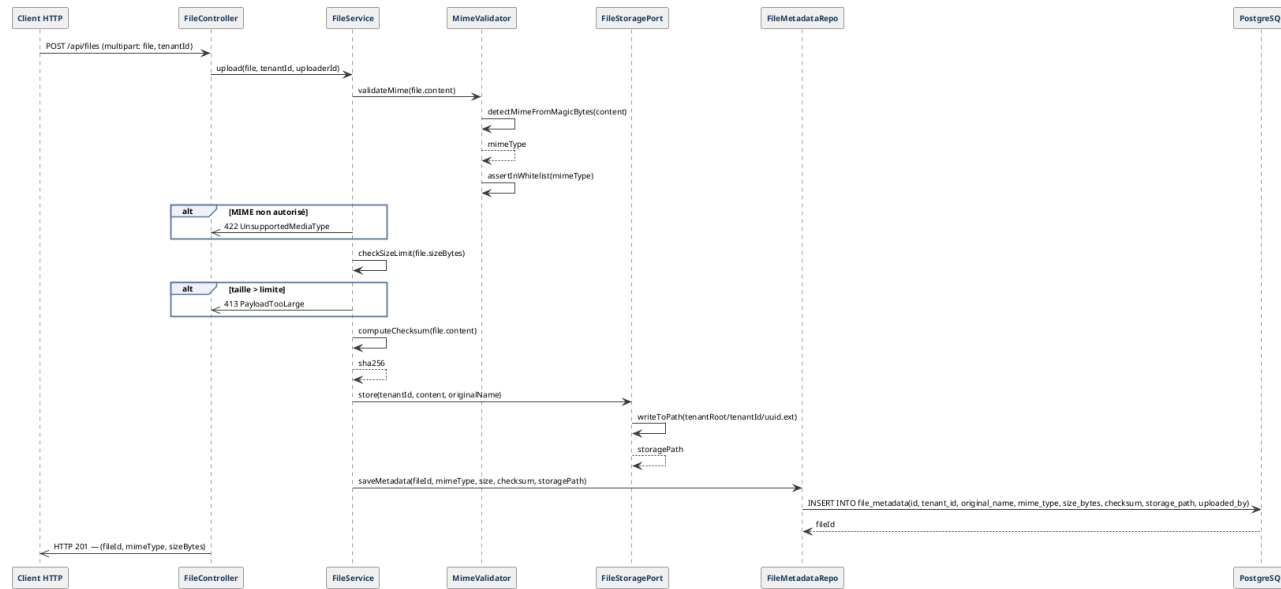
Invariants du file-core :

- Le type MIME est validé sur le contenu réel (magic bytes), pas sur l'extension.
- Le chemin de stockage est confiné à la racine tenant (pas de traversée de répertoire).
- Un fichier marqué `isDeleted` n'est plus accessible via l'API.
- Le checksum (SHA-256) garantit l'intégrité du fichier stocké.
- La suppression physique est bloquée si le fichier est encore référencé.

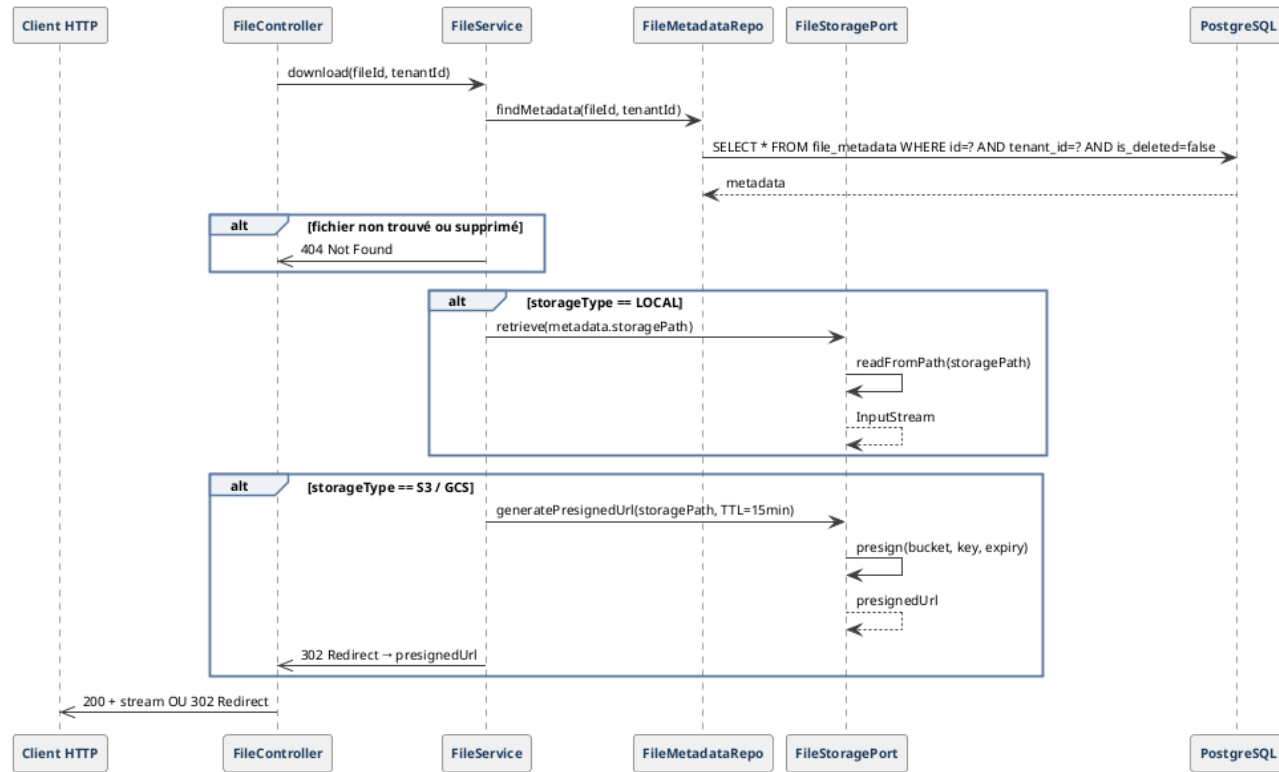
Diagramme des cas d'utilisation — file-core



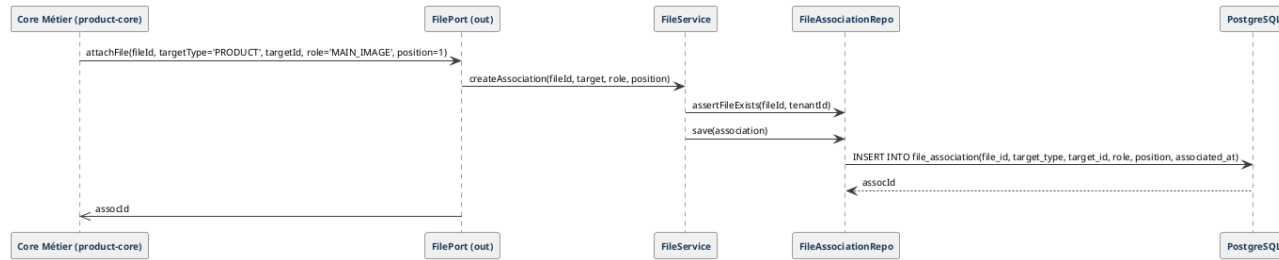
DS-F-01 — Téléversement d'un fichier avec validation MIME et checksum



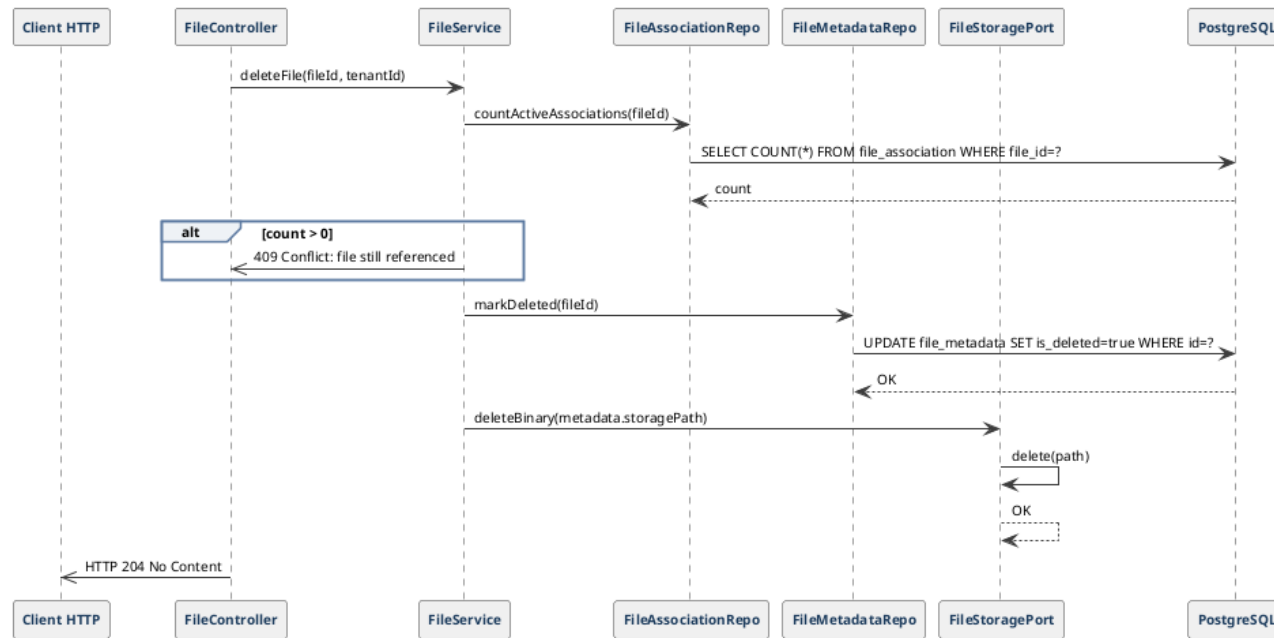
DS-F-02 — Téléchargement d'un fichier (direct ou URL signée)



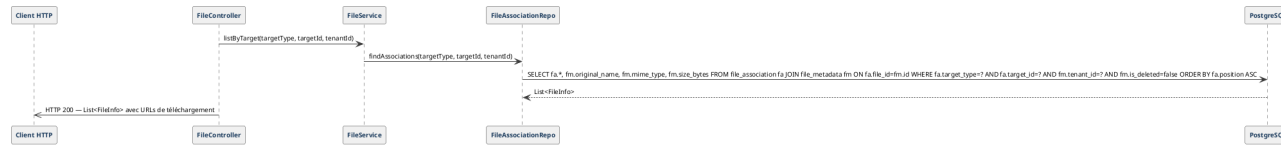
DS-F-03 — Association d'un fichier à une entité métier



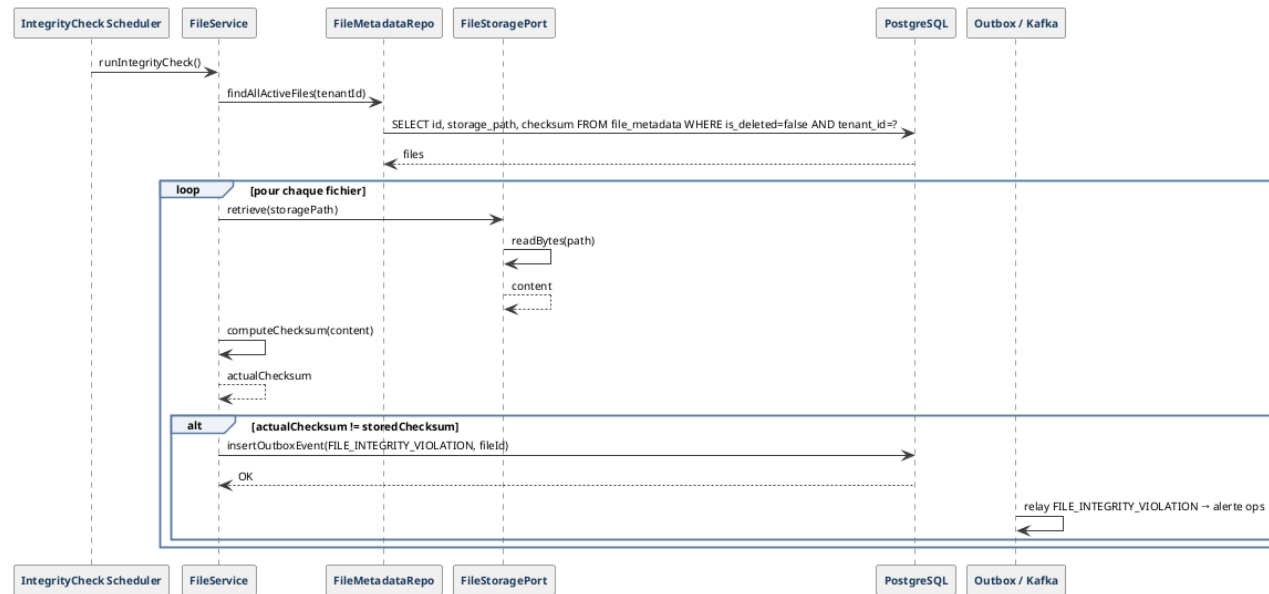
DS-F-04 — Suppression sécurisée d'un fichier avec vérification des références



DS-F-05 — Listage des fichiers associés à une entité



DS-F-06 — Vérification périodique d'intégrité des fichiers stockés



Conclusion Générale

Synthèse de l'architecture

Ce cahier d'analyse a décrit l'intégralité des seize cores fonctionnels constituant la plateforme **RT-Comops Backend**. L'ensemble forme un **monolithe modulaire à architecture hexagonale**, conçu pour répondre aux exigences de maintenabilité, de sécurité et d'évolutivité d'un système multi-tenant industriel.

Chaque core respecte les mêmes conventions structurelles :

- une **couche domaine** indépendante de toute infrastructure, exposant ses contrats via des interfaces entrantes (*port/in*) et sortantes (*port/out*) ;
- des **adaptateurs** qui implémentent ces interfaces sans polluer la logique métier (contrôleurs WebFlux, repositories R2DBC, clients Kafka/Redis/Elasticsearch) ;
- un **cloisonnement strict** par tenant, garanti par le `TenantRoutingConnectionFactory` et la résolution dynamique des schémas PostgreSQL.

Cohérence inter-cores

La communication entre cores repose sur deux mécanismes complémentaires, décrits en introduction et mis en œuvre de manière uniforme dans l'ensemble des modules :

1. **Appel synchrone par port/out** — utilisé pour les dépendances fonctionnelles directes (résolution d'un `PartyRef` par `tp-core`, vérification des droits par `roles-core`, lecture du catalogue par `inventory-core`). Le couplage reste minimal : seuls des identifiants ou des instantanés dénormalisés transitent entre cores, jamais d'objets domaine vivants.
2. **Outbox transactionnel + Kafka** — utilisé pour la propagation d'événements asynchrones (provisionnement de tenant, création de compte, déclenchement de facturation, alertes de stock). L'écriture dans `kernel.outbox_event` au sein de la même transaction que l'agrégat garantit l'atomicité ; le relais de fond assure la livraison vers le topic `iwm.events.business`.

Couverture fonctionnelle

Domaine	Cores	Rôle clé
Transverse	kernel-core, common-core	Cycle de vie tenant, outbox, types partagés
Identité & Accès	actor-core, auth-core, roles-core	Acteurs, JWT, RBAC
Organisation	organization-core, tp-core, settings-core	Structures, tiers, paramétrage
Opérations	product-core, inventory-core, resource-core, sales-core	Catalogue, stocks, ressources, commandes
Finance	accounting-core, treasury-core	Comptabilité, trésorerie
Services	administration-core, file-core	Gouvernance, fichiers

Invariants transverses

Tous les cores partagent un ensemble d'invariants non négociables, hérités de l'architecture globale :

- **Isolation tenant systématique** — toute entité persistée porte un `tenantId`. Aucun accès cross-tenant n'est possible par construction, y compris via les tables polymorphiques du `common-core`.
- **Traçabilité immuable** — chaque opération significative produit une entrée dans `AuditTrail` (`kernel-core`) en insertion seule, sans possibilité de modification ni de suppression.
- **Cohérence événementielle garantie** — le patron outbox transactionnel élimine le risque de perte d'événement en cas d'échec entre l'écriture de l'agrégat et la publication Kafka.
- **Sécurité par défaut** — validation MIME sur contenu réel (`file-core`), rotation obligatoire des refresh tokens (`auth-core`), vérification de la liste noire JWT à chaque requête (`kernel-core`), contrôle des encours crédit (`actor-core`).